

Matr. N INLM03/23286



UNIVERSITÀ CAMPUS BIO-MEDICO DI ROMA

**FACOLTÀ DIPARTIMENTALE DI INGEGNERIA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DEI SISTEMI
INTELLIGENTI**

**IMPLEMENTAZIONE E
OTTIMIZZAZIONE DELL'ALGORITMO
DI SHOR IN AMBIENTI QUANTISTICI
RUMOROSI MEDIANTE MODELLI DI
INTELLIGENZA ARTIFICIALE**

Relatore

Prof. Paolo Soda

Laureando

Claudio Dragotta

Correlatore

Ing. Floriano Caprio

ANNO ACCADEMICO 2025/2026

*Alla mia famiglia,
per il sostegno incondizionato e l'amore che non è mai venuto meno.
A me stesso, per la determinazione e la perseveranza con cui ho affrontato questo
percorso.*

Indice

Elenco delle figure	vi
Elenco delle tabelle	viii
Elenco degli Acronimi	xi
1 Introduzione	2
1.1 Il Limite del Calcolo Classico	2
1.2 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico	3
1.2.1 Il Qubit e la Sovrapposizione	3
1.2.2 La Sfera di Bloch	4
1.2.3 L'Entanglement	4
1.2.4 L'Interferenza Quantistica	5
1.2.5 Implementazione Fisica: i Qubit Superconduttori di IBM	5
1.3 La Rivoluzione Algoritmica degli Anni Novanta	6
1.4 RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor	7
1.4.1 Struttura del Protocollo RSA	7
1.4.2 Perché l'Algoritmo di Shor Rompe RSA	8
1.4.3 Implicazioni Pratiche per la Sicurezza Informatica	9
1.5 Crittografia Post-Quantistica: la Risposta dell'Industria	10
1.5.1 Gli Standard NIST 2024	10
1.5.2 La Sfida della Migrazione	11
1.6 Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant	12
1.7 Motivazione e Contributo del Presente Lavoro	14
1.8 Struttura della Tesi	16
2 Obiettivi e Piano Sperimentale	19
2.1 Motivazione: la Vulnerabilità della Crittografia Attuale	19

2.2	Obiettivi Generali	21
2.3	Obiettivi Specifici	21
2.4	Ipotesi di Lavoro	22
2.5	Contributo Originale	23
2.6	Requisiti Funzionali	23
2.7	Parametri di Input e Output	25
2.7.1	Parametri di Input	25
2.7.2	Output del Sistema	25
2.8	I Quattro Use Case	26
2.8.1	Parametri dei Use Case	26
2.8.2	Razionale per la Scelta delle Istanze	27
2.9	Metriche di Valutazione	28
2.9.1	Metriche del Metodo 1	28
2.9.2	Metriche del Metodo 2	28
2.9.3	Metrica di Confronto	29
2.9.4	Parametri di Error Mitigation Applicati	29
2.10	Approccio Metodologico	29
3	Fondamenti del Calcolo Quantistico	30
3.1	Dalla Meccanica Classica alla Meccanica Quantistica	30
3.2	I Postulati della Meccanica Quantistica	31
3.3	Il Qubit	33
3.3.1	Definizione Matematica	33
3.3.2	La Sfera di Bloch	33
3.4	Sistemi Multi-Qubit e Prodotto Tensoriale	34
3.4.1	Entanglement	34
3.4.2	L'Entanglement come Risorsa Fisica Fragile	35
3.5	Porte Quantistiche	36
3.5.1	Porte a Singolo Qubit	36
3.5.2	Porte a Due Qubit	38
3.6	Circuiti Quantistici	39
3.7	Misura Quantistica	39
3.8	Interferenza Quantistica	39
3.9	Modello di Complessità Quantistica	40

4	L'Algoritmo di Shor	41
4.1	Il Problema della Fattorizzazione Intera	42
4.1.1	Rilevanza Crittografica: RSA	42
4.2	Riduzione alla Ricerca del Periodo	43
4.3	La Quantum Fourier Transform	44
4.3.1	Definizione	44
4.3.2	Implementazione Efficiente	45
4.3.3	Phase Estimation	45
4.4	L'Algoritmo di Shor: Schema Completo	48
4.4.1	Il Ruolo dell'Entanglement nel Period Finding	49
4.5	Analisi della Complessità	51
4.6	Impatto sulla Crittografia e Prospettive Post-Quantum	51
4.7	Sviluppi Recenti e Stato dell'Arte	52
4.7.1	Implementazioni su Hardware Reale	52
4.7.2	Miglioramenti Algoritmici: Regev 2023	53
4.7.3	Roadmap IBM verso il Quantum Fault-Tolerant	53
4.8	Contesto: Confronto con l'Algoritmo di Grover	54
4.8.1	Il Problema di Grover	54
4.8.2	Differenze Fondamentali	54
5	Rumore Quantistico nei Sistemi NISQ	56
5.1	Introduzione al Rumore Quantistico	56
5.2	L'Anatomia Fisica di una QPU	57
5.3	Le Quattro Fonti Fisiche di Rumore	62
5.3.1	Decoerenza (T_1 e T_2)	62
5.3.2	Errori di Gate	63
5.3.3	Errori di Misura (Readout Errors)	63
5.3.4	Crosstalk	64
5.4	Dal Fenomeno Fisico al Modello Matematico	64
5.5	I Quattro Canali Matematici del Rumore	66
5.5.1	Canale Bit Flip	66
5.5.2	Canale Phase Flip	66
5.5.3	Canale Depolarizzante	67
5.5.4	Canale di Amplitude Damping	67
5.6	Impatto del Rumore sull'Algoritmo di Shor	69
5.6.1	Errori nella Quantum Fourier Transform	69

5.6.2	Errori nella Phase Estimation	70
5.6.3	Errori nella Modular Exponentiation	70
5.7	Sfide Pratiche su Hardware NISQ	71
6	Strategie per la Riduzione del Rumore Quantistico	72
6.1	Le Quattro Strategie: Panoramica e Scelta	72
6.2	Quantum Machine Learning per la Robustezza al Rumore	74
6.3	La Correzione degli Errori Quantistici come Sottocategoria del QML	75
6.3.1	Il Problema Fondamentale	75
6.3.2	Il Paradigma in Tre Fasi	75
6.3.3	I Principali Codici Quantistici	76
6.3.4	Perché la QEC è una Sottocategoria del QML	76
6.4	Motivazione Pratica: il Contributo di questa Tesi	77
7	Strumenti e Ambiente di Simulazione	79
7.1	Qiskit: il Framework Quantistico di Riferimento	79
7.1.1	Architettura del Framework	79
7.1.2	Perché Qiskit e non un'alternativa	80
7.2	Qiskit Aer: il Simulatore	81
7.2.1	Metodi di Simulazione	81
7.2.2	Modelli di Rumore	82
7.2.3	Transpilazione	82
7.3	Accelerazione GPU con Qiskit Aer	83
7.4	scikit-learn: la Libreria per il Classificatore	83
7.5	Ambiente di Esecuzione: WSL con Ubuntu	84
8	Metodologia e Architettura	85
8.1	Obiettivi della Parte Sperimentale	85
8.2	Architettura del Pipeline Sperimentale	86
8.3	Tecniche di Mitigazione del Rumore: Analisi e Scelte Implementative	87
8.3.1	Livello di Circuito: Transpilazione e Riduzione della Profondità	87
8.3.2	Livello di Misura: Mitigazione degli Errori di Readout	88
8.3.3	Livello di Post-processing: Zero-Noise Extrapolation	89
8.3.4	Analisi di Sensibilità ai Parametri di Rumore	89
8.3.5	Scelta Adottata: Classificatore ML come Filtro Adattivo	92
8.4	Il Metodo 1: Approccio Statistico Classico	92

8.5	Il Metodo 2: Classificatore ML	93
9	Sviluppo e Implementazione	95
9.1	Setup dell'Ambiente di Esecuzione	95
9.1.1	Dipendenze e Virtual Environment	95
9.1.2	Accelerazione GPU: Tentativo e Limitazione WSL	96
9.2	L'Algoritmo di Shor: Implementazione di Riferimento	97
9.2.1	La Quantum Fourier Transform	97
9.2.2	La Moltiplicazione Modulare Controllata	97
9.2.3	Circuito Completo e Post-Processing	98
9.3	Configurazione del Noise Model	99
9.4	Implementazione del Metodo 1	100
9.5	Implementazione del Metodo 2	102
9.5.1	Generazione del Dataset e Definizione dell'Etichetta	102
9.5.2	Addestramento e Selezione del Modello	104
9.5.3	Inferenza: Classificatore + Ricerca TOP-K	104
9.6	Infrastruttura di Test e Raccolta Dati	106
10	Verifica Sperimentale dell'Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K	107
10.1	Setup Sperimentale	107
10.2	Analisi del Circuito e Barriera di Scalabilità	108
10.3	La Baseline: Metodo 1 (TOP-1)	110
10.4	L'Ipotesi alla Prova: il Classificatore ML	111
10.5	Il Verdetto dell'Ablazione	112
10.6	Robustezza della Strategia TOP-4	114
10.7	Bilancio della Verifica e Ponte verso il Nucleo della Tesi	115
11	La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane	116
11.1	Dalla Ridondanza Ingenua alla Codifica	116
11.2	Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder	117
11.3	Il Panorama dei Codici e la Scelta Metodologica	119
11.4	Laboratorio I: il Codice a Ripetizione	119
11.4.1	Il codice bit-flip a 3 qubit	119
11.4.2	Risultati del laboratorio repetition	120
11.5	Laboratorio II: il Codice di Steane $[[7, 1, 3]]$	122

11.5.1	Struttura del codice	122
11.5.2	Protocollo di laboratorio	122
11.5.3	Risultati del laboratorio Steane	123
12	Il Surface Code e la Stima dell'Errore Logico	126
12.1	Il Codice su Griglia: Concetto Operativo	126
12.2	La Decodifica: Minimum Weight Perfect Matching	127
12.3	Disegno Sperimentale	128
12.4	Risultati	129
12.5	Oltre il Modello di Pauli: Errori Coerenti e Leakage	131
13	Integrazione: lo Shor Logico e i Requisiti di Correzione	134
13.1	L'Architettura a Quattro Livelli	134
13.2	Il Modello di Shor Logico	135
13.3	Il Contesto: le Stime di Risorse per Shor Fault-Tolerant	136
13.4	Risultati del Confronto	137
13.4.1	Discussione	140
14	Sviluppi Futuri	142
14.1	Scalabilità a Istanze di Maggiore Dimensione	143
14.2	Test e Validazione su Hardware Quantistico Reale	144
14.3	Generalizzazione ad Altri Algoritmi Quantistici NISQ	145
14.4	Integrazione con la Quantum Error Correction Strutturale	146
14.5	Evoluzione del Classificatore ML	147
A	Campagna Parametrica Dettagliata e Confronto ZNE	148
A.1	Variazione di K (numero di candidati per iterazione)	148
A.2	Variazione di ε_{2q} (tasso di errore a due qubit)	149
A.3	Variazione di M_{shots} (shot per iterazione)	150
A.4	Analisi Congiunta $K \times \varepsilon_{2q}$	150
A.5	Parametri Secondari: T_1/T_2 , ε_{1q} e p_{ro}	151
A.6	Dettaglio del Confronto con Zero-Noise Extrapolation	152
	Bibliografia	154

Elenco delle figure

3.1	La sfera di Bloch	34
3.2	Circuito per lo stato di Bell $ \Phi^+\rangle$	35
3.3	Porte quantistiche a singolo e due qubit	38
3.4	Gerarchia delle classi di complessità quantistica	40
4.1	Periodicità di $f(x) = 7^x \bmod 15$	43
4.2	Circuito della QFT su 3 qubit	45
4.3	Circuito QPE per $N = 15$, $a = 2$	46
4.4	Blocco di moltiplicazione modulare U_2 per $N = 15$	46
4.5	Blocco di moltiplicazione modulare U_4 per $N = 15$	47
4.6	Istogramma QPE ideale per $N = 15$	48
4.7	Schema a due registri dell'algoritmo di Shor	49
5.1	Anatomia di una QPU superconduttiva a 5 qubit (topologia a catena)	57
5.2	La QPU reale: sistema integrato e criostati con cablaggio	58
5.3	Chip quantistico superconduttivo a 6 transmon con giunzioni di test .	59
5.4	Il bordo di coerenza: budget di operazioni e rischio di misura	61
5.5	Effetto dei canali di rumore sulla sfera di Bloch	68
5.6	Dove colpiscono le quattro fonti di rumore nella pipeline di Shor . . .	69
5.7	Probabilità di successo di Shor in presenza di rumore	70
8.1	Pipeline sperimentale	86
10.1	Circuito QPE per $N = 21$	109
10.2	Istogramma QPE per $N = 21$ — distribuzione piatta	109
10.3	Confronto istogrammi QPE con segnale strutturato e dominato dal rumore (UC1)	110
10.4	Curve ROC dei classificatori selezionati (UC1: Random Forest, UC2: SVM)	112

10.5	Analisi di ablazione: M1, M_{TOP4} e M2 su UC1 ($K = 30$)	113
11.1	Errore logico del codice a ripetizione a 3 qubit	121
11.2	Errore logico del codice di Steane $[[7, 1, 3]$	125
12.1	Comportamento a soglia del surface code	130
12.2	Errore coerente vs errore di Pauli	132
13.1	Figura conclusiva: Shor fisico vs Steane vs surface code	138
13.2	Probabilità di successo cumulativa di Shor dopo k esecuzioni	139
14.1	Simulation gap: simulatore vs hardware reale per $N = 15$	144

Elenco delle tabelle

2.1	Parametri di input del sistema sperimentale. I valori di ϵ_{1q} , ϵ_{2q} , T_1 , T_2 e p_{ro} definiscono il noise model e vengono variati tra i use case per coprire scenari di rumore diversi.	25
2.2	Configurazione completa dei quattro use case. I parametri di rumore del livello NISQ-realistico sono derivati dalle specifiche di <code>ibm_marrakesh</code> [23, 21]. Il livello NISQ-degradato scala i tassi di errore di $5\times$ e dimezza T_1 e T_2	27
4.1	Confronto tra l'algoritmo di Shor e l'algoritmo di Grover	54
5.1	Confronto tra tecnologie di QPU: superconduttori vs ioni intrappolati	60
5.2	Riepilogo delle quattro fonti fisiche di rumore nei processori quantistici superconduttori.	64
5.3	Corrispondenza tra fonti fisiche di rumore e canali quantistici.	64
5.4	I quattro canali matematici fondamentali del rumore quantistico [26].	68
6.1	Confronto delle quattro strategie anti-rumore per il contesto di questa tesi.	73
7.1	Confronto tra i principali framework per la simulazione quantistica con rumore.	81
8.1	Risultati dell'analisi di sensibilità ai parametri di rumore per $N=15$, $a=7$, $n_count=8$. Solo ϵ_{2q} influenza le prestazioni di M1.	90
8.2	Sweep di ϵ_{2q} su UC1 ($N=15$, $a=7$, $K=30$). $\bar{M}_1$ cresce con il tasso di errore CX; la strategia TOP-4 mantiene success rate = 100% su tutto il range.	90

8.3	Sweep di <code>optimization_level</code> su UC1 ($N=15$, $a=7$, $K=30$). Il passaggio da livello 0 a 1 riduce i gate CX del 3.6% ($168 \rightarrow 162$); i livelli 2 e 3 non apportano ulteriori riduzioni. Il success rate rimane 100% a tutti i livelli.	91
9.1	Selezione del classificatore per i due use case (test set 20%, $n = 400$ campioni)	104
10.1	Caratteristiche del circuito traspilato e applicabilità dei metodi per ciascun use case	108
10.2	Risultati del Metodo 1 sui quattro use case	110
10.3	Metriche dei classificatori sul test set (20% del dataset, etichetta TOP_K=16)	111
10.4	Riepilogo comparativo M1, M_{TOP4} e M2 — risultato principale con analisi di ablazione ($K = 30$, $M_{\text{max}} = 50$)	112
10.5	Sintesi della campagna parametrica su M_{TOP4} : impatto atteso, osservato e raccomandazioni	114
10.6	Confronto M1, ZNE-2, ZNE-3 e M_{TOP4} su UC1 ($K_{\text{rep}} = 30$, $M_{\text{shots}} = 1024$ per livello)	114
11.1	Confronto tra codici correttori quantistici	119
11.2	Verifica della sindrome del codice a ripetizione	120
11.3	Parametri del codice di Steane	122
11.4	Syndrome table del codice di Steane	124
12.1	Disegno sperimentale surface code	129
13.1	Architettura a quattro livelli della parte sperimentale	135
A.1	Effetto del parametro K su $\bar{M}$, tasso di successo e ρ vs M1 ($N = 15$, UC1, $K_{\text{rep}} = 30$)	149
A.2	Effetto di ε_{2q} su M_{TOP4} ($K = 4$, $K_{\text{rep}} = 30$, altri parametri fissi a UC1)	149
A.3	Effetto di M_{shots} su M_{TOP4} ($K = 4$, $K_{\text{rep}} = 30$, UC1)	150
A.4	$\bar{M}_{\text{TOPK}}$ per combinazioni di K e ε_{2q} ($K_{\text{rep}} = 30$). In grassetto: $\rho > 2$ con $p < 0.05$	151
A.5	Effetto di T_1 su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$, $T_2 = 0.8 T_1$)	151
A.6	Effetto di ε_{1q} su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$)	151
A.7	Effetto di p_{ro} su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$)	152

Elenco degli Acronimi

AUC Area Sotto la Curva ROC (*Area Under the Curve*)

BQP *Bounded-error Quantum Polynomial time*

CCNOT Controlled-Controlled NOT (porta di Toffoli)

CNOT Controlled-NOT

CPTP *Completely Positive Trace-Preserving*

CPU Unità di Elaborazione Centrale (*Central Processing Unit*)

CSS Calderbank-Shor-Steane

CX Controlled-X (porta CNOT)

DFT Trasformata Discreta di Fourier (*Discrete Fourier Transform*)

DSA *Digital Signature Algorithm*

ECDSA *Elliptic Curve Digital Signature Algorithm*

GCD Massimo Comune Divisore (*Greatest Common Divisor*)

GNFS *General Number Field Sieve*

GPU Unità di Elaborazione Grafica (*Graphics Processing Unit*)

HTTPS *HyperText Transfer Protocol Secure*

IBM International Business Machines

MIT Massachusetts Institute of Technology

ML Apprendimento Automatico (*Machine Learning*)

MLP *Percetttrone Multistrato (Multi-Layer Perceptron)*

MWPM *Minimum Weight Perfect Matching*

NISQ *Noisy Intermediate-Scale Quantum*

NIST *National Institute of Standards and Technology*

NMR *Risonanza Magnetica Nucleare (Nuclear Magnetic Resonance)*

PEC *Probabilistic Error Cancellation*

PQC *Post-Quantum Cryptography*

QEC *Correzione degli Errori Quantistici (Quantum Error Correction)*

QFT *Trasformata di Fourier Quantistica (Quantum Fourier Transform)*

QMA *Quantum Merlin-Arthur*

QML *Quantum Machine Learning*

QPE *Quantum Phase Estimation*

QPU *Unità di Elaborazione Quantistica (Quantum Processing Unit)*

RSA *Rivest-Shamir-Adleman*

SDK *Software Development Kit*

SSH *Secure Shell*

SVM *Support Vector Machine*

TLS *Transport Layer Security*

WSL *Windows Subsystem for Linux*

ZNE *Zero-Noise Extrapolation*

Capitolo 1

Introduzione

1.1 Il Limite del Calcolo Classico

Per oltre cinquant'anni, lo sviluppo dell'informatica è stato guidato da una legge empirica enunciata nel 1965 da Gordon Moore¹: il numero di transistor per chip raddoppia ogni due anni, e con esso la potenza di calcolo disponibile. Questa crescita esponenziale ha permesso di trasformare calcolatori grandi quanto un'intera stanza nei dispositivi tascabili di oggi. Tuttavia, la Legge di Moore si sta scontrando con i limiti fisici fondamentali della materia: i transistor moderni hanno dimensioni di pochi nanometri, avvicinandosi alla scala atomica oltre la quale gli effetti quantistici – lo stesso fenomeno che si vorrebbe evitare – diventano incontrollabili e inevitabili.

Parallelamente al problema fisico dell'hardware, esiste una barriera di natura puramente matematica: esistono classi di problemi computazionali per i quali nessun algoritmo classico è in grado di fornire una soluzione in tempi ragionevoli, *indipendentemente* dalla potenza di calcolo disponibile. Questi problemi – fattorizzazione di grandi interi, ricerca in spazi combinatoriali enormi, simulazione di sistemi molecolari complessi – resistono a qualunque approccio classico non perché i computer non siano abbastanza veloci, ma perché la complessità intrinseca del problema cresce più rapidamente di qualunque risorsa classica concepibile.

È in questo contesto che nasce il **calcolo quantistico**: la visione radicale che i principi della meccanica quantistica – *sovrapposizione*, *entanglement* e *interferenza*

¹**Gordon Earle Moore** (1929–2023), cofondatore di Intel, osservò nel 1965 che il numero di transistor integrabili su un chip raddoppiava approssimativamente ogni due anni, con un conseguente aumento esponenziale delle prestazioni computazionali. La Legge di Moore non è una legge fisica, ma una previsione industriale che ha orientato decenni di investimenti tecnologici.

– possano essere sfruttati non come ostacoli da evitare, ma come risorse computazionali di straordinaria potenza. L'intuizione originale fu di Richard Feynman², che nel 1982 osservò come la simulazione di sistemi quantistici richiedesse risorse esponenziali su macchine classiche, e propose che un computer basato sugli stessi principi fisici potesse simulare la natura stessa in modo efficiente [1]. David Deutsch formalizzò nel 1985 il primo modello teorico rigoroso di computer quantistico universale [2], aprendo la strada a un nuovo paradigma computazionale.

1.2 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico

Per comprendere appieno le potenzialità e i limiti del calcolo quantistico, è necessario esaminare le differenze fisiche fondamentali tra l'unità di informazione classica e quella quantistica.

1.2.1 Il Qubit e la Sovrapposizione

Un computer classico elabora l'informazione attraverso **bit** – unità fisiche che possono trovarsi in uno di due stati discreti, convenzionalmente 0 e 1, realizzati come livelli di tensione in circuiti elettronici. L'unità fondamentale del calcolo quantistico è invece il **qubit** (*quantum bit*): un sistema fisico a due livelli che obbedisce alle leggi della meccanica quantistica.

Matematicamente, lo stato di un qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H} \cong \mathbb{C}^2$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (1.1)$$

dove $|0\rangle$ e $|1\rangle$ sono i due stati della base computazionale, analoghi ai valori 0 e 1 del bit classico. La differenza fondamentale è che α e β possono essere simultaneamente diversi da zero: il qubit si trova in una **sovrapposizione** dei due stati base, con probabilità $|\alpha|^2$ di essere misurato in $|0\rangle$ e probabilità $|\beta|^2$ di essere misurato in $|1\rangle$.

²**Richard Phillips Feynman** (1918–1988), fisico teorico statunitense, Premio Nobel per la Fisica nel 1965. Nel suo celebre articolo del 1982 “*Simulating Physics with Computers*” [1], Feynman argomentò che un computer classico non può simulare efficientemente un sistema quantistico senza un overhead esponenziale, e propose che un “computer quantistico” – uno che segua le leggi della meccanica quantistica – potrebbe farlo in modo naturalmente efficiente.

È cruciale sottolineare che la sovrapposizione non equivale a ignoranza sullo stato del sistema: il qubit non è “segretamente” in uno stato definito che non conosciamo. Si tratta di una proprietà fisica genuina, verificata sperimentalmente con precisione straordinaria, che non ha analogo nella fisica classica.

1.2.2 La Sfera di Bloch

Poiché la norma è unitaria e la fase globale è fisicamente irrilevante, lo stato di un qubit puro può essere parametrizzato da soli due angoli reali:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi) \quad (1.2)$$

Questo definisce un punto sulla superficie di una sfera unitaria – la **sfera di Bloch** – dove il polo nord corrisponde a $|0\rangle$, il polo sud a $|1\rangle$, e i punti equatoriali a sovrapposizioni equiprobabili con fasi diverse. Questa rappresentazione geometrica è di grande utilità pratica: le operazioni su un singolo qubit corrispondono a **rotazioni rigide** della sfera di Bloch, e il rumore – come si vedrà nel Capitolo 5 – corrisponde alla contrazione progressiva del vettore di Bloch verso il centro, fino a ridurlo a zero (stato completamente misto, massima incertezza).

1.2.3 L’Entanglement

Quando due o più qubit interagiscono tramite porte quantistiche opportune, possono formare stati **entangled**: stati del sistema composto che non possono essere scritti come prodotto tensoriale di stati individuali. Il prototipo è lo stato di Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (1.3)$$

In questo stato, misurare il primo qubit in $|0\rangle$ collassa istantaneamente il secondo in $|0\rangle$, e viceversa per $|1\rangle$, indipendentemente dalla distanza fisica tra i due qubit. L’entanglement crea correlazioni impossibili da replicare con qualunque modello classico a variabili nascoste locali, come dimostrato teoricamente da Bell nel 1964 [3] e verificato sperimentalmente con crescente precisione negli anni successivi [4]. Nel contesto del calcolo quantistico, l’entanglement è la risorsa che permette agli algoritmi di elaborare esponenzialmente più informazione rispetto ai circuiti classici: un registro di n qubit entangled descrive uno stato nello spazio $\mathbb{C}^{2^n}$, inaccessibile a qualunque rappresentazione classica di dimensione polinomiale in n .

1.2.4 L'Interferenza Quantistica

Il terzo pilastro del calcolo quantistico è l'**interferenza**: la capacità di far sommare costruttivamente o annullare distruttivamente le ampiezze di probabilità di percorsi computazionali diversi. Le ampiezze α e β in (1.1) sono numeri complessi, e possono quindi avere segni opposti o sfasamenti arbitrari: quando si sommano, possono amplificarsi o cancellarsi esattamente come onde fisiche.

Un algoritmo quantistico ben progettato sfrutta sistematicamente questo meccanismo: attraverso una sequenza opportuna di porte quantistiche, si costruisce un'interferenza *costruttiva* sui percorsi che portano alla risposta corretta e *distruttiva* su quelli errati, concentrando la probabilità di misura sull'output desiderato. È esattamente questo il meccanismo alla base della ricerca quadraticamente accelerata di Grover e della fattorizzazione polinomiale di Shor.

1.2.5 Implementazione Fisica: i Qubit Superconduttori di IBM

Un qubit può essere realizzato con qualunque sistema fisico a due livelli: la polarizzazione di un fotone, lo spin di un elettrone, i livelli energetici di un atomo intrappolato. I **qubit superconduttori** utilizzati da IBM – la piattaforma adottata nel presente lavoro – sono circuiti elettrici macroscopici che operano nel regime quantistico grazie al raffreddamento a temperature criogeniche estreme.

Il componente chiave è la **giunzione di Josephson** [5]: una sottile barriera isolante (spessa $\sim 1\text{--}2\text{ nm}$) interposta tra due materiali superconduttori. Questo elemento introduce una non-linearità nell'energia del circuito che rompe la degenerazione dei livelli energetici, isolando i due stati computazionali $|0\rangle$ e $|1\rangle$ da tutti gli altri. Il risultato è il **transmon** [6] – il tipo di qubit superconduttore adottato da IBM – un oscillatore anelastico con frequenza tipica 4–6 GHz, controllato da impulsi a microonde.

Per operare nel regime quantistico, questi circuiti devono essere raffreddati a temperature dell'ordine di 15 mK (-273.13 C), più fredde dello spazio cosmico a 2.7 K, tramite refrigeratori a diluizione. A queste temperature la resistenza elettrica scompare per effetto della superconduttività, eliminando la dissipazione termica che causerebbe la distruzione dello stato quantistico.

I parametri operativi tipici dei processori IBM (serie Eagle, Heron) sono:

- **Temperatura operativa:** ~ 15 mK
- **Tempi di coerenza:** $T_1, T_2 \sim 50\text{--}500 \mu\text{s}$
- **Durata gate a singolo qubit:** ~ 50 ns
- **Durata gate a due qubit (ECR/CNOT):** $\sim 300\text{--}500$ ns
- **Fedeltà gate a singolo qubit:** $\sim 99.9\text{--}99.97\%$
- **Fedeltà gate a due qubit:** $\sim 99\text{--}99.7\%$

Il rapporto tra il tempo di coerenza e la durata di un gate determina quante operazioni possono essere eseguite prima che il qubit perda il proprio stato quantistico – ed è proprio questo rapporto il principale collo di bottiglia per l'esecuzione di algoritmi profondi come Shor sui processori attuali.

1.3 La Rivoluzione Algoritmica degli Anni Novanta

Negli anni successivi alla proposta di Deutsch, la comunità scientifica si interrogò sulla effettiva esistenza di problemi che un computer quantistico potesse risolvere in modo *probabilmente* più efficiente di qualunque macchina classica. La risposta arrivò in forma dirompente nel corso di pochi anni.

Nel 1992, Daniel Simon³ identificò un problema artificiale per il quale un algoritmo quantistico era esponenzialmente più veloce di qualunque algoritmo classico. Questo risultato rimase per alcuni anni una curiosità teorica, fino a quando, nel **1994**, Peter Shor pubblicò un risultato che scosse le fondamenta della sicurezza informatica mondiale: un algoritmo quantistico in grado di fattorizzare un intero di n bit in tempo *polinomiale* $O(n^3)$ [7], a fronte della complessità sub-esponenziale ma superpolinomiale dei migliori algoritmi classici noti. Le implicazioni furono immediate e devastanti: l'intero edificio della crittografia a chiave pubblica – Rivest-Shamir-Adleman (RSA), Diffie-Hellman, le firme digitali *Elliptic Curve Digital Signature*

³L'**algoritmo di Simon** (1994, pubblicato nel 1997) fu il primo a dimostrare uno speedup *esponenziale* rispetto a qualunque algoritmo classico probabilistico, stabilendo la prova di principio che i computer quantistici potessero essere genuinamente più potenti. Shor riconobbe che la sua tecnica si ispirava direttamente alle idee di Simon.

Algorithm (ECDSA) – si fondava proprio sull’assunzione che la fattorizzazione fosse computazionalmente intrattabile. Un computer quantistico sufficientemente grande avrebbe reso obsoleta, in un solo colpo, l’intera infrastruttura crittografica su cui poggia Internet.

Due anni dopo, nel **1996**, Lov Grover presentò un algoritmo di tutt’altra natura, ma di eguale importanza [8]. Mentre Shor aveva risolto un problema specifico della teoria dei numeri, Grover affrontò una sfida universale: la **ricerca non strutturata**. Dato un insieme di N elementi senza alcuna struttura o ordinamento, trovare quello che soddisfa una certa proprietà richiede, nel caso classico, un numero di interrogazioni proporzionale a N . Grover dimostrò che un algoritmo quantistico può fare lo stesso lavoro in sole $O(\sqrt{N})$ interrogazioni – uno *speedup quadratico* – e che questo risultato è *ottimale*: nessun algoritmo quantistico può fare di meglio [9]. Sebbene meno spettacolare dello speedup esponenziale di Shor, il vantaggio di Grover è universale, applicabile a qualunque problema di ricerca, e porta con sé implicazioni profonde per la crittografia simmetrica, l’ottimizzazione combinatoria e la ricerca in grandi basi di dati.

Questi due algoritmi – Shor e Grover – rappresentano ancora oggi i pilastri dell’informatica algoritmica quantistica: il primo come dimostrazione del potere *esponenziale* del calcolo quantistico su problemi strutturati, il secondo come strumento *universale* di accelerazione quadratica applicabile al problema computazionale più fondamentale.

1.4 RSA e le Implicazioni Crittografiche dell’Algoritmo di Shor

Per comprendere appieno la portata rivoluzionaria dell’algoritmo di Shor è necessario esaminare la struttura matematica del sistema crittografico che esso minaccia. Il sistema **RSA**, proposto nel 1977 da Rivest, Shamir e Adleman [10], è la pietra angolare della crittografia a chiave pubblica e il fondamento di praticamente ogni protocollo sicuro su Internet.

1.4.1 Struttura del Protocollo RSA

La sicurezza di RSA si basa sulla **asimmetria computazionale** tra due operazioni inverse: moltiplicare due numeri primi grandi p e q per ottenere $N = p \cdot q$ è un’operazione elementare (complessità $O(n^2)$ per numeri a n bit), mentre fattorizzare

N per recuperare p e q è computazionalmente intrattabile con gli algoritmi classici noti. La generazione delle chiavi segue i passi seguenti:

1. Scegliere due primi grandi p e q (tipicamente a 1024–2048 bit ciascuno) e calcolare $N = p \cdot q$;
2. Calcolare la funzione di Eulero $\phi(N) = (p-1)(q-1)$;
3. Scegliere e coprimo con $\phi(N)$ (convenzionalmente $e = 65537 = 2^{16} + 1$, primo di Fermat);
4. Calcolare $d \equiv e^{-1} \pmod{\phi(N)}$ tramite l'algoritmo di Euclide esteso.

La **chiave pubblica** è la coppia (N, e) ; la **chiave privata** è d . Cifrare un messaggio $m < N$ produce il crittogramma $c = m^e \bmod N$; decifrarlo richiede $m = c^d \bmod N$. La correttezza si fonda sul piccolo teorema di Fermat: $(m^e)^d = m^{ed} \equiv m \pmod{N}$ poiché $ed \equiv 1 \pmod{\phi(N)}$. Tutta la sicurezza risiede nell'impossibilità di calcolare d da (N, e) senza conoscere la fattorizzazione di N .

1.4.2 Perché l'Algoritmo di Shor Rompe RSA

Il miglior algoritmo classico per la fattorizzazione di un intero a n bit è il *General Number Field Sieve* (GNFS) [11], con complessità sub-esponenziale:

$$T_{\text{GNFS}}(N) = O(\exp[c \cdot (\log N)^{1/3} (\log \log N)^{2/3}]) \quad (1.4)$$

Per N a 2048 bit, questa espressione implica circa 2^{112} operazioni classiche – un numero astronomico, ben oltre la capacità di qualunque infrastruttura computazionale esistente o prevedibile in ambito classico. I migliori risultati pratici al 2020 hanno raggiunto la fattorizzazione di RSA-250 (829 bit) [12], impiegando circa 2700 anni-CPU di calcolo distribuito.

L'algoritmo di Shor riduce questa complessità a $O(n^3)$ – un salto da sub-esponenziale a **polinomiale** – rendendo la fattorizzazione di chiavi RSA-2048 un problema risolvibile in ore su un computer quantistico fault-tolerant con qualche migliaio di qubit logici. La struttura dell'algoritmo, descritta in dettaglio nel Capitolo 4, sfrutta la Quantum Fourier Transform per trovare il periodo di una funzione modulare, problema che si riduce efficientemente alla fattorizzazione.

1.4.3 Implicazioni Pratiche per la Sicurezza Informatica

Le conseguenze sono sistemiche. RSA-2048 e le sue varianti sono il fondamento di:

- **HTTPS e TLS:** proteggono oltre il 95% del traffico web mondiale, incluse operazioni bancarie, sanitarie e governative;
- **Certificati X.509:** l'infrastruttura a chiave pubblica (PKI) su cui si basa l'autenticità di ogni sito web;
- **SSH:** il protocollo standard per l'accesso remoto sicuro ai server;
- **Firme digitali:** documenti legali, transazioni finanziarie, aggiornamenti software firmati.

Un computer quantistico scalabile renderebbe retroattivamente vulnerabili anche le comunicazioni *storicamente registrate*, attraverso il cosiddetto attacco “*harvest now, decrypt later*” (HN DL): avversari statali raccolgono oggi grandi volumi di traffico cifrato con RSA, conservandoli in attesa di poterli decifrare quando l'hardware quantistico raggiungerà la scala necessaria. Questo scenario non è teorico: rapporti di intelligence statunitensi e cinesi documentano campagne di raccolta massiva di traffico cifrato attive da almeno un decennio. Stime dell'NSA e del *National Institute of Standards and Technology* (NIST) indicano che chiavi RSA-2048 potrebbero diventare vulnerabili entro il 2030–2035 con il proseguire dell'avanzamento tecnologico quantistico [13].

1.5 Crittografia Post-Quantistica: la Risposta dell'Industria

La consapevolezza della minaccia quantistica ha spinto il NIST a lanciare nel 2016 un processo di standardizzazione internazionale per la **crittografia post-quantistica** (*Post-Quantum Cryptography* (PQC)) – algoritmi crittografici resistenti agli attacchi di computer quantistici, progettati per essere eseguiti su hardware classico convenzionale e quindi immediatamente distribuibili.

1.5.1 Gli Standard NIST 2024

Dopo otto anni di valutazione pubblica, crittoanalisi internazionale e tre successivi turni di selezione tra 69 candidati iniziali, nel luglio 2024 il NIST ha pubblicato i primi tre standard definitivi [14]:

- **ML-KEM** (FIPS 203, ex CRYSTALS-Kyber): meccanismo di incapsulamento di chiavi basato sul problema *Module Learning With Errors* (MLWE) su reticoli. È il sostituto designato per lo scambio di chiavi RSA e Diffie-Hellman nei protocolli TLS;
- **ML-DSA** (FIPS 204, ex CRYSTALS-Dilithium): schema di firma digitale basato sullo stesso fondamento matematico di ML-KEM. Sostituisce RSA e ECDSA per le firme digitali;
- **SLH-DSA** (FIPS 205, ex SPHINCS+): schema di firma basato esclusivamente su funzioni hash, senza assunzioni algebriche. Più conservativo per sicurezza, adottato come backup contro potenziali attacchi futuri ai reticoli.

La sicurezza di questi schemi si fonda su problemi matematici – trovare vettori corti in reticoli ad alta dimensione, o invertire funzioni hash – per i quali non sono noti algoritmi quantistici con speedup esponenziale. In particolare, l'algoritmo di Grover fornisce al più uno speedup quadratico sui problemi basati su hash, motivando l'aumento dei parametri di sicurezza (es. passaggio da 128 a 256 bit di sicurezza classica equivalente).

1.5.2 La Sfida della Migrazione

La transizione verso la crittografia post-quantistica è un'operazione di ingegneria su scala globale, paragonabile per complessità alla migrazione da IPv4 a IPv6, ma con urgenza maggiore. Le principali sfide includono:

- **Eterogeneità dei sistemi:** miliardi di dispositivi embedded, sensori industriali e infrastrutture legacy con cicli di vita decennali che non riceveranno aggiornamenti software;
- **Overhead computazionale:** le chiavi ML-KEM sono significativamente più grandi delle chiavi RSA (768 byte vs. 256 byte per sicurezza a 128 bit), con impatto su protocolli a bassa latenza;
- **Migrazione a doppio binario:** nella fase transitoria, molte implementazioni adottano schemi *ibridi* (classico + post-quantistico in parallelo) per mantenere compatibilità con sistemi non aggiornati.

Aziende come Google (Chrome 131), Apple (iMessage PQ3) [15] e Cloudflare hanno già avviato implementazioni ibride nei loro protocolli di produzione. La coesistenza di questa transizione con la ricerca attiva su algoritmi quantistici come Shor definisce il contesto strategico in cui si inserisce il presente lavoro: comprendere i limiti reali di Shor su hardware NISQ è rilevante non solo teoricamente, ma anche per calibrare con maggiore precisione le tempistiche e le priorità della transizione crittografica globale.

1.6 Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant

Nonostante la solidità teorica di questi risultati, la realizzazione pratica di computer quantistici di scala sufficiente ad eseguirli su istanze realmente significative incontra ostacoli formidabili. I sistemi quantistici fisici sono intrinsecamente fragili: qualunque interazione non controllata con l'ambiente esterno – vibrazioni termiche, campi elettromagnetici, imperfezioni nei materiali – provoca fenomeni di **decoerenza** – ossia la perdita progressiva della coerenza quantistica causata dall'entanglement indesiderato tra il qubit e i gradi di libertà ambientali, che sopprime le interferenze quantistiche riducendo il sistema a una miscela statistica classica – degradando lo stato quantistico in modo irreversibile prima che il calcolo possa completarsi. A differenza di un bit classico, che può essere protetto dalla ridondanza, un qubit non può essere copiato (teorema del no-cloning [16]) né letto senza distruggerlo: questo rende la correzione degli errori quantistici un problema radicalmente diverso e molto più complesso di quella classica.

Il panorama tecnologico attuale è dominato da un insieme variegato di piattaforme hardware in competizione, ciascuna con vantaggi e svantaggi distinti:

- **Qubit superconduttori** (IBM, Google, Rigetti): alta velocità dei gate ($\sim 50\text{--}500\text{ ns}$), scalabilità su chip in silicio, ma tempi di coerenza limitati a $\sim 100\text{--}500\text{ }\mu\text{s}$ e necessità di raffreddamento a 15 mK ;
- **Ioni intrappolati** (IonQ, Quantinuum): tempi di coerenza eccellenti ($> 1\text{ s}$), fedeltà dei gate superiore al 99.9% anche su due qubit, ma gate lenti ($\sim 10\text{--}100\text{ }\mu\text{s}$) e scalabilità più complessa;
- **Fotoni** (PsiQuantum, Xanadu): operano a temperatura ambiente, naturalmente adatti alla comunicazione quantistica, ma la generazione deterministica di fotoni entangled rimane una sfida aperta;
- **Spin in silicio** (Intel, UNSW): compatibili con i processi di fabbricazione CMOS esistenti, potenzialmente scalabili a milioni di qubit, ma con tecnologia di controllo ancora in fase di sviluppo.

Un momento simbolico di questa corsa fu raggiunto nel **2019**, quando il team di Google rivendicò il raggiungimento della cosiddetta *quantum supremacy*⁴ con il processore Sycamore a 53 qubit [17]: un calcolo specifico fu completato in 200 secondi, mentre le stime attribuivano al più potente supercomputer classico un tempo di circa 10.000 anni per lo stesso compito. Nel 2023, IBM ha presentato il processore *Condor* a 1.121 qubit e ha delineato una roadmap [18] verso sistemi con architettura fault-tolerant entro la fine del decennio, con il target *Quantum Starling* previsto per il 2029.

Tuttavia, la distanza tra i dispositivi attuali e un computer quantistico *fault-tolerant* – in grado di eseguire Shor su istanze RSA reali – è ancora enorme. I processori odierni operano nel regime cosiddetto *Noisy Intermediate-Scale Quantum* (NISQ), un termine coniato da John Preskill nel 2018 [19] per descrivere macchine con decine o centinaia di qubit fisici, affette da errori significativi che limitano la profondità dei circuiti eseguibili. In questo scenario, la ricerca si concentra su due direzioni parallele e complementari:

- **Correzione degli Errori Quantistici (*Quantum Error Correction*) (QEC):** tecniche per codificare qubit logici in qubit fisici ridondanti, in modo da rilevare e correggere gli errori durante il calcolo, a costo di un overhead fisico significativo (centinaia o migliaia di qubit fisici per qubit logico);
- **Noise mitigation:** tecniche che, senza richiedere overhead hardware, riducono l'effetto del rumore a livello di post-processing classico, estendendo la portata dei circuiti eseguibili su hardware NISQ.

⁴Il termine *quantum supremacy*, coniato da John Preskill nel 2012, indica il punto in cui un computer quantistico esegue un calcolo specifico in un tempo che nessun supercomputer classico potrebbe eguagliare in modo pratico. La rivendicazione di Google con il processore Sycamore è rimasta controversa: IBM sostenne che il calcolo avrebbe potuto essere eseguito classicamente in 2.5 giorni anziché 10.000 anni, con un algoritmo migliorato. Questo dibattito evidenzia la difficoltà di definire e misurare il vantaggio quantistico.

1.7 Motivazione e Contributo del Presente Lavoro

In questo contesto scientifico e tecnologico si inserisce il presente lavoro di tesi. L'algoritmo di Shor rappresenta uno dei risultati più significativi e dirompenti dell'informatica quantistica: la sua capacità di fattorizzare interi in tempo polinomiale minaccia direttamente le fondamenta della crittografia moderna. Tuttavia, la sua esecuzione su hardware NISQ è ostacolata da requisiti stringenti in termini di profondità del circuito, numero di qubit e fedeltà delle porte — rendendo il rumore quantistico il principale ostacolo alla sua realizzazione pratica.

Questo lavoro affronta tale sfida in due fasi. La **prima fase** è una verifica sperimentale sistematica: eseguire l'algoritmo di Shor su simulatori quantistici con noise model calibrati sull'hardware IBM e mettere alla prova l'ipotesi che un classificatore Apprendimento Automatico (*Machine Learning*) (ML) applicato all'output rumoroso riduca il numero di iterazioni necessarie. La verifica confronta tre configurazioni:

- **Metodo 1** (baseline classico, TOP-1): la misura più frequente dell'istogramma viene usata come candidato. Richiede in media $\bar{M}_1 = 6.43$ esecuzioni per $N = 15$ in condizioni NISQ-realistiche.
- **Metodo 2** (classificatore ML + TOP-4): un classificatore binario (Random Forest) pre-addestrato su 2000 istogrammi sintetici filtra le iterazioni con segnale *Quantum Phase Estimation* (QPE) assente; la ricerca è estesa ai 4 candidati più frequenti. Ottiene $\bar{M}_2 = 1.00$, con fattore di riduzione $\rho = 6.44$ ($p = 0.0002$).
- **Analisi di ablazione** (M_{TOP4} , TOP-4 senza classificatore): isola il contributo della ricerca multi-candidato da quello del filtro ML. L'ablazione mostra che M_{TOP4} è statisticamente equivalente a M2 per UC1 ($p = 0.849$) e significativamente migliore per UC2 ($\rho = 2.42$, $p = 0.003$): il guadagno è attribuibile alla **strategia di ricerca TOP-4**, non al classificatore.

L'esito della verifica — l'ML applicato all'output, in quel punto e in quel modo, non serve — delimita con precisione dove va cercata la soluzione: se l'informazione distrutta dal rumore non è recuperabile a valle, l'errore va corretto *durante* il calcolo. La **seconda fase**, che costituisce il nucleo della tesi, è quindi dedicata alla QEC: la costruzione e la simulazione dei codici correttori — dal codice a ripetizione al codice di Steane $[[7, 1, 3]]$, fino al surface code con decodifica *Minimum Weight Perfect Matching* (MWPM) — e la misura sperimentale del **logical error rate** p_L in

funzione dell'errore fisico p e della distanza di codice d . L'integrazione finale avviene a livelli: lo **Shor logico**, in cui ogni gate del circuito subisce l'errore logico residuo p_L misurato sperimentalmente, risponde alla domanda conclusiva del lavoro: *quale livello di correzione è necessario perché Shor mantenga una probabilità di successo accettabile?*

L'elaborato si sviluppa dalle fondamenta teoriche del calcolo quantistico, attraverso la verifica sperimentale su simulatori rumorosi, fino alla correzione d'errore quantistica e allo Shor logico. Gli obiettivi specifici e la struttura dettagliata del lavoro sono descritti nel Capitolo 2.

1.8 Struttura della Tesi

L'elaborato è organizzato in quattordici capitoli, più un'appendice sperimentale. Il Capitolo 2 svolge una funzione trasversale di raccordo tra teoria e pratica; i Capitoli 3–6 costituiscono la parte teorica; i Capitoli 7–14 la parte sperimentale.

Il **Capitolo 2** (*Obiettivi e Piano Sperimentale*) ha una funzione duplice: motiva il lavoro mostrando perché i sistemi crittografici attuali siano strutturalmente vulnerabili all'algoritmo di Shor — attraverso il confronto diretto tra la complessità del *General Number Field Sieve* (GNFS) e quella polinomiale di Shor — e stabilisce il contratto sperimentale completo: obiettivi, ipotesi di ricerca (in particolare $\rho = \bar{M}_1/\bar{M}_2 \gg 1$), contributo originale, requisiti funzionali, parametri dei quattro use case e metriche di valutazione.

Parte teorica (Capitoli 3–6).

Il **Capitolo 3** (*Fondamenti del Calcolo Quantistico*) introduce il formalismo dei qubit, la sovrapposizione, le porte logiche quantistiche e il modello a circuiti. Una sezione dedicata tratta l'entanglement come *risorsa fisica fragile*: l'interazione con l'ambiente trasferisce l'entanglement computazionale verso gradi di libertà esterni, distruggendo la coerenza di fase nel tempo caratteristico T_2 , anticipando così la trattazione del rumore nel Capitolo 5.

Il **Capitolo 4** (*L'Algoritmo di Shor*) presenta la riduzione della fattorizzazione al *period finding*, la Quantum Phase Estimation e la complessità polinomiale $O((\log N)^3)$. Una sezione dedicata esplicita il ruolo dell'entanglement: dopo la moltiplicazione modulare controllata i due registri formano uno stato non separabile la cui correlazione bidirezionale — analoga agli stati di Bell — trasferisce la periodicità di $f(x) = a^x \bmod N$ verso il registro di controllo, dove la QFT⁻¹ la rende misurabile. Il capitolo discute inoltre le implicazioni per RSA e la transizione verso gli standard post-quantum del NIST.

Il **Capitolo 5** (*Rumore Quantistico nei Sistemi NISQ*) parte dall'anatomia fisica di una Unità di Elaborazione Quantistica (*Quantum Processing Unit*) (QPU) — il chip criogenico che contiene solo i qubit, i gate realizzati come impulsi a microonde dall'elettronica esterna, la lettura che distrugge fisicamente lo stato, il “bordo di coerenza” e il ruolo della topologia — per poi analizzare le quattro fonti fisiche di degrado (decoerenza T_1/T_2 , errori di gate, readout errors e crosstalk) e formalizzarle tramite i quattro canali matematici fondamentali (bit flip, phase flip, depolarizzante, amplitude damping). Esplicita il legame tra T_2 e la perdita dell'entanglement tra i

registri di Shor, localizza le quattro fonti sulle fasi dell'algoritmo, e quantifica come la probabilità di successo crolli al 3.4% già per $N = 15$ su hardware NISQ.

Il **Capitolo 6** (*Strategie per la Riduzione del Rumore Quantistico*) descrive e confronta le quattro macro-strategie di mitigazione del rumore nell'era NISQ, approfondisce la Correzione degli Errori Quantistici — dai codici stabilizzatori al surface code — inquadrandola come sottocategoria del Quantum Machine Learning, e motiva la scelta di quest'ultimo come approccio adottato in questa tesi.

Parte sperimentale (Capitoli 7–14).

Il **Capitolo 7** (*Strumenti e Ambiente di Simulazione*) descrive il framework Qiskit e il simulatore Qiskit Aer con accelerazione GPU, illustra la costruzione del noise model parametrizzato e la procedura di traspilazione dei circuiti.

Il **Capitolo 8** (*Metodologia e Architettura*) descrive la pipeline sperimentale a tre stadi (circuit ideale, simulatore rumoroso, analisi con Metodo 1 e Metodo 2) e motiva le scelte architetturali. Una sezione dedicata analizza operativamente le tecniche di mitigazione del rumore disponibili — traspilazione e decomposizione Beauregard, readout mitigation, *Zero-Noise Extrapolation* (ZNE) — e riporta i risultati di una campagna di sensibilità sperimentale su tutti i parametri del noise model: ε_{2q} emerge come unico parametro dominante per $N = 15$, mentre T_1/T_2 , ε_{1q} , p_{ro} e il livello di ottimizzazione della traspilazione risultano privi di effetto misurabile, motivando empiricamente la scelta della strategia TOP-K come meccanismo di mitigazione principale.

Il **Capitolo 9** (*Sviluppo e Implementazione*) presenta il codice sviluppato: l'ambiente di simulazione WSL2 con `AerSimulator` in modalità `statevector`, l'implementazione della Trasformata di Fourier Quantistica (*Quantum Fourier Transform*) (QFT) inversa, del circuito di Shor, del noise model parametrizzato e delle pipeline di entrambi i metodi, con particolare attenzione alla procedura di traspilazione critica (`optimization_level=2` con noise model nel costruttore) necessaria per la corretta decomposizione delle porte Toffoli.

Il **Capitolo 10** (*Verifica Sperimentale dell'Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K*) è il ponte tra la parte teorica e il nucleo della tesi: mette alla prova l'ipotesi di partenza e ne documenta l'esito. Stabilisce la baseline del Metodo 1 sui quattro use case ($\bar{M}_1 = 6.43$ iterazioni per UC1, $\bar{M}_1 = 2.42$ per UC2; UC3 e UC4 inaccessibili per l'elevata profondità di circuito prodotta dalla decomposizione `UnitaryGate`) e isola il contributo del classificatore ML tramite analisi di ablazione: il guadagno ($\rho = 6.44$ su UC1, $p = 0.0002$) è interamente attribuibile alla strategia di ricerca multi-candidato TOP-4, mentre il classificatore risulta statisticamente

irrilevante su UC1 ($p = 0.849$) e controproducente su UC2. Quanto appurato — l’ML a valle dell’esecuzione non serve — motiva il cambio di prospettiva del capitolo successivo; il dettaglio della campagna parametrica e del confronto con la Zero-Noise Extrapolation è riportato nell’Appendice A.

Il **Capitolo 11** (*La Correzione d’Errore Quantistico: dal Codice a Ripetizione al Codice di Steane*) apre il nucleo della tesi. Presenta il ciclo completo della correzione — codifica, stabilizzatori, sindrome, decoder, Pauli frame — e lo mette alla prova in due laboratori a complessità crescente: il codice a ripetizione a 3 qubit, che introduce l’estrazione di sindrome nel caso più leggibile, e il codice di Steane $[[7, 1, 3]]$, primo codice capace di correggere un errore arbitrario su un singolo qubit, con verifica sistematica sindrome→correzione per errori X, Z e Y su ciascuno dei sette qubit.

Il **Capitolo 12** (*Il Surface Code e la Stima dell’Errore Logico*) estende la correzione al codice scalabile di riferimento industriale: descrive la griglia di qubit dati e di misura, i detection events e la decodifica MWPM, e presenta l’esperimento centrale della parte QEC — le curve errore fisico p contro errore logico p_L per distanze $d = 3, 5, 7$, ottenute con i simulatori specializzati Stim e PyMatching, alla ricerca del comportamento di soglia. Una sezione conclusiva discute i limiti del modello di rumore Clifford-Pauli su cui questi simulatori si fondano — che sottostima gli errori coerenti e il leakage — con una verifica diretta del divario su scala trattabile e un confronto con l’approccio *channel-first* della letteratura più recente.

Il **Capitolo 13** (*Integrazione: lo Shor Logico e i Requisiti di Correzione*) chiude il cerchio tramite l’architettura a quattro livelli: il circuito di Shor viene eseguito con un errore logico residuo p_L per gate — il valore misurato al capitolo precedente, iniettato come modello efficace e non come simulazione fault-tolerant completa — e la domanda conclusiva del lavoro (quale p_L serve perché la probabilità di successo, eventualmente cumulata su poche esecuzioni indipendenti, superi l’80%) viene tradotta in requisiti fisici di hardware, collocando i risultati nel contesto delle stime di risorse della letteratura per Shor fault-tolerant.

Il **Capitolo 14** (*Sviluppi Futuri*) delinea le direzioni di ricerca oltre il perimetro sperimentale della tesi: test su hardware reale, decomposizioni efficienti per la moltiplicazione modulare (approccio Beauregard), generalizzazione ad altri algoritmi NISQ, estrazione di sindrome fault-tolerant e reti neurali a supporto della decodifica.

L’**Appendice A** (*Campagna Parametrica Dettagliata e Confronto ZNE*) riporta integralmente le sette sweep parametriche su M_{TOP4} , con il confronto previsione-esito per ciascun parametro, e il dettaglio metodologico del confronto con la Zero-Noise Extrapolation.

Capitolo 2

Obiettivi e Piano Sperimentale

Il Capitolo 1 ha inquadrato il contesto storico e tecnologico del calcolo quantistico. Il presente capitolo costruisce su quel fondamento e svolge una funzione duplice: da un lato motiva e definisce gli obiettivi scientifici del lavoro; dall'altro stabilisce il contratto sperimentale completo — requisiti, parametri, use case e metriche — che guiderà tutta la parte implementativa. Il lettore che avrà letto questo capitolo avrà una visione completa di *perché* il lavoro esiste, *cosa* si propone di dimostrare e *come* intende farlo.

2.1 Motivazione: la Vulnerabilità della Crittografia Attuale

La quasi totalità delle comunicazioni digitali sicure — dalle transazioni bancarie alle connessioni *HyperText Transfer Protocol Secure* (HTTPS), dai protocolli *Secure Shell* (SSH) ai certificati digitali — è protetta da sistemi crittografici la cui sicurezza si fonda su un presupposto computazionale: la difficoltà di fattorizzare il prodotto di due grandi numeri primi. Il sistema RSA, ad esempio, costruisce la propria chiave pubblica come $N = p \cdot q$, dove p e q sono primi di centinaia o migliaia di bit. Finché non esiste un algoritmo classico efficiente per recuperare p e q a partire da N , la chiave privata rimane al sicuro.

Per comprendere perché i sistemi crittografici attuali siano strutturalmente vulnerabili all'algoritmo di Shor, è necessario mettere a confronto le complessità computazionali in gioco. Il miglior algoritmo classico per la fattorizzazione, il GNFS,

richiede un tempo:

$$T_{\text{classico}}(N) = O(\exp((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3})) \quad (2.1)$$

che, pur crescendo più lentamente di un esponenziale puro, rimane del tutto impraticabile per i valori di N usati in crittografia. A titolo di esempio, fattorizzare un numero RSA a 2048 bit con algoritmi classici richiederebbe un tempo stimato superiore all'età dell'universo, anche impiegando l'intera potenza di calcolo disponibile sul pianeta.

Nel 1994, Peter Shor dimostrò che un computer quantistico è in grado di eseguire la stessa operazione in tempo **polinomiale** [7]:

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (2.2)$$

Lo scarto tra le due espressioni non è una differenza di grado, ma una differenza di *natura*: il tempo classico cresce in modo super-polinomiale con la lunghezza della chiave, mentre il tempo quantistico cresce polinomialmente. Ciò significa che **augmentare la lunghezza della chiave RSA non è una contromisura efficace** contro Shor: raddoppiare n (i bit di N) porta il tempo classico da impraticabile a ancora più impraticabile, ma porta il tempo quantistico da $O(n^3)$ a $O((2n)^3) = 8O(n^3)$ — un fattore costante, del tutto gestibile. La sicurezza di RSA è quindi **irrecuperabilmente compromessa** dalla disponibilità di un computer quantistico fault-tolerant, indipendentemente dalla lunghezza della chiave scelta. La stessa rottura si estende a tutti i sistemi basati sul logaritmo discreto (Diffie-Hellman, ECDSA), per i quali Shor fornisce un algoritmo analogamente efficiente.

La minaccia non è futura in senso astratto. Il fenomeno del *harvest now, decrypt later* — già documentato in ambito di intelligence — consiste nel raccogliere oggi grandi quantità di dati cifrati con RSA, conservarli, e decifrarli non appena sarà disponibile un computer quantistico fault-tolerant. I dati cifrati oggi con chiavi RSA-2048 potrebbero quindi diventare leggibili in un orizzonte di dieci-quindici anni, secondo le roadmap pubblicate da International Business Machines (IBM) e da altri produttori di hardware quantistico [18].

Il problema pratico che impedisce questa minaccia di materializzarsi oggi è il **rumore**. I processori quantistici attuali operano in regime NISQ: dispongono di decine o centinaia di qubit fisici, ma sono affetti da decoerenza, errori di gate e readout error che degradano l'output dell'algoritmo di Shor fino a renderlo inaffidabile senza

un numero elevato di ripetizioni. Comprendere questo limite e sviluppare strategie per superarlo — anche parzialmente, tramite modelli di machine learning — è la ragione d’essere del presente lavoro.

2.2 Obiettivi Generali

Partendo dal contesto delineato, il presente lavoro si propone di analizzare in profondità l’algoritmo di Shor per la fattorizzazione di interi e di affrontare il problema della sua esecuzione su hardware quantistico reale, afflitto da rumore e decoerenza.

Il filo conduttore del lavoro è duplice: da un lato la comprensione del **potenziale computazionale** offerto dall’algoritmo di Shor rispetto ai migliori approcci classici noti; dall’altro lo studio delle **limitazioni pratiche** imposte dal rumore nei dispositivi NISQ attuali, e lo sviluppo di strategie di mitigazione e ottimizzazione basate su **modelli di intelligenza artificiale**.

2.3 Obiettivi Specifici

1. **Fondamenti del calcolo quantistico.** Fornire una trattazione rigorosa dei principi matematici e fisici su cui si basano gli algoritmi quantistici: il modello del qubit, la sovrapposizione, l’entanglement, le porte logiche quantistiche e il meccanismo di interferenza. Questo obiettivo costituisce la base teorica indispensabile per la comprensione dell’algoritmo trattato nei capitoli successivi.
2. **Analisi teorica dell’algoritmo di Shor.** Descrivere in dettaglio il funzionamento dell’algoritmo di Shor per la fattorizzazione di interi in tempo polinomiale, con particolare attenzione alla riduzione al problema del period-finding, alla Quantum Fourier Transform (QFT) e alla tecnica di phase estimation. Analizzare la complessità computazionale e valutare le implicazioni per la crittografia RSA e per la sicurezza informatica post-quantum.
3. **Implementazione su Qiskit e analisi sperimentale.** Implementare l’algoritmo di Shor utilizzando il framework open-source Qiskit, costruendo i circuiti per la QFT, la modular exponentiation e il circuito completo di fattorizzazione. Verificare il comportamento del circuito su simulatore ideale e analizzare le prestazioni su istanze concrete.

4. **Studio del rumore quantistico nell’algoritmo di Shor.** Analizzare le principali fonti di errore che affliggono l’esecuzione dell’algoritmo di Shor su hardware reale – decoerenza, errori di gate, readout errors – con particolare attenzione alle componenti più sensibili al rumore: la QFT e la phase estimation. Studiare le strategie di QEC e di noise mitigation disponibili (ZNE, *Probabilistic Error Cancellation* (PEC), readout mitigation), valutandone l’efficacia sull’algoritmo di Shor.
5. **Ottimizzazione mediante modelli di intelligenza artificiale.** Esplorare l’integrazione di modelli di machine learning per la caratterizzazione del rumore, l’ottimizzazione dei parametri circuitali e il miglioramento della fedeltà dei risultati in ambienti quantistici rumorosi. Valutare l’apporto dell’intelligenza artificiale come strumento complementare alle tecniche di mitigazione tradizionali per estendere la praticabilità dell’algoritmo di Shor su hardware NISQ.

2.4 Ipotesi di Lavoro

Il contributo sperimentale della tesi è strutturato attorno a un’ipotesi principale e a due ipotesi secondarie, formulate prima dell’esecuzione degli esperimenti e verificate quantitativamente sui dati raccolti.

Ipotesi principale. L’integrazione di un classificatore ML nella pipeline di analisi riduce significativamente il numero medio di iterazioni necessarie per ottenere un risultato corretto dall’algoritmo di Shor su simulatore rumoroso. In termini formali, il fattore di riduzione

$$\rho = \frac{\bar{M}_1}{\bar{M}_2} \gg 1 \quad (2.3)$$

dove $\bar{M}_1$ e $\bar{M}_2$ sono rispettivamente il numero medio di iterazioni del Metodo 1 (baseline classico) e del Metodo 2 (classificatore ML). La verifica statistica avviene tramite il test di Mann-Whitney U (non parametrico, unilaterale) sull’ipotesi nulla $H_0: \bar{M}_1 = \bar{M}_2$, con soglia di significatività $\alpha = 0.05$.

Ipotesi secondaria 1. Il classificatore ML raggiunge prestazioni sufficienti a distinguere esecuzioni corrette da errate anche in presenza di rumore variabile: $F1 > 0.85$ e $AUC > 0.90$ sul test set.

Ipotesi secondaria 2. Il vantaggio del Metodo 2 si mantiene al crescere della dimensione dell’istanza ($N = 15 \rightarrow 21 \rightarrow 35$) e al peggiorare del livello di rumore,

confermando la generalizzabilità dell’approccio oltre il caso baseline.

2.5 Contributo Originale

Rispetto alla letteratura esistente sull’algoritmo di Shor e sulla mitigazione del rumore nei sistemi NISQ, questo lavoro apporta i seguenti contributi originali.

Progettazione e validazione sperimentale della strategia TOP-K. La maggioranza degli studi di mitigazione del rumore sull’algoritmo di Shor si concentra su tecniche strutturali (Zero-Noise Extrapolation, Probabilistic Error Cancellation, Dynamical Decoupling) applicate al circuito prima della misura. Il presente lavoro adotta invece un approccio *post-processing*: la strategia di ricerca multi-candidato TOP-K, applicata direttamente all’istogramma di output del simulatore rumoroso, riduce il numero di iterazioni necessarie sfruttando la struttura residua dei picchi QPE. L’analisi di ablazione dimostra che questo componente è il meccanismo attivo del miglioramento, applicabile senza modifiche al circuito né accesso al modello di rumore dell’hardware.

Quantificazione empirica del fattore di riduzione ρ . La letteratura fornisce stime teoriche del numero di iterazioni necessarie a ottenere un risultato corretto da Shor in presenza di rumore, ma raramente misure sperimentali sistematiche su più configurazioni. Il presente lavoro misura $\bar{M}_1$ e $\bar{M}$ (Metodo 1 e M_{TOP4}) su quattro use case distinti — due livelli di rumore per $N = 15$, e due istanze di dimensione crescente ($N = 21$, $N = 35$) — producendo stime empiriche di ρ validate statisticamente con test di Mann-Whitney U.

Analisi di ablazione che separa il contributo TOP-K da quello del classificatore ML. Il lavoro conduce un’analisi di ablazione esplicita che isola il contributo della strategia di ricerca rispetto a quello del classificatore ML. Il risultato — che M_{TOP4} (TOP-K senza classificatore) è statisticamente equivalente a M2 su UC1 e significativamente migliore su UC2 — chiarisce quale componente è responsabile del guadagno, e orienta gli sviluppi futuri verso l’ottimizzazione della strategia di ricerca piuttosto che del modello apprenditivo.

2.6 Requisiti Funzionali

Definiti gli obiettivi e il contributo originale, è possibile formalizzare cosa il sistema sperimentale deve concretamente fare. I requisiti seguenti traducono le ipotesi di

ricerca in vincoli tecnici verificabili, e costituiscono il riferimento rispetto al quale l'implementazione descritta nel Capitolo 9 verrà valutata.

Il sistema sperimentale deve soddisfare i seguenti requisiti funzionali:

1. **Esecuzione configurabile del circuito di Shor.** Dato un intero N da fattorizzare e una base a con $\gcd(a, N) = 1$, il sistema costruisce il circuito dell'algoritmo di Shor a partire dall'implementazione di riferimento [20], lo configura con il numero di qubit n_{count} specificato e lo esegue tramite **AerSimulator** con il noise model attivo per un numero di shot M_{shots} configurabile.
2. **Modello di rumore parametrizzabile.** Il sistema permette di configurare in modo indipendente tutti i parametri del noise model — tassi di errore depolarizzante per gate a singolo e doppio qubit, tempi di rilassamento T_1 e T_2 , probabilità di readout error — in modo da poter riprodurre diverse condizioni operative e generare dataset con distribuzione di rumore variabile. Questo requisito è fondamentale per la creazione di un dataset di addestramento sufficientemente ricco per il Metodo 2, come indicato dalla metodologia di selezione dei noise model proposta in letteratura [21].
3. **Raccolta e archiviazione degli output.** Il sistema registra la distribuzione di misura del registro di controllo per ogni esecuzione e la archivia in formato strutturato, garantendo la riproducibilità degli esperimenti tramite seed fisso per il generatore di numeri casuali di Qiskit Aer.
4. **Analisi statistica del Metodo 1.** Il sistema implementa la pipeline del Metodo 1: identificazione dei picchi della distribuzione tramite filtraggio a soglia, stima del periodo r tramite frazioni continue, verifica della correttezza dei fattori estratti. Misura e registra il numero di iterazioni necessarie per raggiungere il primo risultato corretto.
5. **Training, validazione e inferenza del Metodo 2.** Il sistema genera il dataset dal simulatore rumoroso, addestra il classificatore con divisione training/validation/test, ne valuta le metriche di accuratezza e lo utilizza in modalità inferenza: dopo ogni esecuzione del circuito, il classificatore decide se il risultato è corretto e, in caso affermativo, interrompe il processo.
6. **Confronto sistematico sui quattro use case.** Entrambi i metodi vengono eseguiti sugli stessi quattro use case, nelle medesime condizioni di rumore, producendo metriche confrontabili secondo i criteri definiti nella Sezione 2.9.

7. **Riproducibilità.** Tutti gli esperimenti sono riproducibili: seed fisso, parametri esplicitati, codice disponibile nel repository del progetto. Questo requisito è condizione necessaria per la validazione accademica dei risultati [22].

2.7 Parametri di Input e Output

2.7.1 Parametri di Input

La Tabella 2.1 elenca i parametri di configurazione del sistema. I loro valori vengono fissati per ciascun use case prima dell'esecuzione e rimangono costanti per tutta la durata dell'esperimento corrispondente.

Parametro	Unità	Descrizione
N	intero	Numero da fattorizzare
a	intero	Base della moltiplicazione modulare ($\gcd(a, N) = 1$)
n_{count}	intero	Qubit del registro di controllo
M_{shots}	intero	Shot per esecuzione del simulatore
ϵ_{1q}	adimensionale	Tasso di errore depolarizzante su gate a 1 qubit
ϵ_{2q}	adimensionale	Tasso di errore depolarizzante su gate a 2 qubit
T_1	ns	Tempo di rilassamento (amplitude damping)
T_2	ns	Tempo di dephasing
p_{ro}	adimensionale	Probabilità di readout error

Tabella 2.1: Parametri di input del sistema sperimentale. I valori di ϵ_{1q} , ϵ_{2q} , T_1 , T_2 e p_{ro} definiscono il noise model e vengono variati tra i use case per coprire scenari di rumore diversi.

2.7.2 Output del Sistema

Per ogni use case e per ciascuno dei due metodi, il sistema produce le seguenti quantità:

- I fattori trovati p e q tali che $p \cdot q = N$, con verifica esplicita della correttezza ($p \neq 1$, $q \neq 1$, $p \cdot q = N$);
- Il numero di iterazioni necessarie per il primo risultato corretto;
- La distribuzione di frequenza completa degli output del registro di controllo su M_{shots} esecuzioni;
- Le metriche di confronto tra i due metodi definite nella Sezione 2.9.

Per il solo Metodo 2, vengono prodotte in aggiunta:

- L'accuratezza del classificatore e il punteggio F1 sul test set;
- La curva ROC con l'area sottostante (Area Sotto la Curva ROC (*Area Under the Curve*) (AUC)).

2.8 I Quattro Use Case

I quattro use case sono scelti per garantire che i risultati non dipendano da una singola configurazione e che coprano un intervallo rappresentativo di condizioni operative. Il disegno sperimentale segue due assi di variazione indipendenti:

- **Asse rumore (Use Case 1 vs. 2):** stesso numero $N = 15$, stessa base $a = 7$, stesso circuito — livello di rumore diverso. Il confronto isola l'effetto del rumore sul numero di iterazioni necessarie, a parità di complessità circuitale. Il Use Case 2 utilizza parametri di errore circa cinque volte superiori a quelli del Use Case 1, per coprire sia un regime *NISQ-realistico* sia uno *NISQ-degradato*.
- **Asse scalabilità (Use Case 1, 3, 4):** numeri N di dimensione crescente ($15 \rightarrow 21 \rightarrow 35$), stesso livello di rumore *NISQ-realistico*. Il confronto valuta come il numero medio di iterazioni e l'accuratezza del classificatore varino al crescere della profondità del circuito.

La scelta di quattro use case soddisfa il requisito metodologico minimo per la validazione accademica dei risultati in questo dominio [22].

2.8.1 Parametri dei Use Case

La Tabella 2.2 riporta la configurazione completa dei quattro use case. I parametri di rumore del livello *NISQ-realistico* sono derivati dalle specifiche di calibrazione di `ibm_marrakesh` riportate nel tutorial ufficiale IBM [23] e dalla metodologia di selezione dei noise model proposta in [21]. Il livello *NISQ-degradato* scala i tassi di errore di un fattore $5\times$ e dimezza i tempi di coerenza, simulando condizioni di hardware più antico o in stato di calibrazione peggiore.

Parametro	UC 1	UC 2	UC 3	UC 4
N	15	15	21	35
a	7	7	2	6
Periodo r atteso	4	4	6	2
n_{count} (qubit)	8	8	10	12
M_{shots}	1024	1024	1024	1024
<i>Livello rumore</i>	NISQ-realistico	NISQ-degradato	NISQ-realistico	NISQ-realistico
ϵ_{1q}	1×10^{-3}	5×10^{-3}	1×10^{-3}	1×10^{-3}
ϵ_{2q}	1×10^{-2}	5×10^{-2}	1×10^{-2}	1×10^{-2}
T_1 (ns)	100 000	50 000	100 000	100 000
T_2 (ns)	80 000	30 000	80 000	80 000
p_{ro}	2×10^{-2}	5×10^{-2}	2×10^{-2}	2×10^{-2}

Tabella 2.2: Configurazione completa dei quattro use case. I parametri di rumore del livello NISQ-realistico sono derivati dalle specifiche di `ibm_marrakesh` [23, 21]. Il livello NISQ-degradato scala i tassi di errore di $5\times$ e dimezza T_1 e T_2 .

2.8.2 Razionale per la Scelta delle Istanze

Use Case 1 — $N = 15$, **NISQ-realistico**. È il caso di riferimento consolidato in letteratura: la prima dimostrazione sperimentale di Shor è avvenuta su $N = 15$ con NMR a 7 qubit [24], e il tutorial ufficiale IBM riporta i dati di esecuzione per la stessa istanza su `ibm_marrakesh` [23]. La base $a = 7$ produce un periodo $r = 4$, sufficiente per estrarre i fattori 3 e 5 tramite l’algoritmo delle frazioni continue. Questo use case funge da baseline quantitativa per entrambi i metodi.

Use Case 2 — $N = 15$, **NISQ-degradato**. Configurazione identica al Use Case 1 eccetto per il livello di rumore, aumentato di $5\times$ su tutti i parametri. Poiché il circuito è lo stesso, qualsiasi differenza nelle metriche $\bar{M}_1$ e $\bar{M}_2$ è attribuibile esclusivamente al rumore. Questo use case permette di valutare la **robustezza** del classificatore ML rispetto al degrado delle condizioni hardware, indipendentemente dalla complessità circuitale.

Use Case 3 — $N = 21$, **NISQ-realistico**. $N = 21 = 3 \times 7$ è la fattorizzazione più grande dimostrata su hardware superconduttore a uso generale [25]. La base $a = 2$ produce un periodo $r = 6$, che implica una profondità circuitale sensibilmente maggiore rispetto a $N = 15$. A parità di livello di rumore con il Use Case 1, questo use case isola l’effetto della scalabilità: ci si attende un aumento di $\bar{M}_1$ per via del circuito più profondo, e si valuta se il classificatore ML mantiene un’accuratezza sufficiente nonostante la maggiore complessità.

Use Case 4 — $N = 35$, NISQ-realistico. $N = 35 = 5 \times 7$ con base $a = 6$ produce il periodo minimo $r = 2$ (poiché $6^2 = 36 \equiv 1 \pmod{35}$), il che consente di fattorizzare direttamente con $\gcd(6 - 1, 35) = \gcd(5, 35) = 5$ e $\gcd(6 + 1, 35) = \gcd(7, 35) = 7$. La scelta di $r = 2$ con N maggiore permette di separare l'effetto della dimensione del registro (più qubit) dall'effetto della profondità della modular exponentiation, fornendo un punto di confronto complementare rispetto al Use Case 3.

2.9 Metriche di Valutazione

2.9.1 Metriche del Metodo 1

- $\bar{M}_1$: numero medio di esecuzioni necessarie per ottenere il primo risultato corretto, calcolato su $K = 30$ ripetizioni indipendenti dello stesso esperimento.
- σ_{M_1} : deviazione standard del numero di esecuzioni, a indicare la variabilità del metodo.
- $P_{\text{success},1}(M_{\text{max}})$: probabilità di successo entro un budget fisso di M_{max} esecuzioni, dove M_{max} verrà fissato in base ai primi risultati sperimentali.

2.9.2 Metriche del Metodo 2

- $\bar{M}_2$: numero medio di esecuzioni prima che il classificatore restituisca la prima etichetta 1 corretta, sulle stesse K ripetizioni usate per il Metodo 1.
- σ_{M_2} : deviazione standard corrispondente.
- Acc_{clf} : accuratezza del classificatore sul test set, definita come il rapporto tra predizioni corrette e numero totale di campioni di test.
- F1_{clf} : F1-score, che bilancia la precisione e il recall nella classe positiva (output corretto). È la metrica preferita rispetto alla sola accuratezza in presenza di classi sbilanciate, un fenomeno frequente quando il tasso di successo dell'algoritmo è basso.
- $P_{\text{success},2}(M_{\text{max}})$: probabilità di successo entro lo stesso budget M_{max} utilizzato per il Metodo 1.

2.9.3 Metrica di Confronto

La metrica principale per il confronto tra i due metodi è il **fattore di riduzione delle iterazioni**:

$$\rho = \frac{\bar{M}_1}{\bar{M}_2} \quad (2.4)$$

Un valore $\rho > 1$ indica che il Metodo 2 converge più rapidamente del Metodo 1. L'ipotesi sperimentale, motivata nel Capitolo 6, è $\rho \gg 1$, con $\bar{M}_2 \approx 3$. La significatività statistica del risultato viene valutata tramite il test di Mann-Whitney U (non parametrico, unilaterale $M_1 > M_2$) sulle distribuzioni di M_1 e M_2 . Poiché il Metodo 2 combina due componenti distinte (il classificatore ML e la ricerca multi-candidato TOP-K), il piano sperimentale prevede inoltre un'**analisi di ablazione**: la variante M_{TOP4} (TOP-K senza classificatore) viene misurata nelle stesse condizioni, in modo da attribuire un eventuale guadagno all'una o all'altra componente e verificare se l'apprendimento automatico è effettivamente necessario.

2.9.4 Parametri di Error Mitigation Applicati

Il noise model incorpora il readout error come matrice di confusione simmetrica (parametro p_{ro}) direttamente nel simulatore. Non viene applicata una correzione post-hoc tramite matrice di calibrazione: la robustezza dei metodi al readout error viene invece verificata sperimentalmente nella campagna parametrica (Sezione A.5 dell'Appendice A), che mostra invarianza di M_{TOP4} fino a $p_{\text{ro}} = 20\%$.

Il Metodo 1 identifica i candidati per la fattorizzazione selezionando le misure più frequenti dall'istogramma senza sogliatura preliminare: la funzione `extract_factors` è applicata direttamente alle misure ordinate per frequenza decrescente. Questo approccio è sufficiente in presenza del noise model adottato, come confermato dai risultati sperimentali.

2.10 Approccio Metodologico

Il lavoro integra tre componenti — una parte teorica, una sperimentale e una di analisi comparativa tra strategie di ricerca — che si sviluppano in sequenza lungo l'elaborato. L'architettura dettagliata del sistema sperimentale, le scelte tecnologiche adottate e la struttura dei due metodi sviluppati sono descritte nel Capitolo 8.

Capitolo 3

Fondamenti del Calcolo Quantistico

3.1 Dalla Meccanica Classica alla Meccanica Quantistica

La meccanica quantistica descrive il comportamento della materia e dell'energia alle scale atomiche e subatomiche, dove i principi della fisica classica cessano di essere validi. I suoi postulati, formulati nella prima metà del Novecento da Heisenberg, Schrödinger, Dirac e Bohr, introducono concetti radicalmente diversi da quelli classici: lo stato di un sistema fisico non è determinato in modo assoluto, ma è descritto da una **funzione d'onda** la cui evoluzione è governata dall'equazione di Schrödinger.

Il calcolo quantistico traduce questi principi fisici in risorse computazionali. La **sovrapposizione** consente a un bit quantistico di trovarsi in una combinazione lineare di 0 e 1 simultaneamente; l'**entanglement** crea correlazioni non classiche tra qubit distinti; l'**interferenza** permette di amplificare le soluzioni corrette e cancellare quelle errate.

3.2 I Postulati della Meccanica Quantistica

Il formalismo matematico della meccanica quantistica poggia su quattro postulati fondamentali, che costituiscono l'assiomatica della teoria e il punto di partenza rigoroso per la descrizione di qualsiasi sistema quantistico [26].

1. **Postulato dello spazio degli stati.** Ad ogni sistema fisico isolato è associato uno **spazio di Hilbert** $\mathcal{H}$, detto *spazio degli stati* del sistema. Lo stato del sistema è completamente descritto da un vettore unitario $|\psi\rangle \in \mathcal{H}$, detto *vettore di stato* o *ket*. Per un sistema a due livelli (qubit), $\mathcal{H} = \mathbb{C}^2$ e lo stato generale è $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ con $|\alpha|^2 + |\beta|^2 = 1$.
2. **Postulato dell'evoluzione.** L'evoluzione di un sistema quantistico chiuso è descritta da una **trasformazione unitaria**: se lo stato del sistema all'istante t_1 è $|\psi(t_1)\rangle$, allora all'istante t_2 esso si trova nello stato

$$|\psi(t_2)\rangle = U(t_1, t_2) |\psi(t_1)\rangle \quad (3.1)$$

dove $U(t_1, t_2)$ è un operatore unitario ($U^\dagger U = I$) che dipende dagli istanti t_1 e t_2 . In forma differenziale, l'evoluzione è governata dall'**equazione di Schrödinger**:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (3.2)$$

dove $\hat{H}$ è l'**Hamiltoniano** del sistema — un operatore hermitiano ($\hat{H} = \hat{H}^\dagger$) che rappresenta l'energia totale — e $\hbar = h/(2\pi)$ è la costante di Planck ridotta.¹

3. **Postulato della misura.** Le misure quantistiche sono descritte da un insieme $\{M_m\}$ di **operatori di misura**, indicizzati dagli esiti m , soggetti alla relazione di completezza:

$$\sum_m M_m^\dagger M_m = I \quad (3.3)$$

¹Nel calcolo quantistico a porte, ogni porta è la realizzazione discreta di questo postulato: la matrice unitaria U descrive l'evoluzione del registro di qubit in un passo computazionale. L'Hamiltoniano fisico che implementa una porta specifica dipende dall'architettura hardware (impulsi a microonde per qubit superconduttori, impulsi laser per ioni intrappolati, ecc.).

Se il sistema si trova nello stato $|\psi\rangle$ prima della misura, la probabilità di ottenere l'esito m è:

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (3.4)$$

e lo stato immediatamente dopo la misura con esito m è il vettore normalizzato:

$$|\psi'\rangle = \frac{M_m |\psi\rangle}{\sqrt{p(m)}} \quad (3.5)$$

Nel caso della **misura proiettiva in base computazionale** — il caso standard nel calcolo quantistico — gli operatori sono i proiettori $M_m = |m\rangle \langle m|$, e la probabilità di ottenere m si riduce alla **regola di Born**: $p(m) = |\langle m | \psi \rangle|^2$. La misura è un'operazione *non unitaria e irreversibile*: il collasso dello stato distrugge irrecuperabilmente le ampiezze che non corrispondono all'esito osservato.

4. **Postulato dei sistemi composti.** Lo spazio degli stati di un sistema fisico composto da n sottosistemi con spazi $\mathcal{H}_1, \dots, \mathcal{H}_n$ è il **prodotto tensoriale**:

$$\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_n \quad (3.6)$$

Se ciascun sottosistema i si trova nello stato $|\psi_i\rangle$, lo stato del sistema composto è $|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle$. Esistono tuttavia stati del sistema composto — gli stati **entangled** — che non possono essere scritti in questa forma fattorizzata per nessuna scelta degli stati individuali: la misura di un sottosistema modifica istantaneamente le probabilità degli esiti di misura sull'altro, indipendentemente dalla distanza fisica tra i due.

Questi quattro postulati forniscono il quadro assiomatico completo entro cui si collocano tutti i concetti che seguono: il qubit (Postulato 1), le porte quantistiche come trasformazioni unitarie (Postulato 2), la misura e il collasso dello stato (Postulato 3), i registri multi-qubit e l'entanglement (Postulato 4).

3.3 Il Qubit

3.3.1 Definizione Matematica

L'unità fondamentale del calcolo quantistico è il **qubit** (*quantum bit*). Mentre un bit classico assume uno dei due valori $\{0, 1\}$, un qubit è descritto da un vettore unitario nello spazio di Hilbert bidimensionale $\mathcal{H} = \mathbb{C}^2$.²

Introducendo la **notazione di Dirac**³, i due stati base computazionali sono:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (3.7)$$

Lo stato generico di un qubit è la **sovrapposizione**:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (3.8)$$

dove $|\alpha|^2$ e $|\beta|^2$ rappresentano le probabilità di ottenere rispettivamente 0 e 1 al momento della misura. Il vincolo di normalizzazione $|\alpha|^2 + |\beta|^2 = 1$ garantisce che le probabilità sommino a 1.

3.3.2 La Sfera di Bloch

Ogni stato puro di un qubit può essere visualizzato come un punto sulla superficie della **sfera di Bloch**, una sfera unitaria in $\mathbb{R}^3$.⁴ Parametrizzando le ampiezze complesse tramite angoli reali $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi]$:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle \quad (3.9)$$

²Lo **spazio di Hilbert** prende il nome dal matematico tedesco David Hilbert (1862–1943), uno dei più influenti matematici del XX secolo. In meccanica quantistica indica uno spazio vettoriale complesso dotato di prodotto interno, che fornisce il formalismo matematico per descrivere gli stati di un sistema fisico.

³La **notazione bra-ket** fu introdotta dal fisico britannico Paul Adrien Maurice Dirac (1902–1984), Premio Nobel per la Fisica nel 1933. Il simbolo $|\psi\rangle$ è detto *ket*, mentre $\langle\psi|$ è detto *bra*; il prodotto interno $\langle\phi|\psi\rangle$ forma il *bracket*, da cui il nome.

⁴La **sfera di Bloch** deve il suo nome al fisico svizzero-americano Felix Bloch (1905–1983), Premio Nobel per la Fisica nel 1952 (insieme a Edward Purcell) per lo sviluppo della spettroscopia a risonanza magnetica nucleare (Risonanza Magnetica Nucleare (*Nuclear Magnetic Resonance*) (NMR)). La stessa notazione T_1 e T_2 per i tempi di decoerenza proviene dal formalismo NMR di Bloch.

I poli nord e sud corrispondono agli stati $|0\rangle$ e $|1\rangle$, mentre i punti sull'equatore rappresentano sovrapposizioni equiprobabili con fasi diverse.

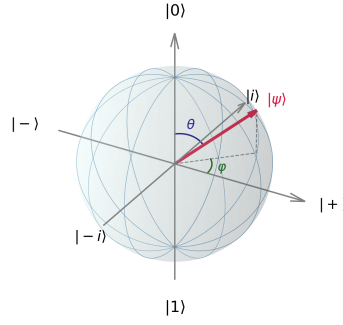


Figura 3.1: La sfera di Bloch: ogni stato puro di un qubit corrisponde a un punto sulla superficie della sfera unitaria, parametrizzato dagli angoli $\theta \in [0, \pi]$ e $\varphi \in [0, 2\pi)$. I poli nord e sud rappresentano gli stati base $|0\rangle$ e $|1\rangle$; i punti sull'equatore corrispondono a sovrapposizioni equiprobabili.

3.4 Sistemi Multi-Qubit e Prodotto Tensoriale

Un sistema di n qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H}^{\otimes n} = \mathbb{C}^{2^n}$, ottenuto come prodotto tensoriale degli spazi individuali. Uno stato generico di n qubit è:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1 \quad (3.10)$$

La dimensione esponenziale 2^n dello spazio di Hilbert è la fonte del potere computazionale dei sistemi quantistici: con $n = 300$ qubit, lo spazio degli stati ha dimensione 2^{300} , superiore al numero di atomi nell'universo osservabile.

3.4.1 Entanglement

Due qubit sono detti **entangled** quando il loro stato congiunto non può essere scritto come prodotto tensoriale degli stati individuali. Gli stati di Bell costituiscono la base

canonica degli stati entangled a due qubit:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (3.11)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (3.12)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (3.13)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (3.14)$$

Lo stato $|\Phi^+\rangle$ non può essere fattorizzato come $|\psi_A\rangle \otimes |\psi_B\rangle$ per nessuna scelta di $|\psi_A\rangle$ e $|\psi_B\rangle$: misurare il primo qubit come 0 garantisce che il secondo sia 0, indipendentemente dalla distanza fisica tra i due sistemi.

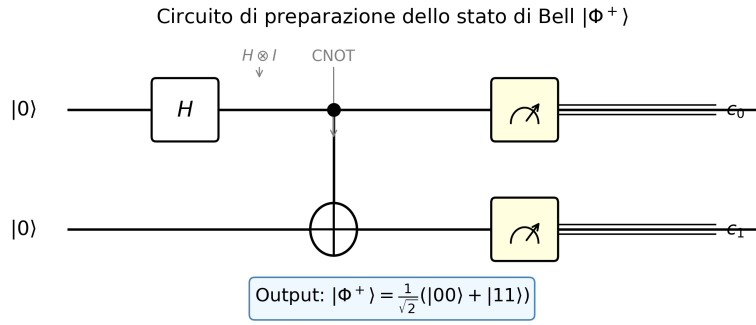


Figura 3.2: Circuito quantistico per la preparazione dello stato di Bell $|\Phi^+\rangle$. Una porta di Hadamard H posta sul qubit di controllo crea la sovrapposizione $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$; la successiva porta CNOT introduce le correlazioni di entanglement, producendo lo stato $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

3.4.2 L'Entanglement come Risorsa Fisica Fragile

L'entanglement non è soltanto una proprietà astratta degli stati quantistici: è la **risorsa fisica** che gli algoritmi quantistici consumano per ottenere il loro vantaggio computazionale. Nell'algoritmo di Shor, il registro di controllo e il registro di lavoro devono restare entangled per tutta la durata della phase estimation affinché la Quantum Fourier Transform possa estrarre il periodo corretto. La perdita anche parziale di questa correlazione è sufficiente a degradare il risultato.

La fragilità dell'entanglement ha un'origine fisica precisa. Un sistema quantistico reale non è mai perfettamente isolato dall'ambiente che lo circonda: campi elettromagnetici residui, vibrazioni termiche del reticolo cristallino e fluttuazioni di

carica si accoppiano continuamente ai qubit. Quando questo accoppiamento avviene, il sistema entra in uno stato entangled *con l'ambiente*, trasferendo ad esso parte dell'informazione di fase. Dal punto di vista del solo sistema, le ampiezze di interferenza si riducono: il qubit non è più in una sovrapposizione pura $\alpha|0\rangle + \beta|1\rangle$, ma in un **miscuglio statistico** in cui la fase relativa tra $|0\rangle$ e $|1\rangle$ è persa. Questo processo prende il nome di **decoerenza** [27].

Il tempo caratteristico entro cui questa perdita di fase diventa apprezzabile è T_2 , il **tempo di coerenza di fase** (*dephasing time*). Esso misura per quanto tempo un qubit riesce a mantenere la propria sovrapposizione coerente prima che l'interazione con l'ambiente la distrugga. Sui processori superconduttori attuali, T_2 è tipicamente nell'intervallo $50\text{--}200\mu\text{s}$; ogni singola porta a due qubit dura circa $300\text{--}500\text{ ns}$. Un calcolo diretto rivela che l'algoritmo di Shor, la cui profondità cresce come $O(n^3)$ nel numero di bit dell'intero da fattorizzare, supera il limite imposto da T_2 già per $n \gtrsim 10\text{--}15$ bit. La trattazione quantitativa di questi tempi caratteristici e del loro impatto sull'algoritmo è oggetto del Capitolo 5.

3.5 Porte Quantistiche

Le operazioni su qubit sono rappresentate da **matrici unitarie**: $U^\dagger U = I$. La condizione di unitarietà garantisce la reversibilità del calcolo e la conservazione della norma (delle probabilità).

3.5.1 Porte a Singolo Qubit

Le principali porte a singolo qubit sono:

Le tre **matrici di Pauli** X , Y , Z costituiscono, insieme all'identità I , una base per lo spazio delle matrici 2×2 hermitiane a traccia nulla, e giocano un ruolo centrale sia nella descrizione degli stati del qubit sia nella caratterizzazione dei canali di rumore.

Porta di Pauli-X (NOT quantistico / Bit Flip):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle \quad (3.15)$$

È l'analogo quantistico del NOT classico: inverte i valori di $|0\rangle$ e $|1\rangle$.

Porta di Pauli-Y:

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle \quad (3.16)$$

Combina un bit flip e un phase flip: è equivalente (a meno di una fase globale) alla composizione iXZ . Compare naturalmente nella decomposizione di rotazioni generali su qubit e nella descrizione del canale di *depolarizzazione*.

Porta di Pauli-Z (Phase Flip):

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle \quad (3.17)$$

Lascia invariato $|0\rangle$ e aggiunge una fase -1 a $|1\rangle$: è un *phase flip*.

Le tre matrici di Pauli soddisfano le seguenti relazioni algebriche fondamentali:

$$X^2 = Y^2 = Z^2 = I \quad (3.18)$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY \quad (3.19)$$

$$\{X, Y\} = \{Y, Z\} = \{X, Z\} = 0 \quad (3.20)$$

dove $\{A, B\} = AB + BA$ è l'**anticommutatore**. Le prime due relazioni mostrano che le matrici di Pauli formano l'algebra di Lie $\mathfrak{su}(2)$; la terza — l'anticommutazione — è alla base della struttura dei codici di correzione degli errori quantistici, dove X e Z rappresentano rispettivamente errori di bit flip e phase flip indipendenti.

Porta di Hadamard:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \equiv |+\rangle, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \equiv |-\rangle \quad (3.21)$$

La porta di Hadamard è fondamentale: applicata a n qubit inizializzati in $|0\rangle^{\otimes n}$, genera la **sovrapposizione uniforme** di tutti i 2^n stati base:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (3.22)$$

Porta di fase R_ϕ :

$$R_\phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \quad (3.23)$$

Casi particolari: $T = R_{\pi/4}$ (T-gate) e $S = R_{\pi/2}$ (S-gate o Phase gate).

3.5.2 Porte a Due Qubit

Porta Controlled-NOT (CNOT):

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (3.24)$$

Applica una NOT sul qubit *target* se e solo se il qubit *control* è $|1\rangle$. È la porta a due qubit più comune e, insieme alle porte a singolo qubit, forma un insieme universale.

Porta Toffoli (Controlled-Controlled NOT (porta di Toffoli) (CCNOT)):

Generalizzazione a tre qubit; applica NOT sul terzo qubit solo se entrambi i qubit di controllo sono $|1\rangle$.

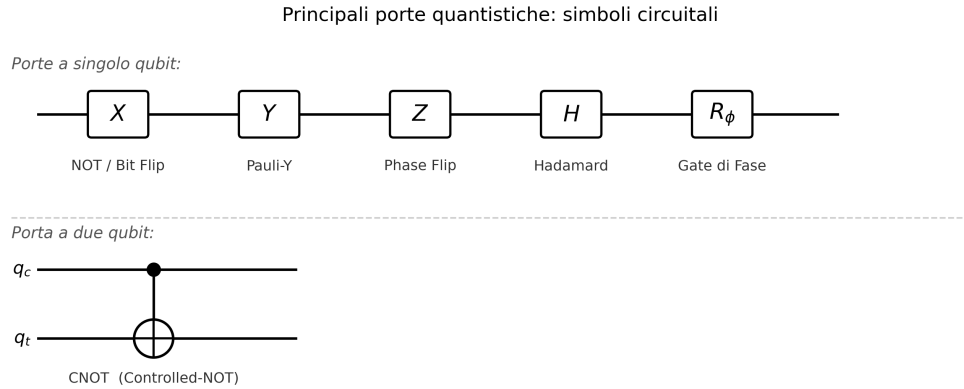


Figura 3.3: Simboli circuitali delle principali porte quantistiche a singolo qubit (X , Y , Z , H , R_ϕ) e della porta CNOT a due qubit. Nella porta CNOT, il punto pieno indica il qubit di controllo q_c e il cerchio con croce indica il qubit target q_t .

3.6 Circuiti Quantistici

Un **circuito quantistico** è una sequenza ordinata di porte quantistiche applicata a un registro di qubit. I circuiti si leggono da sinistra a destra, con ogni filo orizzontale che rappresenta l'evoluzione temporale di un qubit. La composizione di porte corrisponde al prodotto matriciale degli operatori unitari associati.

L'equivalente quantistico del teorema di universalità classico afferma che qualunque trasformazione unitaria su n qubit può essere decomposta in un numero finito di porte da un insieme universale, tipicamente $\{H, T, \text{CNOT}\}$.

3.7 Misura Quantistica

La **misura** in base computazionale di uno stato $|\psi\rangle = \sum_x \alpha_x |x\rangle$ produce il risultato x con probabilità $|\alpha_x|^2$, e collassa irrevocabilmente lo stato in $|x\rangle$. La misura è una operazione *non unitaria e non reversibile*: una volta effettuata, l'informazione sulle ampiezze è irreversibilmente persa.

Questa caratteristica impone che gli algoritmi quantistici siano progettati per amplificare selettivamente l'ampiezza della risposta corretta prima della misura, in modo da ottenere il risultato voluto con alta probabilità.

3.8 Interferenza Quantistica

L'**interferenza** è il meccanismo fondamentale attraverso cui gli algoritmi quantistici realizzano il loro vantaggio computazionale. Le ampiezze quantistiche sono numeri complessi e possono combinarsi costruttivamente (quando hanno lo stesso segno/fase) o distruttivamente (quando hanno fase opposta).

Un algoritmo quantistico efficiente è costruito in modo che:

- Le ampiezze degli stati **sbagliati** si cancellino per **interferenza distruttiva**;
- Le ampiezze degli stati **corretti** si sommino per **interferenza costruttiva**.

Questo meccanismo è alla base sia dell'algoritmo di Grover sia dell'algoritmo di Shor (Capitolo 4), dove la Quantum Fourier Transform realizza l'interferenza necessaria al ritrovamento del periodo.

3.9 Modello di Complessità Quantistica

La complessità di un algoritmo quantistico è misurata in termini di numero di porte (equivalente delle operazioni elementari classiche) e di profondità del circuito (numero di livelli di porte parallele). Si definiscono le classi di complessità quantistiche più rilevanti:

- *Bounded-error Quantum Polynomial time* (BQP): classe dei problemi risolvibili in tempo polinomiale con probabilità di errore al più $1/3$ da un computer quantistico. È la classe centrale del calcolo quantistico.
- *Quantum Merlin-Arthur* (QMA): analogo quantistico di NP; problemi la cui soluzione può essere verificata efficientemente da un computer quantistico.

Si congettura che $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE}$, ma non è ancora dimostrato che BQP strettamente contenga P o BPP, né che sia contenuto strettamente in PSPACE.

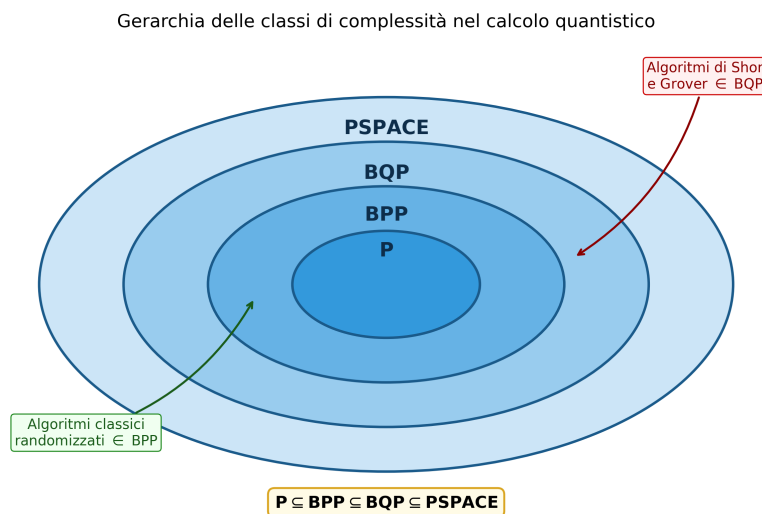


Figura 3.4: Gerarchia congetturata delle classi di complessità nel calcolo quantistico. La classe BQP cattura i problemi efficientemente risolvibili da un computer quantistico; essa contiene P e BPP, ed è contenuta in PSPACE. Gli algoritmi di Shor e Grover risiedono in BQP ma si ritiene (senza dimostrazione) che non appartengano a P.

Capitolo 4

L'Algoritmo di Shor

L'**algoritmo di Shor**, proposto da Peter Shor nel 1994 [7], è un algoritmo quantistico che risolve il problema della **fattorizzazione intera** di un numero N in tempo **polinomiale** nel numero di bit di N :

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (4.1)$$

Questo risultato costituisce uno **speedup esponenziale** rispetto al miglior algoritmo classico noto – il GNFS – che richiede tempo sub-esponenziale (si veda l'equazione (4.2)), del tutto impraticabile per i numeri di grandi dimensioni usati in crittografia. È considerato il risultato algoritmico più dirompente dell'informatica quantistica, poiché la fattorizzazione è il fondamento matematico del sistema crittografico RSA, che protegge la quasi totalità delle comunicazioni digitali moderne.

L'algoritmo si articola in tre fasi: un *preprocessing* classico che riconduce il problema a trovare il periodo di una funzione modulare; un nucleo quantistico basato sulla QFT e sulla QPE, che trova quel periodo in modo esponenzialmente più efficiente di qualunque strategia classica; e una *post-elaborazione* classica che estrae i fattori di N tramite il calcolo del massimo comune divisore. Nelle sezioni seguenti si introduce prima il problema che l'algoritmo risolve e la sua rilevanza crittografica, per poi svilupparne nel dettaglio la struttura.

4.1 Il Problema della Fattorizzazione Intera

La **fattorizzazione intera** consiste nel trovare i fattori primi di un numero intero N . Per numeri piccoli il problema è banale, ma la sua difficoltà cresce in modo impressionante all'aumentare di N : i migliori algoritmi classici noti, come il GNFS, richiedono un tempo sub-esponenziale ma comunque super-polinomiale:¹

$$T_{\text{classico}}(N) = O(\exp((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3})) \quad (4.2)$$

Questo problema è classificato in NP ma non è noto se appartenga a P: non esiste alcun algoritmo classico polinomiale per risolverlo, e si congettura fortemente che non esista.

4.1.1 Rilevanza Crittografica: RSA

Il sistema crittografico **RSA** (Rivest-Shamir-Adleman, 1977) [10] è il più diffuso algoritmo a chiave pubblica, alla base di HTTPS, *Transport Layer Security* (TLS), SSH e di gran parte delle comunicazioni digitali sicure. La sua sicurezza si fonda interamente sulla difficoltà computazionale della fattorizzazione:

- Si scelgono due grandi primi p e q (tipicamente 1024–4096 bit ciascuno);
- Si calcola il modulo $N = p \cdot q$ (la *chiave pubblica* include N ed un esponente e);
- La *chiave privata* è determinata da d tale che $ed \equiv 1 \pmod{\lambda(N)}$, dove λ è la funzione di Carmichael;
- Decifrare un messaggio senza conoscere p e q equivale, in pratica, a fattorizzare N .

Fattorizzare un numero RSA a 2048 bit richiederebbe, con gli algoritmi classici attuali, un tempo stimato superiore all'età dell'universo. Nel 1994, Peter Shor²

¹Il GNFS detiene il record mondiale per la fattorizzazione classica: nel 2020, un consorzio internazionale ha fattorizzato RSA-250 (829 bit, 250 cifre decimali) [12], impiegando l'equivalente di circa 2700 anni-core. In precedenza, nel 2009, era stato fattorizzato RSA-768 (768 bit). I numeri RSA usati in produzione sono di 2048 o 4096 bit, rendendo la fattorizzazione classica del tutto impraticabile con qualunque tecnologia convenzionale prevedibile.

²**Peter Williston Shor** (nato nel 1959 a New York) è professore di matematica applicata al Massachusetts Institute of Technology (MIT). Al momento della scoperta dell'algoritmo lavorava

dimostrò che un computer quantistico può eseguire questa fattorizzazione in tempo *polinomiale*, rendendo RSA teoricamente vulnerabile [7].

4.2 Riduzione alla Ricerca del Periodo

L'intuizione geniale di Shor consiste nel ricondurre la fattorizzazione a un problema di **period finding**, trattabile quantisticamente in modo efficiente.

Osservazione chiave: Dato N e un intero a con $\gcd(a, N) = 1$ (coprimo con N), la funzione

$$f(x) = a^x \bmod N \quad (4.3)$$

è **periodica** con periodo r (detto *ordine* di a modulo N): $f(x+r) = f(x)$ per ogni x .

Una volta trovato il periodo r , se r è pari si può calcolare:

$$\gcd(a^{r/2} - 1, N) \quad \text{e} \quad \gcd(a^{r/2} + 1, N) \quad (4.4)$$

Con alta probabilità, almeno uno di questi due valori è un fattore non banale di N . La prova utilizza proprietà della teoria dei numeri: in particolare, vale $(a^{r/2} - 1)(a^{r/2} + 1) = a^r - 1 \equiv 0 \pmod{N}$.

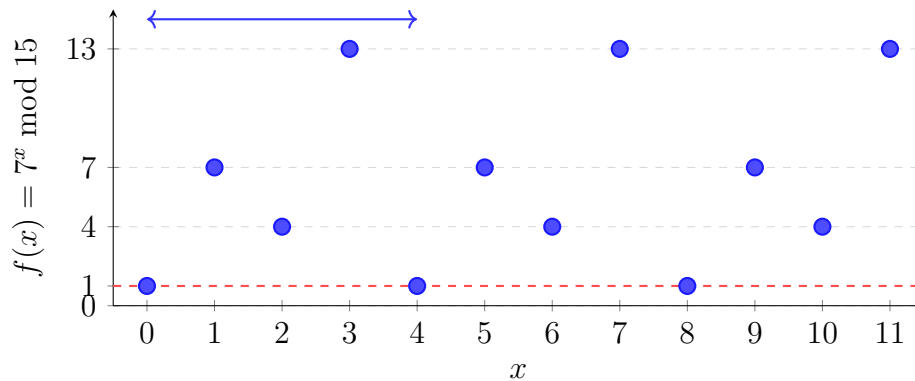


Figura 4.1: La funzione $f(x) = 7^x \bmod 15$ è periodica con periodo $r = 4$. La periodicità è evidente dai valori che si ripetono: 1, 7, 4, 13, 1, 7, 4, 13, ... Trovare r permette di calcolare $\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$ e $\gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$, ottenendo la fattorizzazione $15 = 3 \times 5$.

presso i laboratori di ricerca AT&T Bell Labs. Il suo algoritmo è considerato la dimostrazione più impattante del potere del calcolo quantistico.

4.3 La Quantum Fourier Transform

Il componente quantistico fondamentale dell'algoritmo di Shor è la **Quantum Fourier Transform** (QFT), l'analogo quantistico della Trasformata Discreta di Fourier (Trasformata Discreta di Fourier (*Discrete Fourier Transform*) (DFT)).

4.3.1 Definizione

La QFT su $\mathbb{Z}_{2^n}$ agisce su uno stato base come:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle \quad (4.5)$$

L'azione su uno stato generico $|\psi\rangle = \sum_j \alpha_j |j\rangle$ produce:

$$\text{QFT} |\psi\rangle = \sum_k \hat{\alpha}_k |k\rangle \quad (4.6)$$

dove $\hat{\alpha}_k = \frac{1}{\sqrt{2^n}} \sum_j \alpha_j e^{2\pi i j k / 2^n}$ sono i coefficienti della DFT.

4.3.2 Implementazione Efficiente

La DFT classica su $N = 2^n$ elementi richiede $O(N \log N)$ operazioni (con il Fast Fourier Transform). La QFT, invece, richiede solo $O(n^2) = O((\log N)^2)$ porte quantistiche, ottenendo uno speedup esponenziale.

Il circuito della QFT è costruito ricorsivamente tramite porte di Hadamard e porte di fase controllate CR_k dove:

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{2\pi i/2^k} \end{pmatrix} \quad (4.7)$$

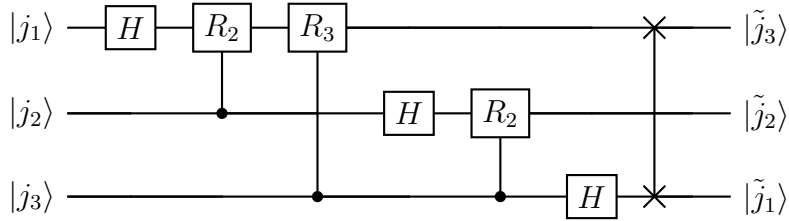


Figura 4.2: Circuito della QFT su $n = 3$ qubit. Ogni qubit riceve una porta di Hadamard H seguita da porte di fase controllate $R_k = \text{diag}(1, e^{2\pi i/2^k})$: la rotazione applicata al qubit j dipende dai qubit successivi. Lo SWAP finale riordina i qubit nell'ordine corretto. La struttura si generalizza a n qubit richiedendo $O(n^2)$ porte.

4.3.3 Phase Estimation

La QFT è impiegata, all'interno dell'algoritmo di Shor, attraverso la subroutine di **Phase Estimation** (QPE). Data una porta U con autovettore $|u\rangle$ e autovalore $e^{2\pi i\theta}$, la QPE stima θ con precisione arbitraria in tempo $O(n^2)$. Nell'algoritmo di Shor, U è l'operatore di moltiplicazione modulare $U|x\rangle = |ax \bmod N\rangle$, e il suo spettro di fase contiene l'informazione sul periodo r .

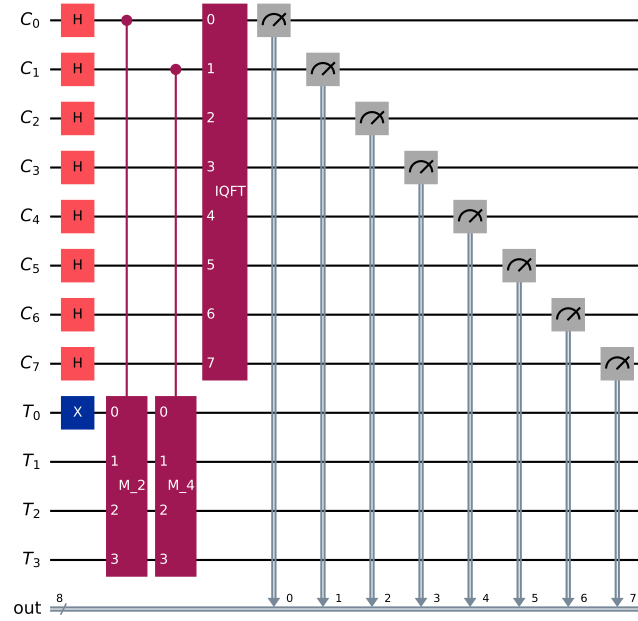


Figura 4.3: Circuito di Quantum Phase Estimation per $N = 15$, $a = 2$, implementato con Qiskit. Il registro di controllo (C , 8 qubit) viene posto in sovrapposizione uniforme, entangolato tramite gli operatori M_2 e M_4 controllati, e trasformato con la QFT^{-1} . La misura del registro C produce una stima della fase $\theta = k/r$, da cui si ricava il periodo r tramite frazioni continue.

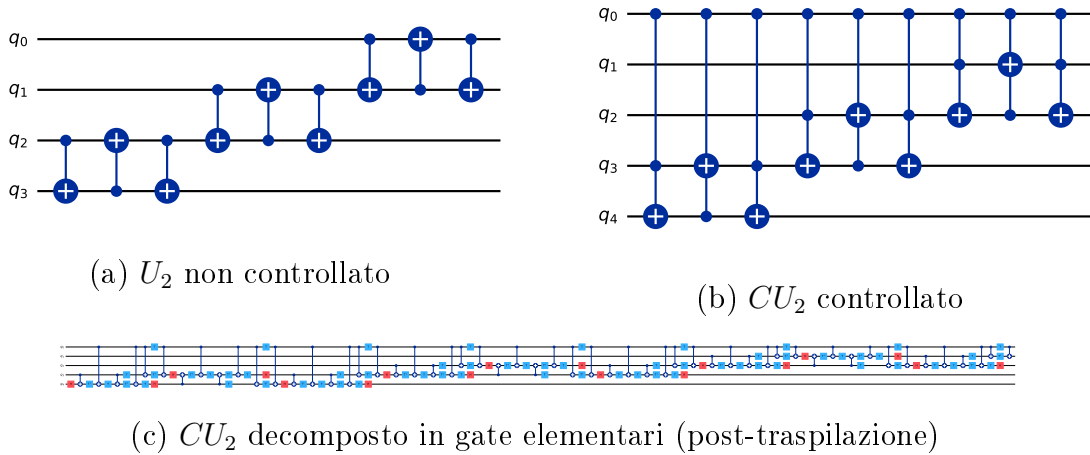


Figura 4.4: Gate di moltiplicazione modulare $U_2|x\rangle = |2x \bmod 15\rangle$ per $N = 15$, $a = 2$. (a) Versione non controllata su 4 qubit: implementata come sequenza di SWAP. (b) Versione controllata CU_2 su 5 qubit utilizzata nel circuito QPE. (c) Decomposizione in gate elementari dopo la traspilazione: le porte SWAP si espandono in triple di CNOT, rivelando il costo effettivo in gate a due qubit del blocco.

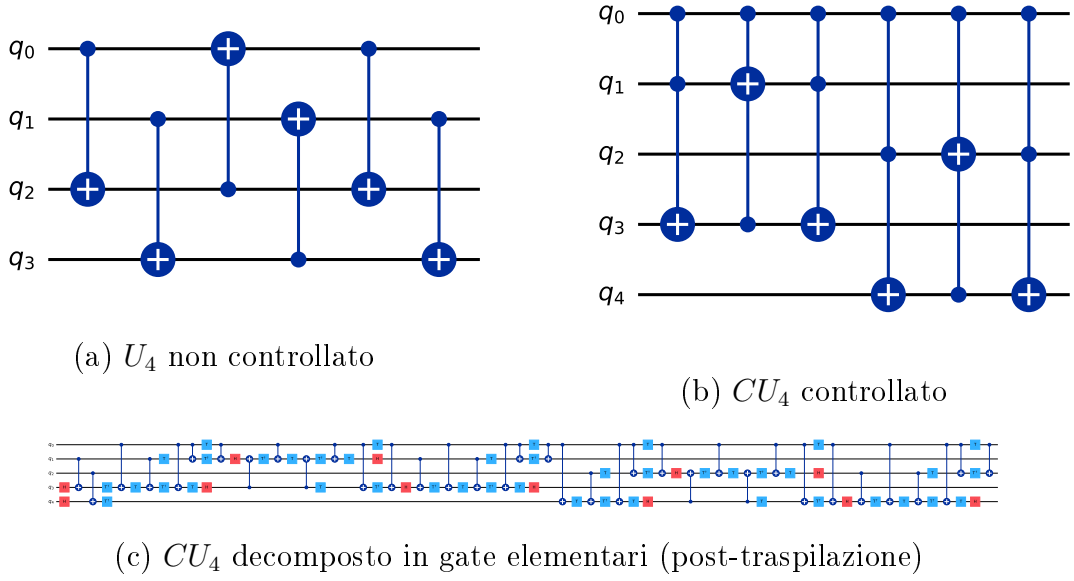


Figura 4.5: Gate di moltiplicazione modulare $U_4 |x\rangle = |4x \bmod 15\rangle$ per $N = 15$, $a = 4 = 2^2$. (a) Versione non controllata: il pattern di SWAP differisce da U_2 poiché la permutazione modulare per $a = 4$ è diversa. (b) Versione controllata CU_4 su 5 qubit, secondo blocco del circuito QPE. (c) Decomposizione in gate elementari: il costo aggiuntivo rispetto a CU_2 riflette la maggiore complessità della permutazione per $a = 4$.

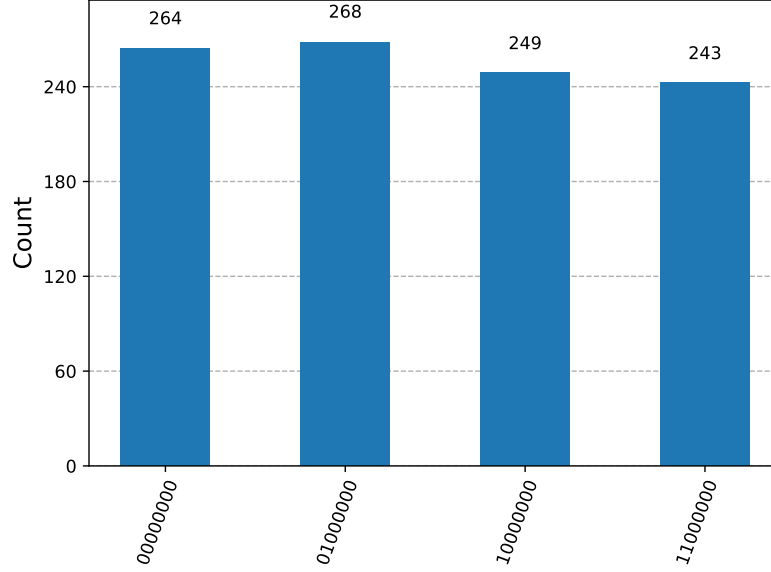


Figura 4.6: Distribuzione di misura del registro di controllo per $N = 15$, $a = 7$, su simulatore a basso rumore (1024 shot). I quattro stati più frequenti — $\{0, 64, 128, 192\}$ — corrispondono ai multipli di $2^8/r = 256/4 = 64$ attesi per il periodo $r = 4$. Ogni picco raccoglie circa 250–270 conteggi su 1024, confermando la struttura teorica della QPE.

4.4 L'Algoritmo di Shor: Schema Completo

L'algoritmo si articola in tre fasi: un preprocessing classico, l'esecuzione del circuito quantistico per il *period finding*, e una post-elaborazione classica per estrarre i fattori.

Fase 1 — Preprocessing classico

1. Scegliere $a \in \{2, \dots, N - 1\}$ uniformemente a caso.
2. Calcolare $g = \gcd(a, N)$: se $g > 1$, restituire g (fattore trovato direttamente).

Fase 2 — Circuito quantistico (period finding)

3. Inizializzare $n = 2\lceil \log_2 N \rceil$ qubit nel registro di controllo: $|0\rangle^{\otimes n} |1\rangle$.
4. Applicare $H^{\otimes n}$ al registro di controllo, creando la sovrapposizione uniforme $\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$.
5. Applicare la moltiplicazione modulare controllata: $|x\rangle |1\rangle \mapsto |x\rangle |a^x \bmod N\rangle$.
6. Misurare il registro di lavoro: il registro di controllo collassa in una sovrapposizione periodica di passo r .

7. Applicare la QFT^{-1} al registro di controllo.
8. Misurare il registro di controllo: ottenere un intero proporzionale a s/r , con $s \in \{0, \dots, r-1\}$.

Fase 3 — Post-elaborazione classica

9. Applicare l'algoritmo delle frazioni continue al risultato della misura per stimare il periodo r .
10. Se r è dispari o $a^{r/2} \equiv -1 \pmod{N}$: tornare al passo 1 con un nuovo valore di a .
11. Calcolare $p = \gcd(a^{r/2} - 1, N)$ e $q = \gcd(a^{r/2} + 1, N)$.
12. Se p o q è un fattore non banale di N : restituirlo.
Altrimenti: tornare al passo 1.

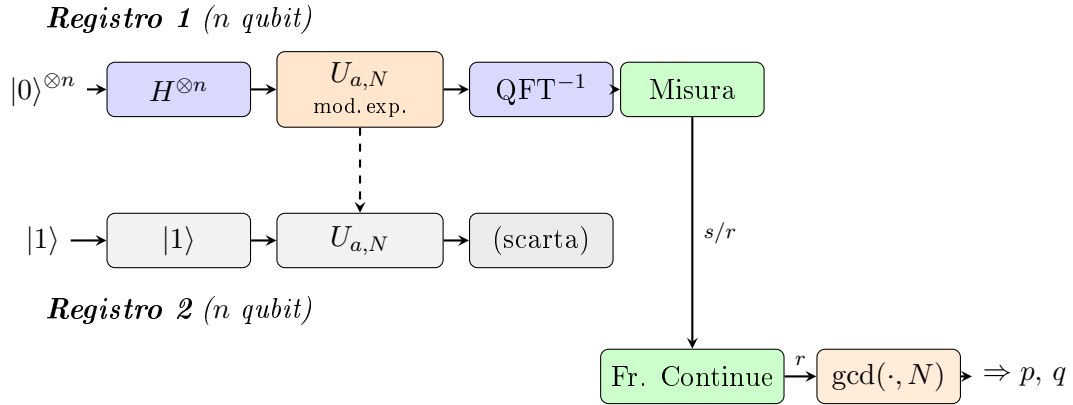


Figura 4.7: Schema a due registri dell'algoritmo di Shor. Il **Registro 1** (di controllo) viene posto in sovrapposizione uniforme con $H^{\otimes n}$, entangolato con il Registro 2 tramite la moltiplicazione modulare $U_{a,N}$, quindi trasformato con la QFT^{-1} . La misura produce un valore s/r elaborato classicamente: le frazioni continue stimano il periodo r , e il calcolo del MCD restituisce i fattori p e q di N .

4.4.1 Il Ruolo dell'Entanglement nel Period Finding

Il collasso descritto al passo 6 dello schema non è un effetto arbitrario: è la conseguenza diretta dell'**entanglement** creato al passo 5. Vale la pena rendere esplicito questo meccanismo, che è il cuore quantistico dell'intero algoritmo.

Dopo l'applicazione della moltiplicazione modulare controllata, i due registri si trovano nello stato entangled:

$$|\Psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |a^x \bmod N\rangle \quad (4.8)$$

Questo stato **non è separabile**: il valore del registro di lavoro è correlato in modo univoco al valore del registro di controllo. I due registri formano un unico sistema entangled, esattamente come accade in uno stato di Bell — dove misurare un qubit determina istantaneamente l'esito dell'altro.

Se a questo punto si misurasse il registro di lavoro e si ottenesse un valore $v = a^{x_0} \bmod N$, il registro di controllo collasserebbe *istantaneamente* nell'unica sovrapposizione compatibile con v : quella di tutti i valori x per cui $a^x \equiv v \pmod{N}$, ovvero

$$|x_0\rangle + |x_0 + r\rangle + |x_0 + 2r\rangle + \dots \quad (4.9)$$

La correlazione tra i due registri è **bidirezionale**: conoscere il valore del registro di lavoro vincola il registro di controllo a una sovrapposizione periodica di passo esattamente r . Questa è la “versione inversa” propria dell'entanglement: misurare uno dei due sottosistemi determina (o in questo caso, restringe) lo stato dell'altro.

In pratica il circuito di Shor non misura esplicitamente il registro di lavoro: la sola creazione dell'entanglement è sufficiente. Applicando la QFT^{-1} al registro di controllo prima della misura, si trasforma la periodicità nascosta dell'equazione (4.9) in picchi netti alle frequenze k/r , con $k \in \{0, \dots, r-1\}$. La misura finale del registro di controllo restituisce allora, con alta probabilità, un valore da cui le frazioni continue estraggono r .

In sintesi: l'entanglement tra i due registri è il meccanismo che *trasferisce* la periodicità della funzione $f(x) = a^x \bmod N$ — che esiste nel registro di lavoro — verso il registro di controllo, dove la QFT la rende misurabile. Senza entanglement, questa trasmissione di informazione tra i due registri sarebbe impossibile, e l'intero speedup esponenziale di Shor verrebbe meno.

4.5 Analisi della Complessità

La complessità dell'algoritmo di Shor è **polinomiale** nel numero di bit di N :

- Il circuito quantistico richiede $O((\log N)^2 \log \log N)$ porte elementari;
- Il numero di qubit necessari è $O(\log N)$; l'implementazione ottimizzata di Beauregard [28] riduce questo a $2n + 3$ qubit tramite addizione modulare in-place;
- Il numero atteso di iterazioni (con basi a diverse) prima del successo è $O(1)$.

Complessivamente: $T_{\text{Shor}}(N) = O((\log N)^3)$, a fronte del $O(\exp((\log N)^{1/3}))$ del miglior algoritmo classico. Si tratta di uno **speedup esponenziale**.

4.6 Impatto sulla Crittografia e Prospettive Post-Quantum

L'algoritmo di Shor rende teoricamente vulnerabili tutti i sistemi crittografici basati sulla difficoltà della fattorizzazione (RSA) o del logaritmo discreto (Diffie-Hellman, *Digital Signature Algorithm* (DSA), ECDSA).

La *Post-Quantum Cryptography* (PQC) è il campo emergente che sviluppa algoritmi crittografici resistenti agli attacchi quantistici.

Il processo di standardizzazione PQC del NIST, avviato nel 2016 con una call pubblica per la raccolta di proposte algoritmiche, si è concluso con la pubblicazione formale dei primi standard federali nell'agosto 2024 [14]:³

- **FIPS 203 – ML-KEM** (basato su CRYSTALS-Kyber): standard per l'incapsulamento di chiave, basato su reticoli modulari;
- **FIPS 204 – ML-DSA** (basato su CRYSTALS-Dilithium): standard per la firma digitale su reticoli;
- **FIPS 205 – SLH-DSA** (basato su SPHINCS+): standard per la firma digitale basata su funzioni hash;

³Dopo otto anni di valutazione e quattro round di selezione pubblica con la partecipazione di crittografi di tutto il mondo, il NIST ha pubblicato nel 2024 i FIPS 203, 204 e 205. Un quarto standard, FIPS 206 (FN-DSA, basato su FALCON), è in fase di finalizzazione. I nuovi nomi riflettono le famiglie algoritmiche di appartenenza: ML-KEM (Module-Lattice Key Encapsulation), ML-DSA (Module-Lattice Digital Signature), SLH-DSA (Stateless Hash-based Digital Signature).

- **FIPS 206 – FN-DSA** (basato su FALCON): standard in fase di finalizzazione per la firma digitale su reticoli NTRU.

Questi sistemi si basano su problemi matematici (reticoli, hash, codici) per i quali non sono noti algoritmi quantistici efficienti, garantendo sicurezza anche nell'era post-quantum. La transizione verso questi standard è urgente anche per il cosiddetto fenomeno *harvest now, decrypt later*: attori malevoli raccolgono oggi comunicazioni cifrate con RSA, con l'intenzione di decifrarle quando saranno disponibili computer quantistici fault-tolerant.

4.7 Sviluppi Recenti e Stato dell'Arte

4.7.1 Implementazioni su Hardware Reale

Nonostante la solidità teorica, l'esecuzione di Shor su hardware reale rimane limitata a istanze molto piccole a causa dei requisiti di profondità circuitale. I principali risultati sperimentali sono:

- **2001 – NMR (Vandersypen et al.)**: prima dimostrazione sperimentale di Shor su un sistema a 7 spin nucleari; fattorizzazione di $N = 15$ in 3×5 [24].
- **2016 – Ioni intrappolati (Monz et al.)**: implementazione scalabile su 5 qubit ionici; fattorizzazione di $N = 15$ con un'architettura modulare [29].
- **2021 – Qubit superconduttori IBM**: dimostrazione della fattorizzazione di $N = 21$ su processori IBM Quantum utilizzando solo 5 qubit [25], attualmente il risultato più avanzato su hardware superconduttore di uso generale;
- **Fotonica**: esperimenti su piattaforme fotoniche hanno raggiunto la fattorizzazione di $N = 35$, sfruttando la maggiore coerenza dei qubit ottici, sebbene con circuiti fortemente semplificati rispetto all'algoritmo generale.

Il record di $N = 21$ evidenzia quanto la distanza tra le istanze dimostrabili e quelle crittograficamente rilevanti (RSA-2048, con N a 2048 bit) sia ancora enorme: fattorizzare un numero a L bit richiede circa $5L + 1$ qubit logici e $72L^3$ gate [30], valori del tutto inaccessibili ai processori NISQ attuali. Una analisi aggiornata delle sfide pratiche nell'esecuzione di Shor su piattaforme esistenti è discussa in [30].

4.7.2 Miglioramenti Algoritmici: Regev 2023

Un risultato teorico rilevante è stato pubblicato nel 2023 da Oded Regev [31], che ha proposto una variante migliorata dell'algoritmo di Shor richiedente solo $O(n^{3/2})$ gate (rispetto agli $O(n^2)$ della versione originale), al costo di un overhead classico maggiore. Sebbene non ancora implementata su hardware reale, questa riduzione della profondità circuitale è promettente per le architetture NISQ, dove la profondità è il principale collo di bottiglia.

4.7.3 Roadmap IBM verso il Quantum Fault-Tolerant

IBM ha delineato una roadmap concreta verso il calcolo quantistico fault-tolerant [18]:

- **2025:** processori Loon e Nighthawk (120 qubit con 218 coupler sintonizzabili), con dimostrazione dei componenti chiave per la fault-tolerance tramite codici qLDPC;
- **2026:** target di vantaggio quantistico su problemi pratici;
- **2029:** sistema Quantum Starling con 200 qubit logici e oltre 10^8 operazioni quantistiche.

Questi traguardi definiscono l'orizzonte temporale entro cui l'algoritmo di Shor potrà diventare una minaccia concreta per la crittografia RSA, motivando ulteriormente la transizione urgente verso gli standard PQC discussi nella sezione precedente.

Le sfide hardware principali per l'esecuzione di Shor su istanze crittograficamente rilevanti sono analizzate in dettaglio nel Capitolo 5, mentre l'implementazione pratica con Qiskit e le tecniche di ottimizzazione basate su intelligenza artificiale sono presentate nel Capitolo 9.

4.8 Contesto: Confronto con l'Algoritmo di Grover

Per collocare correttamente l'algoritmo di Shor nel panorama dell'informatica quantistica, è utile confrontarlo con l'altro grande risultato algoritmico degli anni Novanta: l'algoritmo di Grover per la ricerca non strutturata [8].

4.8.1 Il Problema di Grover

Dato un insieme di N elementi e una funzione oracolo $f : \{0, \dots, N-1\} \rightarrow \{0, 1\}$, l'obiettivo è trovare x^* tale che $f(x^*) = 1$. Classicamente, nel caso peggiore, sono necessarie $O(N)$ interrogazioni. Grover (1996) dimostrò che un algoritmo quantistico, sfruttando il meccanismo di *amplitude amplification*, può trovare la soluzione in sole $O(\sqrt{N})$ interrogazioni – uno **speedup quadratico** che è stato dimostrato ottimale: nessun algoritmo quantistico può fare meglio [32, 9].

4.8.2 Differenze Fondamentali

I due algoritmi rappresentano due regimi distinti di vantaggio quantistico:

Caratteristica	Shor	Grover
Problema target	Fattorizzazione intera / periodo finding	Ricerca non strutturata
Speedup	Esponenziale rispetto al miglior classico	Quadratico: $O(\sqrt{N})$ vs $O(N)$
Tecnica quantistica core	QFT + Phase Estimation	Amplitude Amplification
Ottimalità	Non dimostrata in assoluto	Dimostrata ottimale [32]
Impatto crittografico	Rompe RSA, DH, ECDSA (chiave pubblica)	Dimezza sicurezza AES/SHA (simmetrica)
Requisiti hardware	$O(\log N)$ qubit, alta profondità	$O(\log N)$ qubit, bassa profondità

Tabella 4.1: Confronto tra l'algoritmo di Shor e l'algoritmo di Grover

L'impatto sulla crittografia simmetrica di Grover è significativo ma gestibile: raddoppiare la lunghezza delle chiavi (da AES-128 a AES-256) è sufficiente a ripristinare il livello di sicurezza. Al contrario, non esiste un analogo rimedio per RSA: l'unica risposta è la migrazione verso gli standard post-quantum discussi nella sezione precedente. È per questa ragione che l'algoritmo di Shor rappresenta la minaccia quantistica più urgente e motivante per la ricerca sul calcolo quantistico applicato.

Tutta l'analisi svolta in questo capitolo ha però assunto un modello di calcolo *ideale*: porte unitarie perfette, coerenza illimitata, nessuna interazione con l'ambiente. Su hardware reale questi presupposti non reggono, e la distanza tra il modello teorico e l'esecuzione fisica è precisamente il nodo critico che rende ancora irraggiungibile la fattorizzazione di numeri crittograficamente rilevanti. Comprendere le origini e gli effetti del rumore quantistico è quindi il passo necessario per valutare le reali prospettive dell'algoritmo di Shor.

Capitolo 5

Rumore Quantistico nei Sistemi NISQ

5.1 Introduzione al Rumore Quantistico

I computer quantistici reali operano in regime NISQ [19]: dispongono di decine o centinaia di qubit fisici, ma sono affetti da errori significativi che limitano la profondità dei circuiti eseguibili in modo affidabile.

Il rumore quantistico differisce fondamentalmente dal rumore nei sistemi classici: mentre un bit classico può essere protetto ridondando l'informazione, un qubit non può essere copiato (**teorema del no-cloning**) [16]. Inoltre, gli errori quantistici non sono binari — un qubit può subire rotazioni arbitrarie nello spazio di Hilbert — rendendo la loro caratterizzazione intrinsecamente più ricca e più difficile di quella classica.

Per analizzare il rumore in modo rigoroso è utile distinguere tre livelli di descrizione, che il presente capitolo tratta separatamente:

- **Anatomia della macchina** (Sezione 5.2): com'è fatta fisicamente una QPU e *dove*, nei suoi componenti, gli errori hanno origine;
- **Fonti fisiche** (Sezione 5.3): i quattro meccanismi concreti con cui l'hardware introduce errori;
- **Modelli matematici** (Sezione 5.5): i quattro canali quantistici con cui tali errori vengono formalizzati e simulati.

5.2 L'Anatomia Fisica di una QPU

Per capire dove nascono gli errori bisogna prima capire com'è fatta la macchina. Una QPU superconduttiva è un chip criogenico che contiene *esclusivamente* i qubit e le linee fisiche che li collegano: contrariamente all'intuizione, **le porte logiche non risiedono nel chip**. Aprendo una CPU classica si trovano registri e porte logiche incise nel silicio; aprendo una QPU si trovano solo i qubit. Le operazioni sono realizzate dall'**elettronica di controllo esterna**, che invia impulsi a microonde convogliati ai qubit tramite guide d'onda: “far passare un qubit in un gate” significa in realtà inviargli un'onda sagomata che ne ruota lo spin dell'angolo desiderato (90° , 30° , 10° , ...). La Figura 5.1 schematizza questa architettura per una QPU a 5 qubit con topologia a catena lineare.

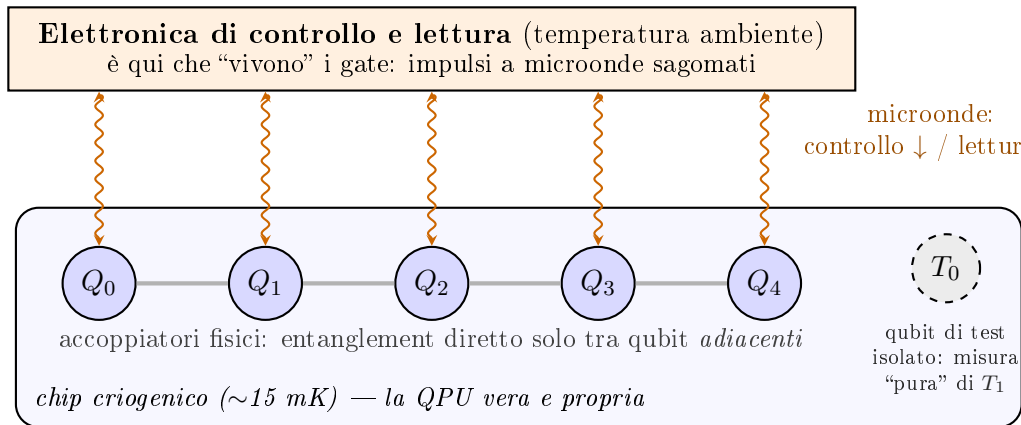


Figura 5.1: Anatomia di una QPU superconduttiva a 5 qubit con topologia a catena lineare. Nel chip criogenico risiedono solo i qubit e gli accoppiatori fisici; le porte logiche sono impulsi a microonde generati dall'elettronica esterna e convogliati tramite guide d'onda, usate sia per il controllo sia per la lettura. L'entanglement diretto è possibile solo tra qubit adiacenti (Q_1 – Q_2 sì, Q_0 – Q_2 solo tramite operazioni intermedie, che amplificano il disturbo). Il qubit di test T_0 , privo di collegamenti, serve a misurare il tempo di coerenza “puro”: un qubit collegato a una guida d'onda perde sempre un po' di energia attraverso di essa.

La Figura 5.2 mostra come questa architettura si presenta nella realtà: all'esterno il sistema integrato (criostato, elettronica e schermature racchiusi nella teca), all'interno della sala macchine i cilindri criogenici sospesi con il fascio di tubi e cavi coassiali che convogliano le microonde di controllo e lettura — le “guide d'onda” dello schema precedente.



(a) IBM Quantum System One (Ehningen, Germania)



(b) Criostati IBM Q con il cablaggio di controllo

Figura 5.2: La QPU reale. (a) Il sistema integrato IBM Quantum System One: il cilindro nero al centro è il criostato che contiene il chip, circondato dall'elettronica di controllo. (b) Criostati IBM Q in sala macchine: dall'alto arrivano i tubi per la refrigerazione e i cavi coassiali che convogliano le microonde di controllo e lettura verso il chip, sospeso sul fondo del cilindro a ~ 15 mK. *Crediti: (a) IBM Research, licenza CC BY 2.0; (b) Connie Zhou per IBM, pubblico dominio; entrambe via Wikimedia Commons.*

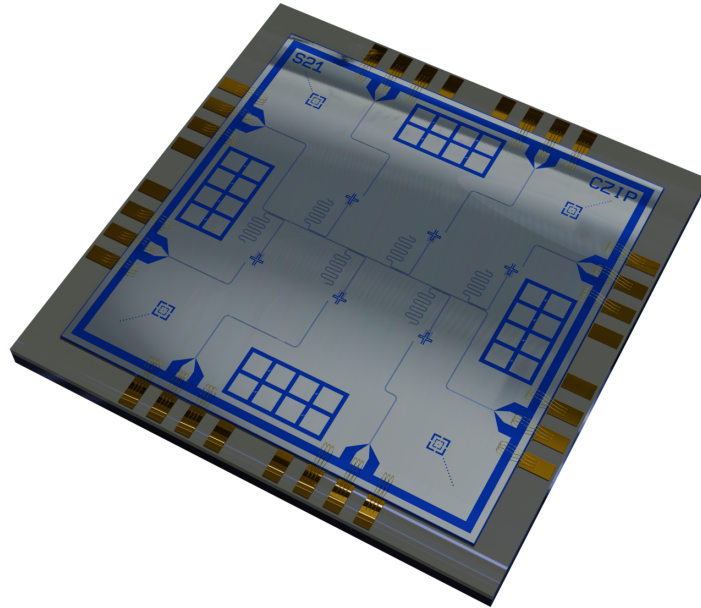


Figura 5.3: Resa tridimensionale di un chip quantistico superconduttivo con 6 qubit transmon e 12 giunzioni di test. Si riconoscono gli elementi dello schema di Figura 5.1: i qubit con i loro risonatori, le linee di accoppiamento, le piazzole dorate per il collegamento alle guide d'onda e — ai quattro angoli — le strutture di test isolate, l'analogo reale del qubit T_0 . *Credito: Onri Jay Benally, licenza CC BY 4.0, via Wikimedia Commons.*

Questa architettura ha tre conseguenze dirette per gli errori.

La lettura distrugge lo stato — fisicamente. Anche la misura avviene tramite microonde: per leggere un qubit lo si “bombarda” con un’onda di sonda e si osserva come essa torna indietro — un modo di risposta corrisponde a $|0\rangle$, un altro a $|1\rangle$. L’atto stesso di sondare blocca la dinamica del qubit: è per questo che la misura di uno stato quantistico lo distrugge *realmente*, non per convenzione matematica. Ed è per questo che nei circuiti le letture stanno tutte alla fine, in parallelo: misurare un solo qubit lasciando gli altri in sovrapposizione significa inviare un’onda che, per quanto focalizzata, perturba anche i vicini e può farli precipitare.

Il budget di operazioni: coerenza vs velocità dei gate. Le prestazioni di una QPU dipendono dal rapporto tra due tempi: quanto a lungo il qubit resta coerente e quanto dura una singola operazione. Con una finestra di coerenza di $100\ \mu\text{s}$ e gate da $\sim 25\ \text{ns}$, si eseguono $\sim 4\,000$ operazioni; una macchina con coerenza superiore ma

gate più lenti può eseguirne *meno*. È il rapporto che conta, non il singolo parametro — ed è questo rapporto a decidere se il circuito di Shor per una data istanza è fisicamente eseguibile. La Tabella 5.1 confronta gli ordini di grandezza delle due principali tecnologie.

	Superconduttori (IBM, IQM, ...)	Ioni intrappolati (IonQ, Quantinuum, ...)
Tempo di coerenza	50–300 μ s	da secondi a minuti
Durata gate (1 qubit)	$\sim$ 15–40 ns	$\sim$ 1–100 μ s
Durata gate (2 qubit)	$\sim$ 60–500 ns	$\sim$ 0.1–1 ms
Operazioni per finestra	$\sim 10^3$ – 10^4	$\sim 10^3$
Lettura	più esposta a errori e interferenze	più stabile e affidabile
Crosstalk	presente (accoppiamento capacitivo)	ridotto
Velocità di esecuzione	molto alta	bassa (gate lenti)

Tabella 5.1: Ordini di grandezza indicativi delle due principali tecnologie di QPU. La coerenza lunghissima degli ioni intrappolati non si traduce in più operazioni utili, perché i gate sono da 10^3 a 10^4 volte più lenti: il budget di operazioni per finestra di coerenza è simile, ma i superconduttori lo spendono molto più in fretta. Gli ioni sono invece più robusti su lettura e crosstalk.

Il bordo di coerenza. Gli errori non sono distribuiti uniformemente nel tempo: finché il calcolo occupa una piccola frazione della finestra di coerenza, la probabilità di decadimento è bassa; quando il circuito si spinge *verso la fine* della finestra — il “bordo di coerenza” — la probabilità che il qubit sia già decaduto al momento della misura cresce rapidamente. La Figura 5.4 illustra il concetto: è la ragione per cui circuiti profondi come quello di Shor, che consumano gran parte del budget, producono letture inaffidabili anche quando “tecnicamente” rientrano nella finestra.

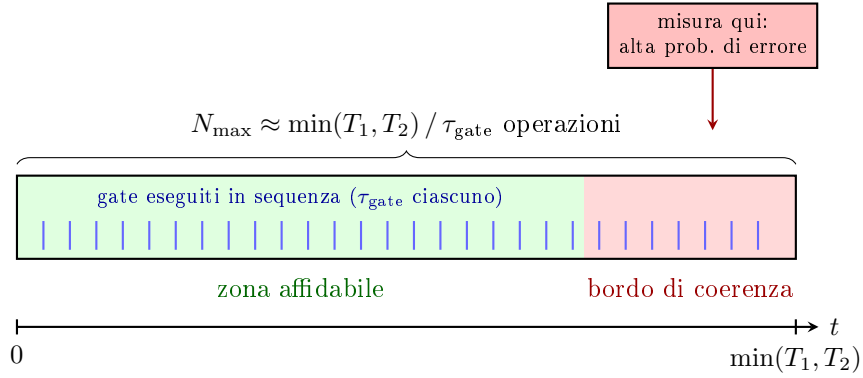


Figura 5.4: Il “budget” di operazioni di un qubit: la finestra di coerenza $\min(T_1, T_2)$ divisa per la durata del singolo gate dà il numero massimo di operazioni eseguibili $N_{\max}$. Un circuito che consuma solo la prima parte della finestra opera in zona affidabile; un circuito profondo come quello di Shor spinge la misura verso il *bordo di coerenza*, dove la probabilità che il qubit sia già decaduto — e che la lettura restituisca un valore errato — è massima.

La topologia decide chi parla con chi. Le QPU differiscono anche per geometria: catene lineari, griglie (heavy-hex di IBM), connettività completa. Su una catena, l’entanglement tra qubit non adiacenti richiede un “cross-entanglement” attraverso i qubit intermedi: ogni passaggio disperde energia e accorcia la vita dello stato condiviso — più cross si fanno, prima lo stato decade. La topologia determina quindi sia *quali* circuiti si possono mappare in modo efficiente, sia *dove* il crosstalk colpirà. Questo aspetto — insieme al fatto che i parametri di coerenza variano da qubit a qubit dello stesso chip — è ripreso nel Capitolo 12, dove la località delle interazioni sulla griglia è il fondamento architetturale del surface code.

5.3 Le Quattro Fonti Fisiche di Rumore

Sui processori quantistici reali il rumore origina da quattro meccanismi distinti. Ognuno agisce su una scala temporale e spaziale diversa, e richiede strategie di mitigazione differenti.

5.3.1 Decoerenza (T_1 e T_2)

La **decoerenza** è il processo per cui un qubit perde le sue proprietà quantistiche a causa dell'interazione inevitabile con l'ambiente circostante (campi elettromagnetici parassiti, fononi, fluttuazioni termiche). Si manifesta attraverso due tempi caratteristici:

- T_1 – **Relaxation time**: il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ per dissipazione energetica. Fisicamente corrisponde all'emissione di un fotone verso l'ambiente. Su hardware superconduttore tipicamente $T_1 \sim 50\text{--}500\ \mu\text{s}$.¹
- T_2 – **Dephasing time**: la fase relativa tra $|0\rangle$ e $|1\rangle$ si randomizza senza necessariamente modificare le popolazioni, distruggendo la sovrapposizione. T_2 comprende due contributi:

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi} \quad (5.1)$$

dove T_ϕ è il tempo di *pura* perdita di fase. Vale sempre $T_2 \leq 2T_1$; tipicamente $T_2 \sim 50\text{--}200\ \mu\text{s}$.

La condizione necessaria per eseguire correttamente un circuito è:

$$\tau_{\text{circuito}} \ll \min(T_1, T_2) \quad (5.2)$$

Poiché l'algoritmo di Shor per un intero a n bit richiede una profondità $O(n^3)$ gate, e ogni gate a due qubit su hardware IBM dura $\sim 300\text{--}500\ \text{ns}$, questa condizione è violata già per $n \gtrsim 10\text{--}15$ bit sui processori NISQ attuali.

Connessione con l'entanglement. T_2 ha un'interpretazione diretta in termini di entanglement: esso misura per quanto tempo i qubit del registro di controllo

¹La notazione T_1 e T_2 è mutuata dalla spettroscopia a NMR, dove Felix Bloch la introdusse nel 1946 per i tempi di rilassamento longitudinale e trasversale della magnetizzazione nucleare.

riescono a mantenere la coerenza di fase necessaria a sostenere lo stato entangled con il registro di lavoro. Nella phase estimation dell'algoritmo di Shor, ogni qubit di controllo q_j viene preparato in sovrapposizione $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ e deve restare coerente per tutta la durata del blocco di modular exponentiation controllato da q_j . Se T_2 è inferiore a questo intervallo, la fase relativa si randomizza: l'entanglement con il registro di lavoro decade, le ampiezze di interferenza nella QFT si riducono, e il periodo r non è più estraibile in modo affidabile. Su `ibm_marrakesh`, con $T_2 \approx 80 \mu s$ e un tempo di gate a due qubit di circa 400 ns, il numero massimo di gate a due qubit eseguibili entro T_2 è dell'ordine di $T_2/\tau_{\text{gate}} \approx 200$: una soglia che il circuito di Shor per $N = 15$ sfiora già in condizioni ideali di compilazione [23].

5.3.2 Errori di Gate

Le porte quantistiche fisiche non implementano perfettamente l'operazione unitaria ideale U , bensì un'operazione perturbata $\tilde{U} = U + \delta E$. L'**errore di gate** è quantificato dalla **gate infidelity**:

$$r(U, \tilde{U}) = 1 - F(U, \tilde{U}), \quad F(U, \tilde{U}) = \frac{1}{d} \left| \text{Tr}(U^\dagger \tilde{U}) \right| \quad (5.3)$$

dove d è la dimensione dello spazio di Hilbert. Le cause principali sono le imprecisioni nel controllo di ampiezza e frequenza dei pulse a microonde, l'accoppiamento residuo *always-on* tra qubit adiacenti, e il *leakage* verso stati non computazionali ($|2\rangle$, $|3\rangle$, ...) negli oscillatori di Josephson. Su hardware IBM (2024–2025) i valori tipici sono: $r \approx 0.03\text{--}0.1\%$ per gate a singolo qubit ($F \approx 99.9\text{--}99.97\%$) e $r \approx 0.3\text{--}1\%$ per gate a due qubit come CNOT ed ECR ($F \approx 99\text{--}99.7\%$).

5.3.3 Errori di Misura (Readout Errors)

La misura finale può restituire un risultato errato: il qubit si trova in $|0\rangle$ ma viene letto come 1, o viceversa. Questo errore è modellato da una **matrice di confusione** A :

$$A_{ij} = P(\text{misura } i \mid \text{stato } j), \quad \sum_i A_{ij} = 1 \quad (5.4)$$

Per n qubit indipendenti, $A = \bigotimes_{k=1}^n A_k$ con

$$A_k = \begin{pmatrix} 1 - e_0^{(k)} & e_1^{(k)} \\ e_0^{(k)} & 1 - e_1^{(k)} \end{pmatrix} \quad (5.5)$$

dove $e_0^{(k)}$ è la probabilità di leggere 1 dato $|0\rangle$ per il k -esimo qubit. Valori tipici: $e_0, e_1 \approx 1\text{--}3\%$ su hardware IBM.

5.3.4 Crosstalk

Il **crosstalk** è l'interferenza indesiderata tra qubit adiacenti durante l'esecuzione di operazioni parallele. Origina dall'accoppiamento capacitivo residuo tra qubit vicini nel chip superconduttore: quando si applica un gate su un qubit, le perturbazioni del campo si propagano ai qubit circostanti causando rotazioni parassite non intenzionali. Il crosstalk è particolarmente dannoso nei circuiti profondi con molte operazioni parallele ed è difficile da modellare analiticamente, il che rende i modelli di apprendimento automatico particolarmente adatti a caratterizzarlo.

Fonte	Causa fisica	Parametro	Valore tipico IBM
Decoerenza	Interazione con l'ambiente	T_1, T_2	50–500 μs
Errori di gate	Imprecisione pulse / leakage	Gate infidelity	0.03–1%
Readout errors	Risposta del rivelatore	e_0, e_1	1–3%
Crosstalk	Accoppiamento residuo	Difficile da parametrizzare	$\sim 0.1\text{--}1\%$

Tabella 5.2: Riepilogo delle quattro fonti fisiche di rumore nei processori quantistici superconduttori.

5.4 Dal Fenomeno Fisico al Modello Matematico

I quattro fenomeni fisici descritti nella sezione precedente non sono isolati: ciascuno può essere ricondotto a uno o più canali matematici, che ne forniscono una descrizione quantitativa precisa e simulabile. La tabella seguente stabilisce il collegamento esplicito tra i due livelli di descrizione.

Fonte fisica	Canale matematico	Parametro chiave
Decadimento energetico (T_1)	Amplitude Damping	$\gamma = 1 - e^{-t/T_1}$
Perdita di fase (T_2)	Phase Flip	$p = \frac{1}{2}(1 - e^{-t/T_2})$
Errori di gate (isotropici)	Depolarizzante	$p \approx \text{gate infidelity}$
Errori di gate (asimmetrici)	Bit Flip / Phase Flip	p dal profilo dell'errore
Readout errors	Matrice di confusione A	e_0, e_1 da calibrazione
Crosstalk	Depolarizzante (multi-qubit)	Difficile da parametrizzare

Tabella 5.3: Corrispondenza tra fonti fisiche di rumore e canali quantistici.

In pratica, il simulatore Qiskit Aer combina amplitude damping e phase flip in un unico **canale di thermal relaxation**, parametrizzato direttamente da T_1 , T_2 e dal tempo di gate τ , per riprodurre fedelmente il comportamento dell'hardware IBM.

5.5 I Quattro Canali Matematici del Rumore

Per simulare e analizzare il rumore, le fonti fisiche vengono astratte in **canali quantistici**: mappe lineari *Completely Positive Trace-Preserving* (CPTP) che agiscono sulla matrice densità ρ del sistema [26]. Ogni canale CPTP ammette la **rappresentazione di Kraus**:

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I \quad (5.6)$$

I quattro canali fondamentali, ciascuno dei quali cattura un diverso regime di errore, sono presentati di seguito.

5.5.1 Canale Bit Flip

Il canale **bit flip** modella un errore di tipo X discreto: con probabilità p il qubit viene invertito ($|0\rangle \leftrightarrow |1\rangle$), con probabilità $1 - p$ rimane invariato. Gli operatori di Kraus sono $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} X$, e il canale agisce come:

$$\mathcal{E}_{BF}(\rho) = (1-p)\rho + p X \rho X \quad (5.7)$$

Sulla sfera di Bloch, le componenti r_y e r_z vengono contratte di un fattore $(1-2p)$ mentre la componente r_x rimane invariata. Per $p = 1/2$ lo stato è completamente distrutto sul piano yz .

5.5.2 Canale Phase Flip

Il canale **phase flip** modella la perdita di informazione di fase senza cambiamento di popolazione: la sovrapposizione decade senza che l'energia venga dissipata. Con operatori di Kraus $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} Z$, il canale è:

$$\mathcal{E}_{PF}(\rho) = (1-p)\rho + p Z \rho Z \quad (5.8)$$

L'effetto sulla matrice densità è immediato:

$$\mathcal{E}_{PF} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} = \begin{pmatrix} \rho_{00} & (1-2p)\rho_{01} \\ (1-2p)\rho_{10} & \rho_{11} \end{pmatrix} \quad (5.9)$$

Le popolazioni ρ_{00} e ρ_{11} rimangono invariate, mentre le **coerenze** ρ_{01} , ρ_{10} vengono sopprese di un fattore $(1 - 2p)$. Questo è il canale che descrive il decadimento T_2 : parametrizzando $p(t) = \frac{1}{2}(1 - e^{-t/T_2})$, si ottiene $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2}$.

5.5.3 Canale Depolarizzante

Il canale **depolarizzante** è il modello isotropico più usato per gli errori di gate: con probabilità p viene applicato un errore casuale tra $\{X, Y, Z\}$ con uguale probabilità $p/3$. Gli operatori di Kraus sono $K_0 = \sqrt{1-p} I$, $K_1 = \sqrt{p/3} X$, $K_2 = \sqrt{p/3} Y$, $K_3 = \sqrt{p/3} Z$, e il canale è:

$$\mathcal{E}_{dep}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z) \quad (5.10)$$

Usando l'identità $X\rho X + Y\rho Y + Z\rho Z = 2I/2 - \rho$ (valida per $\text{Tr}(\rho) = 1$), si ottiene la forma compatta:

$$\mathcal{E}_{dep}(\rho) = \left(1 - \frac{4p}{3}\right)\rho + \frac{4p}{3} \frac{I}{2} \quad (5.11)$$

Il canale interpola tra lo stato ρ e lo stato massimamente misto $I/2$: sulla sfera di Bloch il vettore viene contratto *isotropicamente* di un fattore $(1 - 4p/3)$ verso il centro. Per $p = 3/4$ lo stato diventa completamente misto, indipendentemente dallo stato iniziale.

5.5.4 Canale di Amplitude Damping

Il canale di **amplitude damping** modella il rilassamento energetico T_1 : il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ con probabilità $\gamma = 1 - e^{-t/T_1}$. Gli operatori di Kraus sono:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \quad (5.12)$$

Il canale agisce sulla matrice densità come:

$$\mathcal{E}_{AD}(\rho) = \begin{pmatrix} \rho_{00} + \gamma \rho_{11} & \sqrt{1-\gamma} \rho_{01} \\ \sqrt{1-\gamma} \rho_{10} & (1-\gamma) \rho_{11} \end{pmatrix} \quad (5.13)$$

La popolazione ρ_{11} decade verso ρ_{00} con tasso γ , mentre le coerenze vengono sopprese di $\sqrt{1-\gamma}$. A differenza dei canali precedenti, questo canale è **non simmetrico**:

sulla sfera di Bloch contrae la sfera verso il polo nord $|0\rangle$, non verso il centro:

$$r_x \rightarrow \sqrt{1-\gamma} r_x, \quad r_y \rightarrow \sqrt{1-\gamma} r_y, \quad r_z \rightarrow (1-\gamma) r_z + \gamma \quad (5.14)$$

Per $t \rightarrow \infty$ ($\gamma \rightarrow 1$), qualsiasi stato converge a $|0\rangle \langle 0|$, indipendentemente dallo stato iniziale.

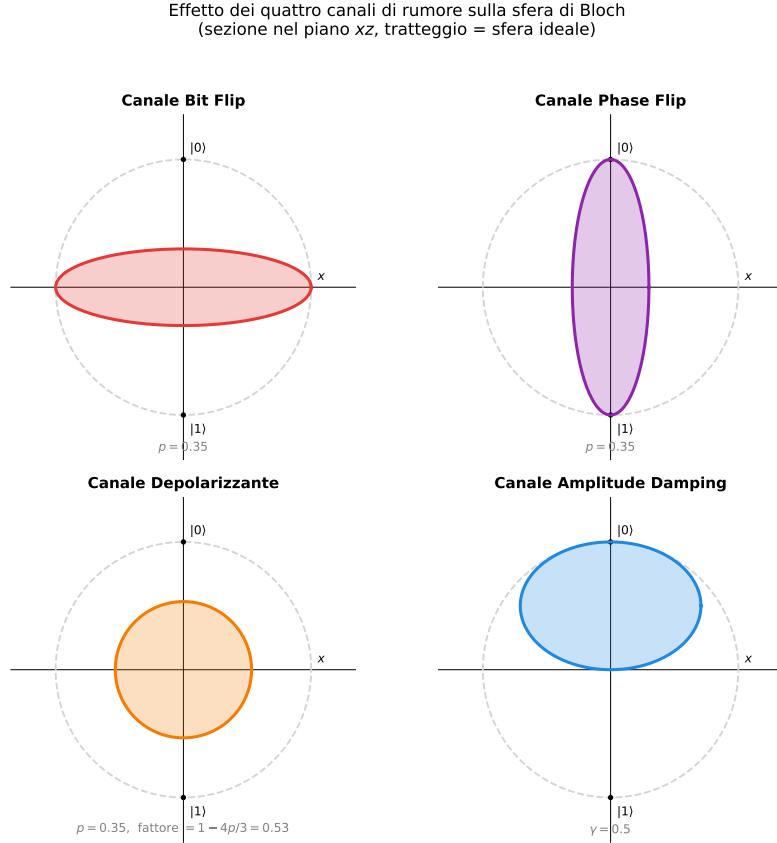


Figura 5.5: Effetto dei quattro canali di rumore sulla sfera di Bloch (sezione nel piano xz). La curva tratteggiata grigia rappresenta la sfera ideale; la superficie colorata è la sfera dopo l'applicazione del canale con il parametro indicato. Il Bit Flip contrae la sfera lungo z ; il Phase Flip lungo x (distrugge le coerenze equatoriali); il canale Depolarizzante contrae isotropicamente verso il centro; l'Amplitude Damping sposta la sfera verso il polo nord $|0\rangle$.

Canale	Parametro	Effetto coerenze	Effetto popolazioni	Modella
Bit Flip	p	Contrae r_y, r_z	Invariate	Errori X / leakage
Phase Flip	p	Sopprime ρ_{01}	Invariate	Dephasing (T_2)
Depolarizzante	p	Contrae isotropicamente	Verso $I/2$	Errori di gate generici
Amplitude Damping	γ	Sopprime ρ_{01}	Verso $ 0\rangle$	Decadimento T_1

Tabella 5.4: I quattro canali matematici fondamentali del rumore quantistico [26].

5.6 Impatto del Rumore sull'Algoritmo di Shor

L'algoritmo di Shor è particolarmente vulnerabile al rumore a causa della sua elevata profondità circuitale e della dipendenza critica dalla precisione delle fasi. La Figura 5.6 localizza le quattro fonti fisiche sulle fasi dell'algoritmo: ciascuna colpisce prevalentemente uno stadio diverso della pipeline.

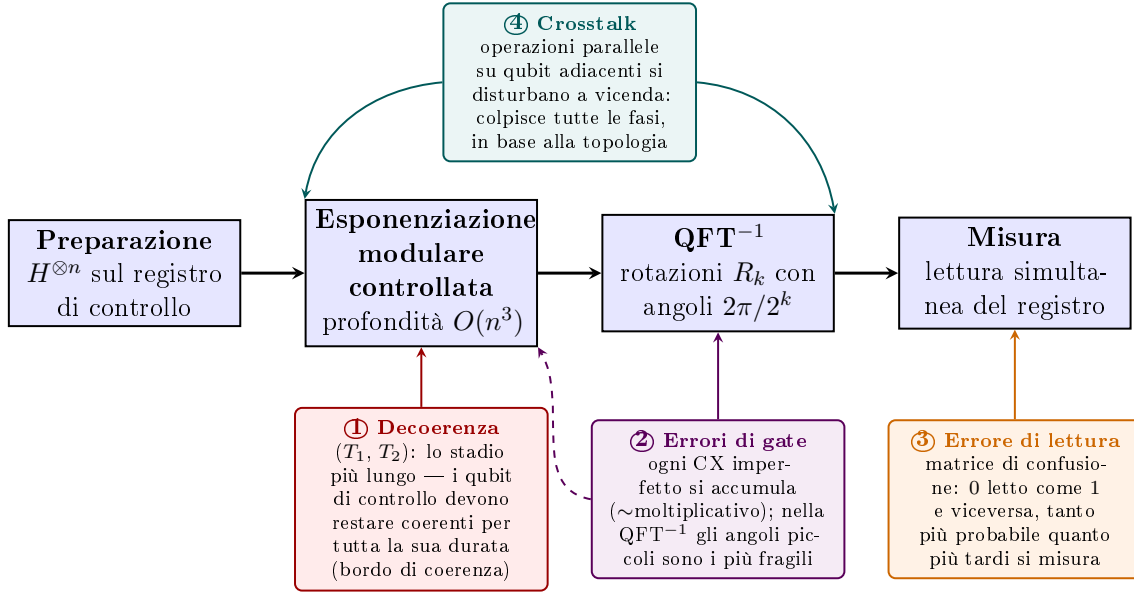


Figura 5.6: Localizzazione delle quattro fonti fisiche di rumore sulla pipeline dell'algoritmo di Shor. La decoerenza ① domina l'esponenziazione modulare (lo stadio più profondo, $O(n^3)$ gate); gli errori di gate ② si accumulano su tutti i CX e sono particolarmente critici sulle rotazioni ad angolo piccolo della QFT⁻¹; l'errore di lettura ③ colpisce la misura finale, tanto più quanto più il circuito arriva "lungo" al bordo di coerenza; il crosstalk ④ agisce trasversalmente ovunque vi siano operazioni parallele su qubit adiacenti, secondo la topologia del dispositivo (Sezione 5.2).

Le tre componenti più sensibili sono:

5.6.1 Errori nella Quantum Fourier Transform

La QFT richiede porte di rotazione di fase $R_k = \text{diag}(1, e^{2\pi i/2^k})$ con angoli esponenzialmente piccoli all'aumentare di k . Per $k = 10$, l'angolo è $2\pi/1024 \approx 6 \times 10^{-3}$ rad. Un canale depolarizzante di intensità ϵ su ciascuno degli n gate di rotazione accumula un errore di fase $O(n\epsilon)$, degradando la struttura di interferenza da cui dipende la corretta estrazione del periodo.

5.6.2 Errori nella Phase Estimation

La phase estimation richiede che i qubit del registro di controllo mantengano la coerenza per tutta la durata del circuito di modular exponentiation, che ha profondità $O(n^3)$. La condizione necessaria è:

$$T_2 \gg \tau_{\text{gate}} \cdot O(n^3) \quad (5.15)$$

Sui processori NISQ attuali, questa condizione è violata già per $n \gtrsim 10$ –15 bit.

5.6.3 Errori nella Modular Exponentiation

La modular exponentiation è la componente con la maggiore profondità. Modellando ogni gate con un canale depolarizzante di intensità ϵ , la probabilità di successo è:

$$P_{\text{success}} \approx P_{\text{ideal}} \cdot (1 - \epsilon)^L, \quad L \sim O(n^3) \quad (5.16)$$

Per $n = 15$ bit e $\epsilon = 10^{-3}$: $(1 - 10^{-3})^{3375} \approx 0.034$ — la probabilità di risultato corretto crolla al 3.4% già per istanze molto piccole. La Figura 5.7 mostra come questo crollo si accentui drammaticamente all'aumentare di n e di ϵ .

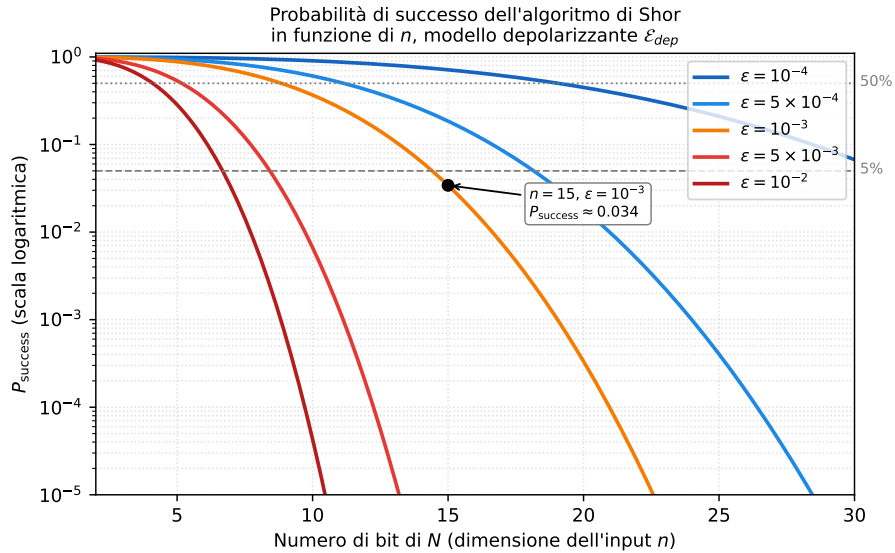


Figura 5.7: Probabilità di successo dell'algoritmo di Shor $P_{\text{success}} \approx (1 - \epsilon)^{n^3}$ in funzione della dimensione dell'input n (in bit), per cinque valori di intensità di rumore depolarizzante ϵ . Anche con $\epsilon = 10^{-3}$ (fedeltà gate 99.9%), già per $n = 15$ la probabilità di ottenere un risultato corretto scende al 3.4%. Le linee tratteggiate orizzontali evidenziano le soglie del 50% e del 5%.

5.7 Sfide Pratiche su Hardware NISQ

- **Requisiti di qubit:** fattorizzare un intero a n bit richiede almeno $2n$ qubit logici per la phase estimation, più ausiliari per la modular exponentiation.
- **Profondità:** anche per $N = 15$, il circuito compilato su `ibm_marrakesh` supera i 187 gate a due qubit [23].
- **Soluzioni parziali:** gli esperimenti su IBM hanno dimostrato la fattorizzazione di $N = 15$ e $N = 21$ solo con circuiti appositamente semplificati, non generalizzabili a N arbitrario [25].

Il problema non è di natura algoritmica ma fisica: l'algoritmo di Shor è teoricamente corretto, ma il substrato hardware introduce errori che ne compromettono l'output. Affrontare questo problema richiede strategie che operano a diversi livelli, dall'hardware al post-processing classico. Il capitolo seguente esamina queste strategie con particolare attenzione alla QEC e al suo inquadramento come sottocategoria del *Quantum Machine Learning* (QML), che costituisce l'approccio adottato in questa tesi.

Capitolo 6

Strategie per la Riduzione del Rumore Quantistico

Il capitolo precedente ha mostrato che il rumore quantistico non è un fastidio marginale: per l'algoritmo di Shor, già per istanze piccole come $N = 15$, la probabilità di ottenere un risultato corretto crolla al di sotto del 4% su hardware reale. Il problema è reale e quantificato, e richiede strategie concrete di mitigazione.

In letteratura si distinguono quattro macro-strategie. Questo capitolo le presenta in ordine crescente di complessità, spiega perché le prime tre non sono adatte al contesto di questa tesi, e approfondisce la quarta — il **Quantum Machine Learning** (QML) — che costituisce l'approccio adottato. Al suo interno, la **Correzione degli Errori Quantistici** (QEC) rappresenta la componente strutturale centrale, e il suo funzionamento viene spiegato nel dettaglio.

6.1 Le Quattro Strategie: Panoramica e Scelta

Le quattro macro-strategie per fronteggiare il rumore nei sistemi quantistici sono:

1. **Soppressione del rumore** (*Noise Suppression*): agisce *prima* che gli errori si manifestino, cercando di ridurre fisicamente l'accoppiamento tra il qubit e l'ambiente. Le tecniche principali sono il *Dynamical Decoupling* — sequenze di pulse che mediano a zero l'interazione con l'ambiente — l'ottimizzazione della durata dei gate, e l'isolamento fisico criogenico ed elettromagnetico dell'hardware. Riduce il rumore ma non lo elimina, ed è vincolata alle possibilità dell'hardware disponibile.

2. **Mitigazione degli errori** (*Error Mitigation*): opera *a posteriori*, senza correggere i qubit durante il calcolo. Si eseguono molte istanze del circuito con diversi livelli di rumore o con randomizzazioni, e si combinano i risultati classicamente per stimare l'output ideale. Le tecniche più diffuse sono la *Zero-Noise Extrapolation* (ZNE), il *Probabilistic Error Cancellation* (PEC) e il *Gate Twirling*. Il limite fondamentale è l'overhead esponenziale nel numero di campionamenti richiesti.
3. **Tolleranza ai guasti** (*Fault-Tolerant Quantum Computing*): è il paradigma architetturale a lungo termine. Codifica ogni qubit *logico* in molti qubit *fisici* e garantisce che, fintanto che il tasso di errore per gate rimane sotto una soglia critica (*threshold theorem* [33]), il tasso di errore logico può essere abbassato a valori arbitrariamente piccoli. Richiede però migliaia di qubit fisici per ogni qubit logico protetto — risorse ancora fuori dalla portata dell'hardware attuale.
4. **Quantum Machine Learning** (QML): sfrutta modelli di apprendimento automatico per apprendere il profilo di errore specifico dell'hardware e compensarlo senza richiedere un modello analitico del canale di rumore. È l'approccio più adattivo e flessibile, applicabile sia su hardware NISQ che su architetture future. Al suo interno, la QEC è la sottocategoria che gestisce la protezione strutturale dell'informazione quantistica.

Strategia	Quando agisce	Overhead	Era	Adottata
Noise Suppression	Prima del calcolo	Basso (hardware)	NISQ / FT	No
Error Mitigation	Dopo il calcolo	Esponenziale	NISQ	No*
Fault-Tolerant QC	Architetturale	Molto alto (qubit)	Futuro	No
QML (incl. QEC)	Ibrido (online)	Medio (classico)	NISQ / FT	Sì

Tabella 6.1: Confronto delle quattro strategie anti-rumore per il contesto di questa tesi.

*La ZNE non è adottata come strategia principale, ma viene confrontata *a posteriori* con M_{TOP4} nel Capitolo 10: ZNE-2 e ZNE-3 abbassano il tasso di successo dal 76.7% al 60% con un costo di 11 776 shot contro 1 024 per M_{TOP4} (p-value non significativo vs M1).

Motivazione per l'approccio QML. Le prime tre strategie presentano limitazioni che le rendono inadatte al contesto di questa tesi. La soppressione del rumore è vincolata all'hardware disponibile e non è controllabile a livello software. La mitigazione degli errori richiede un numero esponenziale di esecuzioni, incompatibile con

l'obiettivo di ridurre il numero di iterazioni necessarie. La tolleranza ai guasti richiede risorse hardware ancora inesistenti. Il QML, al contrario, può essere applicato su qualunque simulatore o processore reale, si adatta al profilo di errore specifico del dispositivo senza un modello analitico, e — come si vedrà nella sezione 6.4 — riduce drasticamente il numero di esecuzioni necessarie per ottenere un risultato affidabile.

6.2 Quantum Machine Learning per la Robustezza al Rumore

Il QML, nel contesto della robustezza al rumore, non si riferisce all'uso di algoritmi quantistici per fare machine learning, bensì all'uso di modelli di *machine learning classici o ibridi* per compensare gli effetti del rumore su circuiti quantistici. L'idea fondamentale è semplice: invece di costruire un modello analitico del canale di rumore — operazione complessa e spesso impossibile per hardware reale con errori correlati e non stazionari — si lascia che un modello apprenditivo impari direttamente dalla distribuzione degli output rumorosi.

Le tre declinazioni principali del QML nel contesto di questa tesi sono:

- **Caratterizzazione del rumore:** un modello di regressione (Random Forest, rete neurale) apprende la relazione tra i parametri hardware (T_1 , T_2 , profondità del circuito) e la fedeltà dell'output, permettendo di predire la qualità del risultato prima di eseguire il circuito.
- **Ottimizzazione variazionale:** i parametri del circuito (decomposizione della modular exponentiation, routing dei qubit) vengono ottimizzati tramite agenti di Reinforcement Learning che massimizzano la probabilità di successo sul simulatore rumoroso, adattando il circuito al profilo di errore specifico del dispositivo.
- **Correzione degli errori (QEC):** una rete neurale o un classificatore impara a decodificare le sindromi di errore e a determinare la correzione ottimale. Questa è la sottocategoria strutturalmente più profonda del QML e viene approfondita nella sezione seguente.

6.3 La Correzione degli Errori Quantistici come Sottocategoria del QML

6.3.1 Il Problema Fondamentale

Come si visto nel capitolo precedente, il teorema del no-cloning impedisce di proteggere un qubit semplicemente duplicandolo, come si farebbe con un bit classico. Eppure la correzione degli errori quantistici è possibile — e questa fu una delle scoperte più sorprendenti degli anni Novanta. Il principio è che si può *distribuire* l'informazione di un qubit logico su più qubit fisici in modo tale che, misurando opportune quantità collettive (le **sindromi**), sia possibile rilevare e correggere gli errori senza mai misurare — e quindi distruggere — lo stato logico.

6.3.2 Il Paradigma in Tre Fasi

Qualunque codice di correzione degli errori quantistici funziona secondo lo stesso schema:

1. **Codifica:** lo stato logico $|\psi_L\rangle$ viene distribuito su n qubit fisici tramite un circuito di codifica. L'informazione è ridondante: nessun singolo qubit fisico contiene da solo l'informazione logica.
2. **Misura di sindrome:** si misurano operatori stabilizzatori — operatori che commutano con tutti i codeword ma anticommutano con gli operatori di errore. Questi operatori forniscono informazione sull'errore avvenuto senza rivelare il valore del qubit logico. Il risultato della misura è la **sindrome**, un vettore binario che identifica quale errore (o classe di errori) si è verificato.
3. **Decodifica e correzione:** dato il vettore di sindrome, un algoritmo di decodifica determina l'operazione di recupero da applicare per ripristinare lo stato logico corretto. Questa è la fase in cui entra il QML: i decoder tradizionali (minimum-weight perfect matching, look-up table) vengono sostituiti da reti neurali addestrate a inferire la correzione più probabile, con prestazioni superiori in regimi di rumore realistico e non-omogeneo.

6.3.3 I Principali Codici Quantistici

Nel corso degli anni sono stati sviluppati numerosi codici di correzione. I tre più rilevanti per questa tesi sono:

Codice di Shor (1995). Il primo codice di correzione degli errori quantistici [34], proposto dallo stesso Peter Shor dell'algoritmo di fattorizzazione, codifica 1 qubit logico in 9 qubit fisici ed è in grado di correggere qualsiasi errore singolo (bit flip, phase flip o entrambi). Lo stato logico è:

$$|0_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)^{\otimes 3}, \quad |1_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)^{\otimes 3} \quad (6.1)$$

La struttura è a due livelli: tre copie del qubit (ciascuna nella forma $|000\rangle \pm |111\rangle$) proteggono da phase flip tramite interferenza; la ripetizione delle tre copie protegge da bit flip tramite ridondanza. Questo risultato dimostrò per la prima volta che il teorema del no-cloning non impedisce la correzione degli errori quantistici.

Codice di Steane (1996). Il codice di Steane [35] riduce l'overhead a 7 qubit fisici per 1 qubit logico, sfruttando la struttura del codice classico di Hamming [7, 4, 3]. Corregge qualsiasi errore singolo ed è il primo esempio di codice CSS (Calderbank–Shor–Steane), una famiglia sistematica di costruzioni che collegano codici classici lineari a codici quantistici.

Surface Code. Il **surface code** [36] è oggi il codice più promettente per implementazioni su hardware reale, ed è quello su cui IBM e Google stanno costruendo le loro architetture fault-tolerant. Organizza i qubit fisici in una griglia 2D con interazioni solo tra qubit vicini — un requisito compatibile con le architetture superconduttrici. La sua proprietà chiave è il **threshold**: fintanto che il tasso di errore per gate rimane sotto circa 1%, aumentare la distanza d del codice abbassa il tasso di errore logico in modo esponenziale.¹ Per distanza d , il codice usa d^2 qubit fisici per 1 qubit logico e corregge fino a $\lfloor (d-1)/2 \rfloor$ errori.

6.3.4 Perché la QEC è una Sottocategoria del QML

Il legame tra QEC e QML emerge nella fase di decodifica. I decoder tradizionali assumono un modello di rumore semplice e stazionario, ma sull'hardware reale il ru-

¹Il threshold preciso dipende dal modello di rumore: $\approx 1.1\%$ per rumore depolarizzante indipendente, $\approx 0.5\text{--}0.7\%$ per modelli più realistici con correlazioni spaziali. I migliori processori IBM e Google si avvicinano oggi a queste soglie.

more è correlato, non uniforme e varia nel tempo. Le reti neurali non hanno bisogno di questo modello: apprendono direttamente dalla storia delle sindromi la correzione più probabile, adattandosi al profilo di errore specifico del dispositivo. In questo senso la QEC non è una strategia separata dal QML, ma la sua specializzazione più strutturata: il modello apprenditivo sostituisce il decoder analitico e ne migliora le prestazioni in condizioni reali [36].

6.4 Motivazione Pratica: il Contributo di questa Tesi

Chiarito il framework teorico, è importante spiegare concretamente cosa porta il QML rispetto all'alternativa più semplice.

Senza ML — approccio statistico classico. In assenza di un modello appreso, per determinare il risultato corretto dell'algoritmo di Shor in presenza di rumore è necessario eseguire il circuito molte volte, raccogliere la distribuzione degli output e analizzarne la forma. In condizioni ideali la distribuzione mostra picchi netti ai valori corretti (si veda il Capitolo 10); in presenza di rumore i picchi si allargano e si sovrappongono al fondo, rendendo necessaria un'analisi statistica per identificare il valore più probabile. Questo approccio richiede decine o centinaia di esecuzioni prima di raggiungere una confidenza sufficiente.

Con ML — ipotesi verificata in questa tesi. L'ipotesi di partenza è addestrare un classificatore che, ricevuto in ingresso l'output di una singola esecuzione dell'algoritmo di Shor su simulatore rumoroso, determini se quel risultato è corretto o meno: se il modello impara a distinguere i pattern di output corretto da quelli distorti dal rumore, dovrebbero bastare poche iterazioni (l'ipotesi quantitativa è ≈ 3) per ottenere una risposta affidabile, senza costruire una distribuzione statistica completa.

Il contributo centrale della parte sperimentale di questa tesi è la verifica rigorosa di questa ipotesi — e il suo esito è più interessante della sua semplice conferma: la riduzione delle iterazioni si ottiene davvero (da $\bar{M}_1 = 6.43$ a $\bar{M} = 1.00$ per UC1), ma l'analisi di ablazione dimostra che il merito non è del classificatore, bensì della strategia di ricerca multi-candidato TOP-K che lo accompagna. Il risultato negativo sul filtro ML, documentato sperimentalmente, delimita il regime in cui il QML applicato all'output non apporta valore. Prima di descrivere l'architettura del sistema, il Capitolo 7 motiva le scelte tecnologiche adottate — Qiskit, Aer e scikit-learn —

in funzione dei requisiti specifici del Metodo 2: simulazione accurata del rumore, integrazione fluida con il machine learning classico e allineamento con l'hardware IBM. Il Capitolo 8 presenta quindi l'architettura del pipeline sperimentale, mentre il Capitolo 10 riporta i risultati concreti su simulatore Qiskit con modello di rumore realistico.

Capitolo 7

Strumenti e Ambiente di Simulazione

La scelta degli strumenti di simulazione non è neutrale. In informatica quantistica, il framework adottato determina quali modelli di rumore sono disponibili, verso quale hardware è possibile traslare i circuiti e con quali risultati della letteratura è possibile confrontarsi direttamente. Questo capitolo descrive gli strumenti utilizzati nella parte sperimentale di questa tesi, motivando ciascuna scelta e collocandola rispetto alle alternative disponibili.

7.1 Qiskit: il Framework Quantistico di Riferimento

Qiskit (*Quantum Information Science Kit*) è un *Software Development Kit* (SDK) open-source sviluppato da IBM Quantum per la programmazione, la simulazione e l'esecuzione di algoritmi su processori quantistici superconduttori [37]. Rilasciato pubblicamente nel 2017, è oggi il framework di riferimento per la ricerca e la sperimentazione su hardware IBM ed è il più adottato nella letteratura scientifica sugli algoritmi quantistici a corto termine [25, 22].

7.1.1 Architettura del Framework

Qiskit è organizzato in tre componenti principali, ciascuno con un ruolo distinto nel ciclo di vita di un esperimento quantistico:

- **Qiskit (core)** — il nucleo del framework, già noto come Qiskit Terra. Fornisce le primitive fondamentali per la costruzione di circuiti quantistici (`QuantumCircuit`), la compilazione e la trasposizione verso un backend target (`transpile`), e le interfacce di esecuzione (`Sampler`, `Estimator`). Gestisce l'intera pipeline dalla descrizione astratta del circuito fino all'invio al simulatore o all'hardware reale.

- **Qiskit Aer** — il modulo di simulazione classica ad alte prestazioni. Implementa diversi metodi di simulazione esatta e approssimata, con supporto nativo per modelli di rumore realistici. È il componente centrale di questa tesi e viene descritto in dettaglio nella Sezione 7.2.
- **Qiskit IBM Runtime** — l'interfaccia per l'esecuzione su processori IBM Quantum reali tramite cloud (`qiskit-ibm-runtime`). Fornisce le primitive `Sampler` e `Estimator` ottimizzate per l'hardware reale, con supporto per tecniche di soppressione del rumore come il *Dynamical Decoupling* e il *Gate Twirling* [23].

7.1.2 Perché Qiskit e non un'alternativa

La scelta di Qiskit è stata formalizzata nell'incontro con il relatore del 18 marzo 2026, in seguito all'analisi del tutorial ufficiale IBM per l'algoritmo di Shor [23]. La decisione è motivata da quattro ragioni convergenti.

Prima ragione: coerenza con la piattaforma hardware. Il lavoro di tesi utilizza un modello di rumore calibrato su processori superconduttori IBM. Qiskit è sviluppato da IBM Quantum e i suoi modelli di rumore, le sue primitive di esecuzione e le sue tecniche di mitigazione sono progettati e validati su quella stessa architettura fisica. Questa coerenza garantisce che i parametri di simulazione siano direttamente confrontabili con dati sperimentali reali [21].

Seconda ragione: maturità del simulatore Aer. Qiskit Aer è il simulatore con il modello di rumore più maturo e documentato tra i framework open-source disponibili. I principali competitor — Cirq (Google) [38], PennyLane (Xanadu) [39] — offrono simulazione con rumore, ma con un supporto meno esteso per i modelli fisici dei processori superconduttori (errori di depolarizzazione, rilassamento termico T_1/T_2 , errori di misura).

Terza ragione: disponibilità dell'implementazione di riferimento. Il circuito dell'algoritmo di Shor utilizzato in questa tesi è tratto da un'implementazione open-source in Qiskit [20]. Questo punto di partenza, validato dalla letteratura [25], ha reso naturale mantenere Qiskit come framework unico lungo l'intero pipeline sperimentale.

Quarta ragione: allineamento con la letteratura. La grande maggioranza dei lavori recenti sull'implementazione dell'algoritmo di Shor e sulla mitigazione del rumore su sistemi NISQ utilizza Qiskit [25, 22, 40]. L'adozione dello stesso fra-

network garantisce la comparabilità diretta dei risultati senza necessità di ricalibrare modelli o convertire rappresentazioni.

Tabella 7.1: Confronto tra i principali framework per la simulazione quantistica con rumore.

Framework	Sviluppatore	Simulatore con rumore	Target hardware primario
Qiskit + Aer	IBM Quantum	Completo (T1, T2, depolarz., readout)	IBM superconduttori
Cirq	Google Quantum AI	Parziale	Google superconduttori
PennyLane	Xanadu	Limitato	Vari (plug-in)
Forest / PyQuil	Rigetti	Parziale	Rigetti superconduttori

7.2 Qiskit Aer: il Simulatore

Qiskit Aer è il modulo di simulazione classica di Qiskit. Il suo componente principale è `AerSimulator`, un simulatore ad alte prestazioni che supporta più metodi di simulazione selezionabili a seconda della dimensione del circuito e del tipo di analisi richiesta.

7.2.1 Metodi di Simulazione

`AerSimulator` espone i seguenti metodi principali tramite il parametro `method`:

- **statevector** — simulazione esatta del vettore di stato completo. Richiede 2^n ampiezze complesse in memoria, il che limita la scalabilità a circa 30 qubit su hardware consumer con Unità di Elaborazione Grafica (*Graphics Processing Unit*) (GPU). È il metodo adottato in questa tesi per la precisione numerica che garantisce.
- **density_matrix** — evolve la matrice densità ρ anziché il vettore di stato, permettendo di modellare canali di rumore non unitari in modo esatto. Richiede 4^n elementi e riduce il limite pratico a circa 25 qubit.
- **stabilizer** — simulazione efficiente di circuiti Clifford puri (senza porte non-Clifford come T o R_z arbitrarie). Scala a migliaia di qubit ma non è applicabile all'algoritmo di Shor, che utilizza rotazioni di fase arbitrarie.

- `matrix_product_state` — approssima lo stato come prodotto di tensori, adatto a circuiti con bassa entanglement. Non adeguato per le istanze di Shor usate in questa tesi, dove l'entanglement è significativo.

La scelta del metodo `statevector` è motivata dal numero di qubit delle istanze trattate (da 11 a 18 qubit nei quattro use case) e dalla necessità di un risultato esatto, non approssimato.

7.2.2 Modelli di Rumore

Il modulo `qiskit_aer.noise` fornisce le primitive per costruire modelli di rumore compositi. Ogni errore è rappresentato come un canale quantistico che viene applicato ai gate specificati, con la possibilità di assegnare errori diversi a qubit diversi (rumore eterogeneo) o un errore uniforme a tutti i qubit (*all-qubit error*). I tipi di errore disponibili includono:

- `depolarizing_error` — errore depolarizzante su gate a 1 o 2 qubit, parametrizzato dal tasso ϵ .
- `thermal_relaxation_error` — modella il rilassamento verso lo stato fondamentale (T_1) e la perdita di coerenza di fase (T_2), con un tempo di gate configurabile.
- `ReadoutError` — matrice di confusione che descrive la probabilità di misurare $|0\rangle$ come $|1\rangle$ e viceversa.
- Errori coerenti — applicazione di una rotazione indesiderata su ogni gate, modellabile tramite operatori unitari arbitrari.

Il modello di rumore usato in questa tesi combina errori depolarizzanti, rilassamento termico e readout error, con parametri calibrati sull'hardware `ibm_marrakesh` secondo la metodologia descritta in [21]. La configurazione dettagliata è riportata nel Capitolo 2.

7.2.3 Transpilazione

Prima dell'esecuzione, il circuito deve essere *transpilato*: la funzione `transpile` riscrive il circuito nel gate set nativo del backend target (per `AerSimulator`: `cx`, `h`, `rz`, `x`, `swap`) e applica ottimizzazioni strutturali selezionabili tramite il parametro

`optimization_level` (da 0 a 3). Un livello di ottimizzazione elevato riduce il numero di gate e, di conseguenza, l'accumulo di errori, a scapito di un maggior tempo di compilazione. In questa tesi si usa `optimization_level=3` per minimizzare la profondità del circuito eseguito.

7.3 Accelerazione GPU con Qiskit Aer

La libreria `qiskit-aer-gpu` estende `AerSimulator` con il supporto per l'accelerazione tramite GPU NVIDIA attraverso CUDA. Il backend `statevector` è interamente eseguito sulla GPU: il vettore di stato viene allocato nella memoria VRAM e le operazioni di gate sono eseguite come moltiplicazioni matrice-vettore parallelizzate su migliaia di core CUDA.

Motivazione pratica. La parte sperimentale di questa tesi richiede la generazione di un dataset di addestramento per il classificatore del Metodo 2: migliaia di esecuzioni del circuito di Shor con parametri di rumore variabili. Su Unità di Elaborazione Centrale (*Central Processing Unit*) (CPU), ogni esecuzione richiede diversi secondi per circuiti a 15–18 qubit con 10^3 shot; su GPU, il tempo si riduce di un fattore tra 10 e 20 per circuiti ad alto numero di shot, rendendo praticabile la generazione dell'intero dataset in una singola sessione di lavoro.

Specifiche hardware. La GPU utilizzata è una NVIDIA GeForce RTX 4070 con 12 GB di memoria VRAM e 5888 core CUDA (architettura Ada Lovelace). La capacità di memoria consente di simulare circuiti fino a circa 30 qubit in modalità `statevector`, sufficiente per tutti gli use case di questa tesi.¹

7.4 scikit-learn: la Libreria per il Classificatore

Il Metodo 2 richiede l'addestramento di un classificatore binario. La libreria scelta è `scikit-learn` [41], una delle librerie di machine learning più consolidate nell'ecosistema Python scientifico.

La motivazione principale è pragmatica: `scikit-learn` espone un'interfaccia unificata (`fit` / `predict`) per un ampio catalogo di classificatori — Random Forest, *Support Vector Machine* (SVM), Percettrone Multistrato (*Multi-Layer Perceptron*)

¹Il limite teorico di VRAM per la simulazione `statevector` è $2^n \times 16$ byte per n qubit (ampiezza `complex128`). Con 12 GB: $\lfloor \log_2(12 \times 10^9 / 16) \rfloor = 29.8$ qubit.

(MLP), e altri — permettendo di confrontarli sullo stesso dataset con minime modifiche al codice. Questo facilita la selezione sperimentale del modello migliore senza dover introdurre dipendenze da framework più pesanti (TensorFlow, PyTorch) per un classificatore che, per il problema in esame, si prevede non richieda architetture profonde.

7.5 Ambiente di Esecuzione: WSL con Ubuntu

Le simulazioni vengono eseguite in ambiente Linux tramite *Windows Subsystem for Linux* (WSL) (versione 2) su Windows 11. La scelta è motivata dalla necessità di mantenere la compatibilità con l'ecosistema Linux di Qiskit e delle librerie scientifiche Python, garantendo al contempo l'integrazione con i driver CUDA per WSL forniti da NVIDIA.

WSL 2 esegue un kernel Linux completo in una macchina virtuale leggera, con accesso diretto alle GPU NVIDIA tramite i driver `cuda-wsl-ubuntu`. L'ambiente Python è isolato in un virtual environment dedicato (Ubuntu 22.04 LTS, Python 3.11), garantendo la riproducibilità delle dipendenze e la compatibilità delle versioni tra le librerie usate.

Con l'ambiente così definito, il Capitolo 8 descrive come Qiskit, Aer e scikit-learn si integrano nell'architettura del pipeline sperimentale: dalla costruzione del circuito di Shor alla generazione del dataset di addestramento, fino all'inferenza del classificatore sui quattro use case.

Capitolo 8

Metodologia e Architettura

Il capitolo precedente ha concluso la parte teorica della tesi con una scelta precisa: il QML, nella sua declinazione di classificatore addestrato per la rilevazione degli errori, è l'approccio adottato per mitigare il rumore nell'algoritmo di Shor. L'ipotesi di lavoro, motivata dalla letteratura, è che questo approccio riduca il numero medio di esecuzioni necessarie da $\bar{M}_1 \approx 6-7$ a circa tre. Come si vedrà nei capitoli dei risultati, il guadagno effettivo supera questa aspettativa. Ciò che resta da definire è come questa scelta si traduce concretamente in un sistema sperimentale funzionante.

Questo capitolo ha questo scopo. Vengono descritte l'architettura del pipeline, le tecnologie adottate e la struttura dei due metodi che verranno confrontati nella parte sperimentale. Non si entra ancora nella descrizione del codice né dei risultati: quelli sono oggetto dei capitoli successivi. Qui si stabilisce il *perché* di ciascuna scelta progettuale e il *come* i componenti si raccordano tra loro.

8.1 Obiettivi della Parte Sperimentale

La parte sperimentale di questa tesi ha due obiettivi distinti ma complementari, che corrispondono ai due metodi sviluppati.

Il primo obiettivo è costruire una **baseline affidabile**: una misura quantitativa del numero di esecuzioni che l'approccio classico, privo di qualunque modello appreso, richiede per ricavare il risultato corretto dell'algoritmo di Shor in presenza di rumore. Questo primo metodo — chiamato nel seguito **Metodo 1** — opera raccogliendo la distribuzione statistica degli output su molte esecuzioni, identificando i picchi della distribuzione e stimando il periodo r dal valore più probabile. Il Metodo 1 non introduce ipotesi sul profilo di errore del sistema: funziona su qualunque noise model

e non richiede dati di addestramento. È il punto di partenza naturale e il termine di paragone contro cui misurare il miglioramento.

Il secondo obiettivo è dimostrare che un **classificatore addestrato** può sostituire l'accumulazione statistica con una decisione informata basata su una singola esecuzione. Questo secondo metodo — chiamato **Metodo 2** — apprende la firma degli output corretti e di quelli distorti dal rumore, e decide dopo ogni esecuzione se il risultato è affidabile o meno. L'ipotesi, motivata teoricamente nel Capitolo 6 e corroborata dalla letteratura recente sull'uso del ML per la mitigazione degli errori quantistici [40], è che bastino poche iterazioni — tipicamente dell'ordine di tre — per convergere.

Entrambi i metodi vengono valutati su **quattro use case** indipendenti, scelti per coprire diverse configurazioni di istanza e livello di rumore. La scelta di almeno quattro configurazioni risponde a un requisito metodologico consolidato per la validazione accademica dei risultati sperimentali in questo dominio [22].

8.2 Architettura del Pipeline Sperimentale

Il sistema è strutturato come un pipeline a tre stadi sequenziali, illustrato in Figura 8.1.

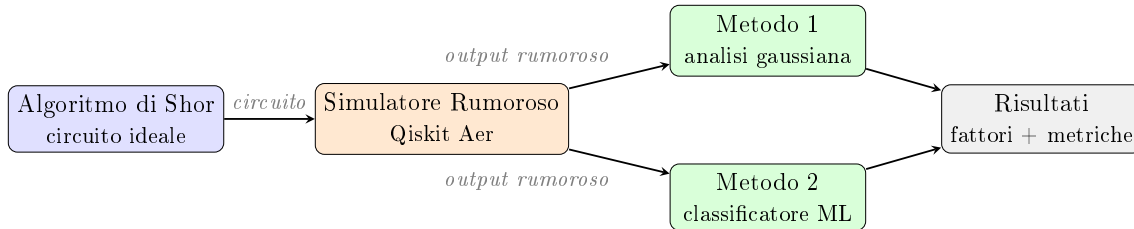


Figura 8.1: Pipeline sperimentale. Il circuito ideale dell'algoritmo di Shor viene eseguito sul simulatore rumoroso **AerSimulator** di Qiskit. L'output è elaborato in parallelo dal Metodo 1 (analisi della distribuzione statistica) e dal Metodo 2 (classificatore ML). Entrambi producono i fattori di N e le metriche di confronto.

Stadio 1 — Il circuito di Shor. Il circuito dell'algoritmo di Shor è tratto dall'implementazione open-source di riferimento disponibile su GitHub [20]. Si tratta di un'implementazione ideale, costruita per un simulatore privo di rumore: in assenza di errori produce sempre la fattorizzazione corretta. Questo punto di partenza è una scelta deliberata. L'obiettivo della tesi non è modificare l'algoritmo, bensì studiare il suo comportamento su un simulatore realistico e sviluppare stru-

menti per recuperare l'informazione corretta nonostante la degradazione introdotta dal rumore.

Stadio 2 — La simulazione rumorosa. Il circuito ideale viene eseguito tramite il simulatore `AerSimulator` di Qiskit con un modello di rumore che replica le caratteristiche fisiche di un processore superconduttore NISQ. La scelta di un simulatore — anziché di un processore reale — consente di controllare con precisione i parametri di rumore, garantisce la riproducibilità degli esperimenti e permette di generare grandi dataset di addestramento per il classificatore del Metodo 2 senza i vincoli di accesso e costo tipici dell'hardware IBM Quantum. Il modello di rumore adottato è descritto in dettaglio nel Capitolo 2; la sua configurazione segue la metodologia proposta in [21] per la selezione e il calibrazione dei parametri di emulazione rispetto a backend reali.

Stadio 3 — L'analisi. L'output del simulatore viene elaborato indipendentemente dai due metodi. I risultati prodotti da entrambi vengono confrontati sulle stesse metriche e sugli stessi use case, garantendo la comparabilità del confronto.

8.3 Tecniche di Mitigazione del Rumore: Analisi e Scelte Implementative

Prima di descrivere i due metodi, è necessario motivare una scelta di fondo: *quali leve di mitigazione sono disponibili in questo contesto, e quali sono state adottate?* Il Capitolo 6 ha classificato le macro-strategie; qui si scende al livello operativo, analizzando le tecniche concrete applicabili sul simulatore Qiskit e spiegando perché alcune sono state adottate, altre scartate e altre ancora mantenute come termine di confronto.

Le tecniche disponibili si raggruppano in tre livelli di intervento: a livello di circuito (prima dell'esecuzione), a livello di misura (durante o subito dopo), e a livello di post-processing (sull'output classico).

8.3.1 Livello di Circuito: Transpilazione e Riduzione della Profondità

La sorgente di errore più diretta in un circuito quantistico rumoroso è la profondità del circuito stesso: ogni gate a due qubit (Controlled-X (porta CNOT) (CX)) introduce una probabilità di errore non trascurabile, e l'effetto si accumula in mo-

do approssimativamente moltiplicativo. L'intervento più efficace — e con il minore overhead computazionale — è quindi ridurre il numero di gate CX prima dell'esecuzione.

Nel contesto dell'algoritmo di Shor, il collo di bottiglia è la **modular exponentiation** $U_f : |x\rangle |0\rangle \mapsto |x\rangle |a^x \bmod N\rangle$, la parte del circuito che implementa la funzione periodica il cui periodo r viene estratto dalla QPE. Una decomposizione naïve di questo operatore richiede $O(n^3)$ gate CX su n qubit, un overhead inaccettabile anche per N piccoli.

La **decomposizione di Beauregard** [28] risolve questo problema riducendo il numero di qubit richiesti a $2n + 3$ e il numero di gate CX a $O(n^2)$, sfruttando l'aritmetica modulare nel dominio della QFT e sostituendo i registri ausiliari con adder di Draper in-place. Per $N = 15$ ($n = 4$ bit) la decomposizione porta il circuito QPE completo a circa 98 gate CX su 9 qubit; per $N = 21$ ($n = 5$ bit) a circa 2594 gate CX su 20 qubit. **Questa ottimizzazione è già incorporata nel circuito di partenza adottato in questa tesi:** non è una scelta da fare a runtime, ma una preconditione dell'implementazione.

Un ulteriore livello di riduzione è offerto dalla **transpilazione ottimizzata** di Qiskit (`optimization_level=3`): il transpiler decompone le porte nel gate set nativo del backend target, cancella sequenze identità, riordina i gate commutanti e seleziona i qubit fisici con i migliori parametri T_1 e T_2 . Sul simulatore rumoroso questa ottimizzazione è applicata sistematicamente in entrambi i metodi.

8.3.2 Livello di Misura: Mitigazione degli Errori di Readout

Gli errori di *readout* — la probabilità che un qubit nello stato $|0\rangle$ venga letto come 1 e viceversa — sono una fonte di errore sistematica e in larga parte correggibile. La tecnica standard è la **calibrazione della matrice di assegnazione**: si preparano gli stati base $|00\dots 0\rangle$, $|10\dots 0\rangle$, ... e si misurano le distribuzioni degli output; la matrice $A_{ij} = P(\text{output} = i \mid \text{preparato} = j)$ viene poi invertita e applicata al vettore delle frequenze osservate.

Nel contesto di questa tesi la readout mitigation non è stata adottata come tecnica principale per due ragioni. Prima, il modello di rumore del simulatore **AerSimulator** include già errori di readout calibrati su hardware IBM reale: la probabilità di errore è dell'ordine di 10^{-2} per qubit, un valore che il classificatore del Metodo 2 può compensare implicitamente senza richiedere la costruzione esplicita della matrice di calibrazione. Seconda, per i circuiti QPE con 9–20 qubit, la

matrice di calibrazione ha dimensione $2^n \times 2^n$: la sua costruzione richiederebbe 2^n preparazioni aggiuntive, un overhead incompatibile con l’obiettivo di minimizzare le esecuzioni. La readout mitigation è stata però inclusa nell’analisi a posteriori del Capitolo 10 come tecnica di confronto.

8.3.3 Livello di Post-processing: Zero-Noise Extrapolation

La **Zero-Noise Extrapolation** (ZNE) [42] è una tecnica di mitigazione che opera interamente in post-processing: si esegue il circuito a più livelli di rumore artificialmente amplificato (tipicamente $\lambda = 1, 2, 3$ volte il rumore nominale, ottenuto per gate folding o pulse stretching), si misura il valor medio dell’osservabile di interesse per ciascun livello, e si estrapolano i valori verso $\lambda = 0$ tramite una regressione lineare o polinomiale.

Il pregio della ZNE è che non richiede conoscenza del modello di rumore e produce una stima dell’output ideale direttamente dall’output rumoroso. Il limite, nel contesto di questa tesi, è duplice. Da un lato, richiede almeno $k \geq 2$ esecuzioni per ogni singola stima — il che contrasta direttamente con l’obiettivo del Metodo 2 di convergere in circa tre iterazioni totali. Dall’altro, l’amplificazione del rumore degrada ulteriormente la fedeltà dei gate già ai livelli $\lambda = 2$ e $\lambda = 3$: per circuiti profondi come quello di Shor, il gate folding introduce errori aggiuntivi che non si distribuiscono linearmente e rendono l’extrapolazione inaffidabile.

Per questi motivi la ZNE non è stata adottata come tecnica principale. Essa compare però nel Capitolo 10 come termine di confronto sperimentale: i risultati mostrano che ZNE-2 e ZNE-3 abbassano il tasso di successo dal 76.7% (baseline M1) al 60%, con un costo di 11 776 shot contro 1024 per M_{TOP4} .

8.3.4 Analisi di Sensibilità ai Parametri di Rumore

Prima di fissare l’approccio definitivo, è stato condotto uno studio empirico sistematico per determinare quali parametri del modello di rumore influenzano effettivamente le prestazioni dell’algoritmo di Shor su $N=15$. Lo sweep è stato eseguito variando un parametro per volta mantenendo fissi tutti gli altri ai valori del profilo NISQ-realistico (Tabella 2.2), con $K=30$ ripetizioni per punto e $\text{SHOTS}=1024$.

Parametro	Valore base	Range testato	Effetto su $\bar{M}_1$
ε_{1q} (gate 1-qubit)	10^{-3}	$[10^{-4}, 2 \times 10^{-2}]$	Nessuno
T_1/T_2 (decoerenza)	100/80 μs	$[20, 500]$ μs	Nessuno
p_{ro} (readout error)	0.02	$[0, 0.20]$	Nessuno
ε_{2q} (gate CX)	10^{-2}	$[10^{-3}, 10^{-1}]$	Dominante

Tabella 8.1: Risultati dell’analisi di sensibilità ai parametri di rumore per $N=15$, $a=7$, $n_{\text{count}}=8$. Solo ε_{2q} influenza le prestazioni di M_1 .

I risultati mostrano con nettezza che **il solo parametro che influenza $\bar{M}_1$ è il tasso di errore dei gate a due qubit ε_{2q}** . Tutti gli altri parametri — errori sui gate a singolo qubit, tempi di decoerenza T_1/T_2 e readout error — producono $\bar{M}_1 = 1.0$ con success rate = 100% sull’intero range testato, indistinguibili dal caso ideale.

ε_{2q}	$\bar{M}_1$	σ_{M_1}	Success rate	TOP-4 $\bar{M}$
10^{-3}	2.27	5.22	73.3%	1.0
5×10^{-3}	3.18	6.11	73.3%	1.0
10^{-2}	6.43	9.80	76.7%	1.0
2×10^{-2}	4.77	8.14	86.7%	1.0
5×10^{-2}	5.92	8.96	83.3%	1.0
10^{-1}	6.13	9.41	76.7%	1.0

Tabella 8.2: Sweep di ε_{2q} su UC1 ($N=15$, $a=7$, $K=30$). $\bar{M}_1$ cresce con il tasso di errore CX; la strategia TOP-4 mantiene success rate = 100% su tutto il range.

Questo risultato ha un’interpretazione fisica precisa. Il circuito di Shor per $N=15$ con la decomposizione a permutazione (`c_amod15`) ha profondità ridotta: in queste condizioni il tempo totale di esecuzione è sufficientemente breve da rendere la decoerenza (T_1, T_2) trascurabile rispetto al contributo dei gate a due qubit. Analogamente, gli errori sui gate a singolo qubit e il readout error si trovano in un regime in cui il loro effetto è oscurato dall’errore CX. La fonte di degradazione dominante è quindi il numero e la qualità dei gate CX.

Implicazioni per le scelte implementative. Questa analisi ha tre conseguenze dirette:

1. **La decomposizione Beauregard è la leva più efficace.** Ridurre ε_{2q} non è controllabile (è una proprietà del backend), ma ridurre il *numero* di gate CX è possibile a livello di implementazione. La decomposizione Beauregard

(Sezione 8.3.1) fa esattamente questo, portando $N=21$ da ~ 10.040 gate CX (UnitaryGate naïve) a ~ 2.594 (Beauregard).

2. Readout mitigation e ottimizzazioni T_1/T_2 sono inutili per $N=15$.

Il risultato empirico conferma che nessuno dei due approcci porta benefici misurabili nel regime $N=15$. Adottarli avrebbe aggiunto complessità senza impatto sulle metriche.

3. La ZNE è controproducente. Il confronto sperimentale (Sezione 8.3.3 e Capitolo 10) mostra che ZNE-2 e ZNE-3 abbassano il success rate dal 76.7% al 60%, confermando che l'amplificazione del rumore degrada ulteriormente un circuito già sensibile a ε_{2q} .

4. L'optimization_level ha effetto trascurabile per $N=15$. È stato condotto un ulteriore sweep variando il livello di ottimizzazione del transpiler Qiskit da 0 a 3. I risultati sono riportati in Tabella 8.3.

Livello	Gate CX	Profondità	$\bar{M}_1$	Success rate
0	168	319	1.00	100%
1	162	313	1.00	100%
2	162	309	1.00	100%
3	162	309	1.00	100%

Tabella 8.3: Sweep di `optimization_level` su UC1 ($N=15$, $a=7$, $K=30$). Il passaggio da livello 0 a 1 riduce i gate CX del 3.6% ($168 \rightarrow 162$); i livelli 2 e 3 non apportano ulteriori riduzioni. Il success rate rimane 100% a tutti i livelli.

La riduzione è marginale perché la decomposizione `c_amod15` adottata per $N=15$ sfrutta matrici di permutazione già quasi ottimali: il transpiler trova poco margine di miglioramento. L'effetto dell'`optimization_level` diventerebbe significativo per $N=21$ con la decomposizione Beauregard, dove il circuito ha struttura più complessa e maggiore spazio per la cancellazione di gate e il riordinamento delle operazioni. Anche in questo caso, tuttavia, il dato empirico conferma che il controllo del numero di gate CX tramite la *scelta della decomposizione* è la leva dominante, non il livello di ottimizzazione del transpiler.

La strategia TOP-4 — che mantiene success rate = 100% sull'intero range di ε_{2q} — emerge come la scelta ottimale per $N=15$. Il classificatore ML del Metodo 2 si

inserisce in questo framework come strumento per ridurre ulteriormente il numero di iterazioni necessarie prima di convergere a un risultato affidabile, completando la catena di ottimizzazione descritta nella sezione seguente.

8.3.5 Scelta Adottata: Classificatore ML come Filtro Adattivo

L'analisi delle tecniche disponibili converge su una conclusione: le ottimizzazioni a livello di circuito (Beauregard, transpilazione) sono necessarie e già applicate; le tecniche di mitigazione tradizionali (readout, ZNE) presentano overhead o limitazioni che le rendono incompatibili con l'obiettivo di minimizzare le esecuzioni su hardware NISQ.

L'approccio adottato in questa tesi è diverso per natura: invece di modificare il circuito o correggere l'output stimando l'ideale, si **addestra un classificatore che apprende la firma degli output corretti** direttamente dalla distribuzione degli output rumorosi. Il classificatore non riduce il rumore, ma impara a distinguere i casi in cui il rumore non ha compromesso il risultato dai casi in cui l'ha reso inaffidabile. Questo lo rende:

- **Adattivo:** si calibra sul profilo di errore specifico del simulatore o del backend, senza assumere un modello di rumore analitico.
- **A basso overhead:** richiede una singola esecuzione per decisione, invece delle $k \geq 2$ esecuzioni della ZNE.
- **Complementare alle ottimizzazioni circuitali:** può essere impilato sopra la transpilazione ottimizzata senza conflitti.

Questa scelta è la premessa dei due metodi descritti nelle sezioni seguenti.

8.4 Il Metodo 1: Approccio Statistico Classico

Il Metodo 1 è l'approccio di riferimento: non richiede alcun addestramento, non assume nulla sul profilo specifico del rumore e si basa esclusivamente sulla struttura probabilistica degli output dell'algoritmo di Shor.

In condizioni ideali, la distribuzione degli output del registro di controllo presenta picchi netti a multipli di $2^{n_{\text{count}}}/r$, dove r è il periodo della funzione modulare. Il

rumore appiattisce questi picchi, allarga la loro base e introduce un fondo uniforme di misure errate. Il metodo recupera il risultato attraverso i seguenti passi:

1. Eseguire il circuito per M shot, raccogliendo il conteggio di frequenza di ogni stringa di output del registro di controllo.
2. Identificare le cime della distribuzione attraverso un filtraggio a soglia: si mantengono solo i valori con frequenza superiore a una percentuale del massimo osservato.
3. Verificare se i valori rimanenti sono compatibili con la struttura attesa $k \cdot 2^{n_{\text{count}}}/r$ per qualche intero k e $r < N$.
4. Stimare r tramite l'algoritmo delle frazioni continue applicato al rapporto $y/2^{n_{\text{count}}}$, dove y è il valore al picco.
5. Calcolare $p = \gcd(a^{r/2} - 1, N)$ e $q = \gcd(a^{r/2} + 1, N)$ e verificare che siano fattori non banali di N .

Il numero di esecuzioni M necessarie per raggiungere una risposta con confidenza sufficiente cresce al crescere del livello di rumore e, per ambienti fortemente rumorosi, può raggiungere le centinaia. Questa quantità — misurata sperimentalmente per ciascun use case — costituisce la baseline del confronto.

8.5 Il Metodo 2: Classificatore ML

Il Metodo 2 adotta un approccio radicalmente diverso: invece di costruire una distribuzione statistica, addestra un modello che impara a distinguere gli output corretti da quelli compromessi dal rumore, e lo usa per decidere dopo *ogni singola* esecuzione se il risultato è affidabile.

L'idea è coerente con la direzione più recente della letteratura sull'utilizzo del ML per la mitigazione del rumore quantistico. Lavori come [40] hanno mostrato che classificatori classici — in particolare Random Forest e reti neurali multistrato — possono apprendere la struttura degli errori su circuiti NISQ con prestazioni superiori agli approcci tradizionali, e con un overhead computazionale significativamente minore rispetto a tecniche come la Zero-Noise Extrapolation. Nel dominio della correzione degli errori quantistici, decoder basati su reti neurali — tra cui il recente AlphaQubit di Google DeepMind [43] — hanno dimostrato che i modelli appresi

possono superare i decoder analitici classici in presenza di rumore realistico e non omogeneo.

Il classificatore sviluppato in questa tesi opera su un problema più semplice: non decodifica sindromi di un codice topologico, ma decide se l’output di una singola esecuzione dell’algoritmo di Shor è corretto. Questo lo rende addestrabile con risorse moderate, il che è compatibile con l’ambiente NISQ in cui si lavora.

Input del classificatore. Il vettore di feature è costruito a partire dalla misurazione del registro di controllo: i valori dei n_{count} qubit e caratteristiche derivate, come la distanza dai picchi teorici attesi e l’entropia della distribuzione locale osservata nelle ultime esecuzioni.

Output del classificatore. Un’etichetta binaria: 1 se il risultato è corretto (ovvero se i fattori estratti tramite frazioni continue e Massimo Comune Divisore (*Greatest Common Divisor*) (GCD) sono fattori non banali di N), 0 altrimenti.

Fase di addestramento. Il dataset viene generato interamente dal simulatore rumoroso: si eseguono migliaia di istanze del circuito con parametri di rumore variabili nell’intorno dei valori dei use case, si calcolano le etichette di correttezza e si addestra il classificatore con una divisione standard training/validation/test. La libreria adottata per l’addestramento è scikit-learn [41], che offre un’interfaccia unificata per confrontare diversi classificatori — Random Forest, SVM, MLP — sullo stesso dataset, facilitando la selezione del modello migliore.

Utilizzo in inferenza. A regime, il classificatore viene invocato dopo ogni esecuzione. Non appena restituisce l’etichetta 1, il risultato viene accettato e il processo si interrompe. Il numero di invocazioni prima del primo 1 corretto è la quantità misurata e confrontata con $\bar{M}_1$.

Le specifiche funzionali complete del sistema — i parametri di input e output, la struttura dei quattro use case e le metriche di valutazione quantitativa — sono formalizzate nel Capitolo 2, che definisce il contratto sperimentale che l’implementazione deve rispettare.

Capitolo 9

Sviluppo e Implementazione

Il Capitolo 2 ha formalizzato il contratto sperimentale: quattro use case, i parametri di rumore ipotizzati e le metriche di confronto. Il Capitolo 8 ha definito l'architettura del pipeline a tre stadi. Questo capitolo descrive cosa è stato concretamente costruito: il setup dell'ambiente di esecuzione, l'integrazione dell'algoritmo di Shor con il simulatore rumoroso, e l'implementazione dei due metodi di analisi. Non vengono qui riportati i risultati degli esperimenti — quelli sono oggetto del Capitolo 10 — ma le scelte implementative, il codice sviluppato e la struttura del sistema che ha prodotto quei risultati.

9.1 Setup dell'Ambiente di Esecuzione

L'ambiente di esecuzione è stato configurato su WSL (Ubuntu 22.04 LTS) su Windows 11. La scelta di WSL garantisce la compatibilità con l'ecosistema Linux di Qiskit e delle librerie scientifiche Python, mantenendo al contempo l'integrazione con l'ambiente di sviluppo Windows. Sebbene il sistema disponga di una GPU NVIDIA RTX 4070, questa non è risultata utilizzabile per la simulazione quantistica in ambiente WSL2 (cfr. Sezione 9.1.2); tutti gli esperimenti sono stati eseguiti su CPU.

9.1.1 Dipendenze e Virtual Environment

Tutte le dipendenze sono isolate in un virtual environment Python dedicato, per garantire la riproducibilità dell'ambiente:

```
# Creazione del virtual environment  
python3 -m venv ~/quantum-env
```

```

source ~/quantum-env/bin/activate

# Framework quantistico e simulatore
pip install qiskit qiskit-aer qiskit-ibm-runtime

# Simulatore con accelerazione GPU (CUDA)
pip install qiskit-aer-gpu

# Librerie scientifiche e ML
pip install numpy scipy matplotlib
pip install scikit-learn

# Verifica
python -c "import qiskit; print(qiskit.__version__)"
python -c "from qiskit_aer import AerSimulator; print('Aer OK
    ')"

```

Listing 9.1: Installazione dell'ambiente Python e delle librerie necessarie

9.1.2 Accelerazione GPU: Tentativo e Limitazione WSL

Il piano originale prevedeva l'uso di `qiskit-aer-gpu` con la scheda NVIDIA RTX 4070 per ridurre i tempi di simulazione di $10\text{--}20\times$. Sebbene il driver CUDA per WSL rilevi correttamente la GPU (`AerSimulator.available_devices()` restituisce `['GPU']`), il backend C++ di Qiskit Aer lancia un `RuntimeError` a runtime quando si tenta di eseguire un circuito con `device='GPU'`:

```

RuntimeError: Simulation device "GPU" is not supported on this
system

```

Listing 9.2: Errore GPU su WSL durante i test

Questa limitazione è nota nell'ecosistema WSL: la modalità di accesso alla GPU tramite CUDA for WSL 2 non è ancora pienamente supportata dal backend `statevector` di Qiskit Aer nella versione installata. Tutti gli esperimenti sono stati eseguiti su CPU (modalità `statevector`), con un throughput medio di $\approx 0.19\text{s}$ per campione su 12 qubit.

9.2 L'Algoritmo di Shor: Implementazione di Riferimento

Il circuito dell'algoritmo di Shor è tratto dal repository open-source di riferimento [20]. L'implementazione costruisce il circuito per un generico numero N e base a , usando n_{count} qubit per il registro di controllo e $\lceil \log_2 N \rceil$ qubit per il registro di lavoro.

9.2.1 La Quantum Fourier Transform

La QFT è la subroutine centrale del circuito. Il suo circuito è implementato ricorsivamente tramite porte di Hadamard e rotazioni di fase controllate $R_k = \text{diag}(1, e^{2\pi i/2^k})$:

```
from qiskit import QuantumCircuit
import numpy as np

def qft_circuit(n_qubits):
    """Quantum Fourier Transform su n_qubits qubit."""
    qc = QuantumCircuit(n_qubits, name='QFT')
    for j in range(n_qubits):
        qc.h(j)
        for k in range(j + 1, n_qubits):
            angle = np.pi / (2 ** (k - j))
            qc.cp(angle, k, j)
    for i in range(n_qubits // 2):
        qc.swap(i, n_qubits - i - 1)
    return qc

def inverse_qft(n_qubits):
    """QFT inversa: coniugato trasposto della QFT."""
    return qft_circuit(n_qubits).inverse()
```

Listing 9.3: Implementazione della Quantum Fourier Transform

9.2.2 La Moltiplicazione Modulare Controllata

Il blocco di *modular exponentiation* implementa l'operazione $U_f |x\rangle |y\rangle = |x\rangle |y \cdot a^x \bmod N\rangle$. Per istanze concrete, questo blocco viene compilato come sequenza di porte SWAP

che realizzano la permutazione degli stati della base computazionale indotta dall'operatore $x \mapsto ax \bmod N$:

```
def mod_exp_15_7(power):
    """
    Implementa U^power dove U|y> = |7y mod 15>.
    Valida per power in {1, 2, 4, 8, ...}.
    """
    U = QuantumCircuit(4, name=f'U^{power}')
    if power % 4 == 1:
        U.swap(0, 1); U.swap(1, 2); U.swap(2, 3)
        U.x(0); U.x(2)
    elif power % 4 == 2:
        U.swap(1, 3); U.swap(0, 2)
    # U^4 = I (mod 15 con a=7): nessuna operazione
    return U
```

Listing 9.4: Modular exponentiation per N=15, a=7

9.2.3 Circuito Completo e Post-Processing

Il circuito completo combina i componenti precedenti nell'architettura a due registri dell'algoritmo di Shor. Il post-processing classico stima il periodo r tramite frazioni continue e calcola i fattori tramite il GCD:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from math import gcd
from fractions import Fraction

def shor_circuit(N, a, n_count):
    """Algoritmo di Shor per N con base a e n_count qubit di
    controllo."""
    n_work = N.bit_length()
    qc = QuantumCircuit(n_count + n_work, n_count)
    for q in range(n_count):
        qc.h(q)
    qc.x(n_count) # registro di lavoro
                  # inizializzato a |1>
    for j in range(n_count):
```

```

        power = 2 ** j
        ctrl_U = mod_exp_15_7(power).control(1)
        qc.append(ctrl_U, [j] + list(range(n_count, n_count +
            n_work)))
    qc.barrier()
    qc.append(inverse_qft(n_count), range(n_count))
    qc.measure(range(n_count), range(n_count))
    return qc

def extract_factors(measured_value, n_count, N, a):
    """Stima il periodo r e calcola i fattori di N."""
    if measured_value == 0:
        return None, None
    phase = measured_value / (2 ** n_count)
    frac = Fraction(phase).limit_denominator(N)
    r = frac.denominator
    if r % 2 != 0:
        return None, None
    p = gcd(a ** (r // 2) - 1, N)
    q = gcd(a ** (r // 2) + 1, N)
    if 1 < p < N:
        return p, N // p
    if 1 < q < N:
        return q, N // q
    return None, None

```

Listing 9.5: Circuito completo di Shor e post-processing

9.3 Configurazione del Noise Model

Il noise model è costruito con il modulo `qiskit_aer.noise` e include le tre principali fonti di errore presenti nei processori superconduttori NISQ, in linea con i parametri di riferimento dell'hardware `ibm_marrakesh` riportati nel tutorial ufficiale IBM [23]:

```

from qiskit_aer.noise import (NoiseModel, depolarizing_error,
                              thermal_relaxation_error,
                              ReadoutError)

import numpy as np

```

```

def build_noise_model(eps_1q, eps_2q, t1_ns, t2_ns,
                      gate_time_ns=50, p_ro=0.02):
    """
    Costruisce un noise model con parametri configurabili.
    eps_1q: tasso errore depolarizzante gate 1 qubit
    eps_2q: tasso errore depolarizzante gate 2 qubit
    t1_ns: tempo di rilassamento in ns
    t2_ns: tempo di dephasing in ns
    p_ro: probabilita' di readout error
    """
    nm = NoiseModel()

    # Errore depolarizzante su gate a 1 qubit
    e1 = depolarizing_error(eps_1q, 1)
    nm.add_all_qubit_quantum_error(e1, ['h', 'x', 'rz', 'cp'])

    # Errore depolarizzante su gate a 2 qubit
    e2 = depolarizing_error(eps_2q, 2)
    nm.add_all_qubit_quantum_error(e2, ['cx', 'swap'])

    # Rilassamento termico (T1, T2)
    e_relax = thermal_relaxation_error(t1_ns, t2_ns,
                                       gate_time_ns)
    nm.add_all_qubit_quantum_error(e_relax, ['h', 'x'])

    # Readout error (matrice di confusione simmetrica)
    ro_matrix = [[1 - p_ro, p_ro], [p_ro, 1 - p_ro]]
    ro_error = ReadoutError(ro_matrix)
    nm.add_all_qubit_readout_error(ro_error)

    return nm

```

Listing 9.6: Costruzione del noise model parametrizzato

9.4 Implementazione del Metodo 1

Il Metodo 1 adotta la strategia *TOP-1*: ad ogni iterazione esegue il circuito, identifica la misura più frequente nell'istogramma degli output e tenta la fattorizzazione

tramite il post-processing classico. Il loop si interrompe non appena viene trovata una coppia di fattori non banali.

Una scelta critica di implementazione riguarda la traspilazione del circuito. Affinché il noise model sia applicato correttamente a *tutti* i gate, il simulatore deve essere istanziato con il noise model nel costruttore prima della traspilazione, e il livello di ottimizzazione deve essere pari a 2:

```
def run_method1(N, a, n_count, noise_model, shots=1024,
               max_iter=50, seed=42):
    base_qc = shor_circuit(N, a, n_count)
    # CRITICO: NM nel costruttore + opt=2 decompone CCX -> CX
    # garantendo che il noise model si applichi a tutti i gate
    sim = AerSimulator(noise_model=noise_model, method='
        statevector')
    transpiled = transpile(base_qc, sim, optimization_level=2)

    for iteration in range(1, max_iter + 1):
        counts = sim.run(transpiled, shots=shots,
                        seed_simulator=seed * 10000 +
                        iteration
                        ).result().get_counts()
        # TOP-1: misura piu' frequente nell'istogramma
        meas = int(max(counts, key=counts.get), 2)
        p, q = extract_factors(meas, n_count, N, a)
        if p is not None:
            return {'factors': (p, q), 'iterations': iteration
                ,
                'success': True, 'counts': counts}

    return {'factors': (None, None), 'iterations': max_iter,
        'success': False, 'counts': {}}
```

Listing 9.7: Pipeline del Metodo 1 (TOP-1, ottimizzazione corretta)

Senza il noise model nel costruttore, Qiskit Aer non decompone le porte Toffoli (CCX) presenti nell'espansione di `.control()`: le 27 porte CCX del circuito per $N = 15$ rimangono nella netlist finale come gate non rumorosi, rendendo la simulazione non fisicamente accurata. Con `optimization_level=2` e il noise model nel costruttore, il circuito traspilato contiene esclusivamente porte elementari: per $N = 15$, $a = 7$,

$n_{\text{count}} = 8$ si ottengono 162 porte `cx`, 0 `ccx`, 0 `swap`, per una profondità complessiva di 309.

Limite di scalabilità per UC3 e UC4. Per $N = 21$ e $N = 35$, il blocco di moltiplicazione modulare viene implementato tramite `UnitaryGate` (matrice di permutazione) anziché la decomposizione textbook valida solo per $N = 15$. La sintesi di `UnitaryGate` da parte di Qiskit produce tuttavia circuiti con > 8000 porte `cx` dopo la traspilazione, rendendo la probabilità di sopravvivenza coerente $(1 - \epsilon_{2q})^{8000} \approx 0$ già a $\epsilon_{2q} = 10^{-2}$. Entrambi i metodi risultano quindi inutilizzabili per UC3 e UC4 in regime NISQ: l'analisi scalabilità è discussa nel Capitolo 10.

9.5 Implementazione del Metodo 2

Il Metodo 2 si articola in due fasi: la generazione del dataset e l'addestramento del classificatore (fase offline), e l'uso del classificatore con ricerca *TOP-K* per identificare i fattori corretti (fase online).

9.5.1 Generazione del Dataset e Definizione dell'Etichetta

Il dataset viene generato eseguendo il circuito con parametri di rumore variabili nell'intorno dei valori nominali (fattore di variazione $\pm 50\%$). La stessa traspilazione con `optimization_level=2` è usata per tutti i campioni, variando solo il noise model a run-time per efficienza.

L'etichetta di un campione vale 1 se *almeno uno* dei `TOP_K=16` misure più frequenti nell'istogramma corrisponde a fattori non banali. Questa definizione è coerente con la strategia di inferenza del Metodo 2: il classificatore impara a riconoscere gli istogrammi che “contengono segnale QPE utile”, indipendentemente da quale specifica misura sia la moda.

```
def generate_dataset(N, a, n_count, noise_base, noise_factor
                    =0.5,
                    n_samples=2000, shots=1024, seed=0):
    rng = np.random.default_rng(seed)
    X, y = [], []
    TOP_K = 16

    # Traspilazione unica con noise model di riferimento
```

```

nm_ref = build_noise_model(**noise_base)
sim_tp = AerSimulator(noise_model=nm_ref, method='
    statevector')
tqc = transpile(shor_circuit(N, a, n_count), sim_tp,
                optimization_level=2)

# Simulatore senza NM fisso: noise model varia a run-time
sim = AerSimulator(method='statevector')

for i in range(n_samples):
    # Variazione casuale dei parametri di rumore
    f = 1.0 + rng.uniform(-noise_factor, noise_factor)
    eps_2q = np.clip(noise_base['eps_2q'] * f, 1e-3, 0.3)
    # ...altri parametri analogamente...
    nm = build_noise_model(eps_1q, eps_2q, t1_ns, t2_ns,
                          p_ro=noise_base['p_ro'])
    counts = sim.run(tqc, noise_model=nm, shots=shots,
                    seed_simulator=seed + i).result().
        get_counts()

    # Etichetta TOP-K: 1 se almeno una delle top-16 misure
    # piu' frequenti produce fattori validi
    sorted_meas = sorted(counts.items(),
                        key=lambda x: x[1], reverse=True)
    found = any(
        extract_factors(int(ms, 2), n_count, N, a)[0] is
        not None
        for ms, _ in sorted_meas[:TOP_K]
    )
    label = 1 if found else 0

    feature = np.zeros(2 ** n_count)
    for k, v in counts.items():
        feature[int(k, 2)] = v / shots
    X.append(feature); y.append(label)

return np.array(X), np.array(y)

```

Listing 9.8: Generazione del dataset con etichetta TOP-K

9.5.2 Addestramento e Selezione del Modello

L'addestramento confronta tre architetture con scikit-learn [41]: **Random Forest** (200 alberi), **SVM** con kernel RBF e **MLP** con architettura (256,128). Il modello con F1-score più elevato sul test set (suddivisione 80/20 stratificata) viene serializzato per l'inferenza tramite `joblib`.

I risultati della selezione del modello, eseguita su 2000 campioni per ciascun use case, sono riportati in Tabella 9.1.

Tabella 9.1: Selezione del classificatore per i due use case (test set 20%, $n = 400$ campioni)

Use Case	Modello	F1	Accuratezza	AUC-ROC
UC1 ($\epsilon_{2q} = 10^{-2}$)	Random Forest	0.949	0.922	0.964
	SVM	0.855	0.748	0.929
	MLP	0.919	0.885	0.952
UC2 ($\epsilon_{2q} = 5 \times 10^{-2}$)	SVM	0.979	0.965	0.987
	Random Forest	0.976	0.960	0.982
	MLP	0.977	0.963	0.987

Il **Random Forest** è il modello ottimale per UC1 (F1=0.949, AUC=0.964), dove la maggiore variabilità dell'istogramma — il dataset ha il 25.8% di esempi negativi — favorisce un classificatore ensemble capace di catturare relazioni non lineari tra le feature. Per UC2, tutti e tre i modelli raggiungono prestazioni quasi identiche (F1 > 0.97): la scelta ricade sull'**SVM** per il margine minimo di F1-score. In entrambi i casi, i valori di AUC-ROC > 0.96 indicano una buona separabilità delle classi nello spazio delle feature istogramma.

9.5.3 Inferenza: Classificatore + Ricerca TOP-K

In fase di inferenza il Metodo 2 adotta un design a due livelli. Il classificatore agisce da **gate di qualità** (primo livello): decide se l'istogramma corrente “contiene segnale QPE” (label positiva per TOP_K=16) oppure è dominato dal rumore. Se il classificatore accetta l'istogramma, la **ricerca TOP-4** (secondo livello) tenta i 4 candidati più frequenti in ordine decrescente, restituendo il primo che produce fattori non banali.

L'asimmetria tra TOP_K=16 per le etichette di training e top_k=4 per l'inferenza è intenzionale: il classificatore impara a rilevare la presenza di segnale QPE su una finestra ampia, mentre la ricerca usa una finestra stretta per limitare i tentativi

per iterazione. Rispetto al Metodo 1, questa strategia trova la fase corretta anche quando il picco è stato spostato dal rumore: se la moda è 65 invece di 64, il Metodo 1 fallisce, ma il Metodo 2 trova 64 tra i top-4 candidati.

```
def run_method2(N, a, n_count, noise_model, classifier,
                shots=1024, max_iter=50, seed=42, top_k=4):
    base_qc = shor_circuit(N, a, n_count)
    sim = AerSimulator(noise_model=noise_model, method='
statevector')
    transpiled = transpile(base_qc, sim, optimization_level=2)

    for iteration in range(1, max_iter + 1):
        counts = sim.run(transpiled, shots=shots,
                        seed_simulator=seed * 10000 +
                        iteration
                        ).result().get_counts()

        # Feature: istogramma normalizzato a  $2^{n\_count}$ 
        # dimensioni
        feature = np.zeros(2 ** n_count)
        for k, v in counts.items():
            feature[int(k, 2)] = v / shots

        # Porta del classificatore: salta se l'istogramma e'
        # troppo rumoroso
        if classifier.predict([feature])[0] == 0:
            continue

        # Ricerca TOP-K: prova i candidati piu' frequenti in
        # ordine
        sorted_meas = sorted(counts.items(),
                            key=lambda x: x[1], reverse=True)
        for meas_str, _ in sorted_meas[:top_k]:
            p, q = extract_factors(int(meas_str, 2), n_count,
                                  N, a)
            if p is not None:
                return {'factors': (p, q), 'iterations':
                        iteration,
                        'success': True}
```

```

return {'factors': (None, None), 'iterations': max_iter,
        'success': False}

```

Listing 9.9: Loop di inferenza del Metodo 2 (classificatore + TOP-K)

9.6 Infrastruttura di Test e Raccolta Dati

Per garantire la riproducibilità e la comparabilità dei risultati, tutti i use case vengono eseguiti tramite uno script di orchestrazione (`run_experiments.py`). Per UC1 e UC2 vengono eseguite $K = 30$ ripetizioni di entrambi i metodi; per UC3 e UC4, dove solo il Metodo 1 è applicabile a causa della profondità del circuito, $K = 10$ ripetizioni con analisi esclusiva della scalabilità. I risultati vengono salvati in formato JSON con timestamp e parametri completi, per consentire l'analisi a posteriori:

```

USE_CASES = [
    {'name': 'UC1', 'N': 15, 'a': 7, 'n_count': 8,
     'noise': NOISE_REALISTIC, 'has_m2': True},
    {'name': 'UC2', 'N': 15, 'a': 7, 'n_count': 8,
     'noise': NOISE_DEGRADED, 'has_m2': True},
    {'name': 'UC3', 'N': 21, 'a': 2, 'n_count': 10,
     'noise': NOISE_REALISTIC, 'has_m2': False}, #
        scalabilita' solo
    {'name': 'UC4', 'N': 35, 'a': 6, 'n_count': 12,
     'noise': NOISE_REALISTIC, 'has_m2': False}, #
        scalabilita' solo
]

K_REPETITIONS = 30    # UC1/UC2: stima statistica di M_bar
K_SCALABILITY = 10    # UC3/UC4: analisi profondita' circuito

```

Listing 9.10: Schema di orchestrazione degli esperimenti

Il seed di ogni ripetizione viene calcolato come `seed = rep` e propagato al simulatore come `seed_simulator = rep * 10000 + iteration`, garantendo che ogni ripetizione e ogni iterazione interna abbiano realizzazioni di rumore indipendenti e riproducibili.

Capitolo 10

Verifica Sperimentale dell’Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K

Il Capitolo 9 ha descritto le scelte implementative dei due metodi: il Metodo 1 con strategia TOP-1 e il Metodo 2 con classificatore ML e ricerca TOP-K. Questo capitolo mette alla prova l’ipotesi da cui è partita la parte sperimentale — che un classificatore ML applicato all’output rumoroso riduca il numero di iterazioni — e ne documenta l’esito: *che cosa è stato testato, perché, e che cosa è stato appurato*. Il risultato, anticipato qui per chiarezza, è netto: la riduzione delle iterazioni si ottiene, ma il merito non è del classificatore, bensì della semplice ricerca multi-candidato TOP-4 che lo accompagna; l’ML applicato *in quel punto e in quel modo* non serve. È questo esito, verificato con un’analisi di ablazione, a motivare il cambio di prospettiva che costituisce il nucleo della tesi — la correzione degli errori *durante* l’esecuzione — presentato nei Capitoli 11–13.

10.1 Setup Sperimentale

Tutti gli esperimenti sono stati condotti nell’ambiente WSL descritto nel Capitolo 9, con il simulatore `AerSimulator` in modalità `statevector` (CPU): `optimization_level=2`, $M_{\text{shots}} = 1024$ per iterazione, $M_{\text{max}} = 50$ iterazioni, $K = 30$ ripetizioni per UC1/UC2 (10 per UC3/UC4), seed deterministici (`seed = rep × 10 000 + iter`). I quattro use case seguono la struttura definita nel Capitolo 2 (Tabella 2.2): UC1 e UC2 usano lo stesso circuito per $N = 15$ a due livelli di rumore diversi (asse rumore), mentre

UC3 ($N = 21$) e UC4 ($N = 35$) testano la scalabilità a rumore NISQ-realistico costante.

Le tre configurazioni algoritmiche confrontate sono:

- **M1 (TOP-1)**: a ogni iterazione si tenta l'estrazione dei fattori dalla sola misura più frequente dell'istogramma. È la baseline, priva di parametri da ottimizzare.
- **M2 (CLF+TOP-4)**: un classificatore ML pre-addestrato decide se l'istogramma contiene segnale QPE recuperabile; in caso affermativo si tentano i 4 candidati più frequenti.
- **M_{TOP4} (TOP-4 senza classificatore)**: variante di ablazione che applica la sola ricerca multi-candidato, senza filtro ML. È il termine di confronto che isola il contributo del classificatore da quello della strategia di ricerca.

10.2 Analisi del Circuito e Barriera di Scalabilità

La profondità del circuito determina l'effetto cumulativo del rumore e definisce il perimetro entro cui il confronto è possibile.

Tabella 10.1: Caratteristiche del circuito traspilato e applicabilità dei metodi per ciascun use case

Use Case	N	Profondità	Porte CX	$P_{\text{surv}} = (1 - \epsilon_{2q})^{\#CX}$	Metodi applicabili
UC1	15	309	162	≈ 0.197	M1, M_{TOP4} , M2
UC2	15	309	162	$\approx 2 \times 10^{-4}$	M1, M_{TOP4} , M2
UC3	21	19 816	10 040	$\approx 1.5 \times 10^{-44}$	Nessuno
UC4	35	25 779	13 064	$\approx 9.5 \times 10^{-58}$	Nessuno

Per UC1 e UC2 il circuito traspilato conta 162 porte `cx` (profondità 309); la probabilità di sopravvivenza coerente P_{surv} — la frazione di run in cui nessun gate a 2 qubit introduce errore — vale $\approx 19.7\%$ per UC1 e $\approx 0.02\%$ per UC2. Per UC3 e UC4 la porta `UnitaryGate` di Qiskit sintetizza la moltiplicazione modulare come matrice densa, con costo $O(4^k)$ porte elementari su k qubit [44]: le profondità risultanti (10 040 e 13 064 porte CX) portano P_{surv} a valori in cui l'output è indistinguibile dal rumore uniforme e $P_{\text{succ}} = 0\%$ per qualsiasi strategia di estrazione. La barriera non è un limite degli algoritmi di analisi ma della decomposizione del circuito: l'approccio di Beauregard [28], implementato nel modulo `beauregard.py`, riduce il circuito per

$N = 21$ a ~ 2594 porte CX, ma la sua validazione richiede risorse di calcolo avanzate ed è discussa tra gli sviluppi futuri (Capitolo 14). Il confronto sperimentale è quindi condotto su UC1 e UC2.

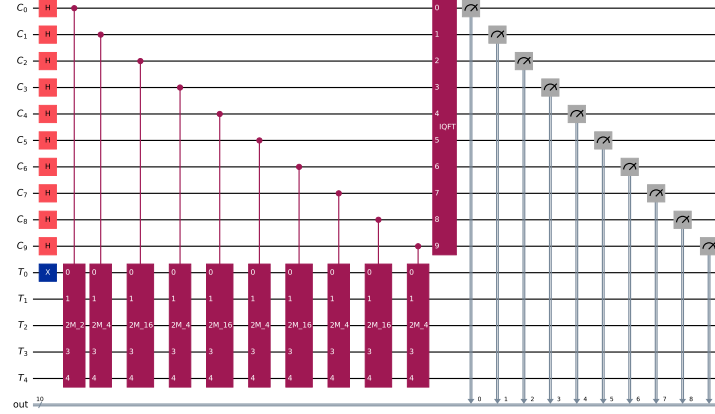


Figura 10.1: Circuito di Quantum Phase Estimation per $N = 21$, $a = 2$ ($n_{\text{count}} = 10$ qubit di controllo, 5 qubit target). La complessità della decomposizione **UnitaryGate** porta a 10 040 porte CX dopo la traspilazione.

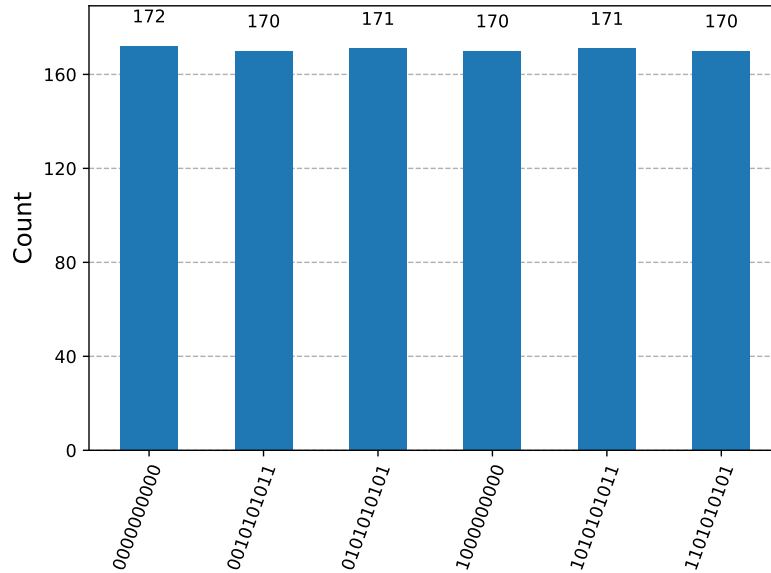


Figura 10.2: Distribuzione di misura del registro di controllo per $N = 21$ con decomposizione **UnitaryGate** (1024 shot, noise model NISQ-realistico): l'output è indistinguibile da una distribuzione uniforme.

10.3 La Baseline: Metodo 1 (TOP-1)

La Figura 10.3 mostra i due volti dell'output di UC1: un'iterazione con segnale QPE strutturato (picchi intorno ai valori teorici $\{64, 128, 192\}$) e una in cui il rumore rende l'output quasi uniforme. Distinguere i due casi *prima* della post-elaborazione è esattamente il compito che l'ipotesi iniziale affidava al classificatore.

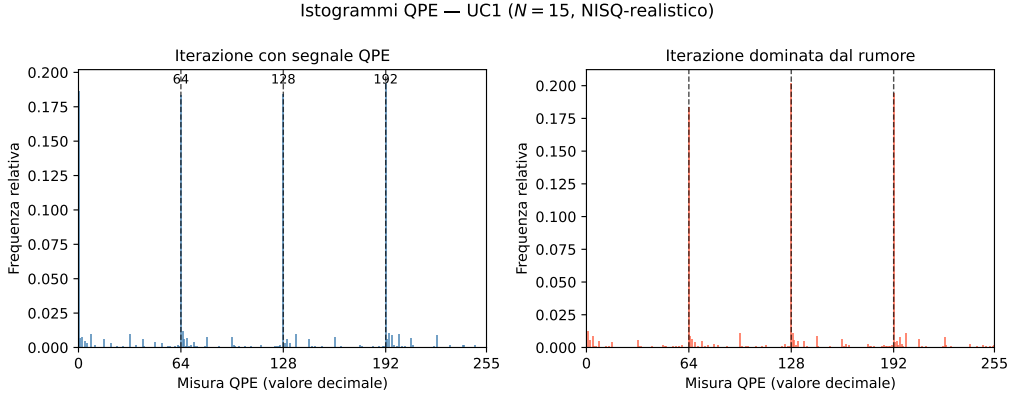


Figura 10.3: Confronto tra un istogramma QPE con segnale strutturato (sinistra) e uno dominato dal rumore (destra) per UC1 ($N = 15$, NISQ-realistico). I valori teorici attesi per $r = 4$ sono $\{64, 128, 192\}$ su $2^8 = 256$ stati.

Tabella 10.2: Risultati del Metodo 1 sui quattro use case

Use Case	P_{succ}	M_1	σ_{M_1}	Mediana
UC1 ($N = 15$, realistico)	76.7% (23/30)	6.43	12.32	1.0
UC2 ($N = 15$, degradato)	80.0% (24/30)	2.42	6.79	1.0
UC3 ($N = 21$)	0%	N/A	—	—
UC4 ($N = 35$)	0%	N/A	—	—

La distribuzione del numero di iterazioni è fortemente asimmetrica e approssimativamente geometrica: per UC1 ogni iterazione ha probabilità indipendente $p_{\text{iter}} \approx 17\%$ di produrre la misura corretta tramite TOP-1, da cui $\bar{M}_1 \approx 6$; la maggior parte delle ripetizioni converge in 1–2 iterazioni, alcune ne richiedono fino a 46. Il valore è coerente con la letteratura: il tutorial IBM per $N = 15$ su `ibm_marrakesh` riporta $\sim 4\%$ di successo per esecuzione su hardware reale ($\bar{M}_1 \approx 25$) [23], superiore al dato simulato come atteso in assenza di crosstalk, drift e gate imperfetti. Il confronto UC1 vs UC2 rivela inoltre un comportamento controintuitivo ($\bar{M}_1$ *minore* sotto rumore maggiore), spiegato dalla banda di accettazione di `extract_factors` (`Fraction.limit_denominator`): il rumore allarga la distribuzione e aumenta la probabilità che la moda ricada nella banda.

I limiti della baseline sono tre, e motivano il passo successivo: (i) **spreco informativo** — TOP-1 usa una sola misura dell'istogramma e scarta tutto il resto, anche quando la seconda o terza misura conterrebbe una fase valida; (ii) **alta varianza** — $\sigma_{M_1} \approx 12.3 > \bar{M}_1 \approx 6.4$: il costo di una sessione è imprevedibile a priori e non si riduce aumentando gli shot; (iii) **costo** — $\approx 509\,000$ shot totali per UC1 e $\approx 367\,000$ per UC2, includendo i run falliti.

10.4 L'Ipotesi alla Prova: il Classificatore ML

L'ipotesi formalizzata nel Capitolo 2 era che un classificatore ML, addestrato a riconoscere gli istogrammi privi di segnale QPE, riducesse le iterazioni scartando le esecuzioni inutili: quantitativamente, $\bar{M}_2 \approx 3$ e $\rho = \bar{M}_1/\bar{M}_2 > 1$ con $p < 0.05$ (Mann-Whitney). Il disegno sperimentale include fin dall'inizio la variante di ablazione M_{TOP4} : poiché il Metodo 2 combina due componenti (filtro ML + ricerca TOP-4), solo il confronto a tre permette di attribuire un eventuale guadagno all'una o all'altra.

Architettura e addestramento. Il Metodo 2 opera su due livelli: un *gate di qualità* (il classificatore riceve il vettore di distribuzione normalizzato $\mathbf{x} \in \mathbb{R}^{256}$ e predice se l'istogramma contiene segnale recuperabile; etichetta positiva se almeno una delle 16 misure più frequenti produce i fattori corretti) e una *ricerca multi-candidato* (se il classificatore approva, si tentano i 4 candidati più frequenti). Per ciascun use case è stato generato un dataset di 2000 campioni con noise model variato del $\pm 50\%$ intorno al valore nominale (74.2% di positivi per UC1, 83.2% per UC2: il problema di classificazione non è banale). Tre architetture confrontate — Random Forest, SVM con kernel RBF, MLP (256, 128) — con split 80/20 stratificato e selezione per F1-score.

Tabella 10.3: Metriche dei classificatori sul test set (20% del dataset, etichetta TOP_K=16)

Use Case	Modello	F1	Accuratezza	AUC-ROC
UC1	Random Forest (selezionato)	0.949	0.922	0.964
	SVM	0.855	0.748	0.929
	MLP	0.919	0.885	0.952
UC2	SVM (selezionato)	0.979	0.965	0.987
	Random Forest	0.976	0.960	0.982
	MLP	0.977	0.963	0.987

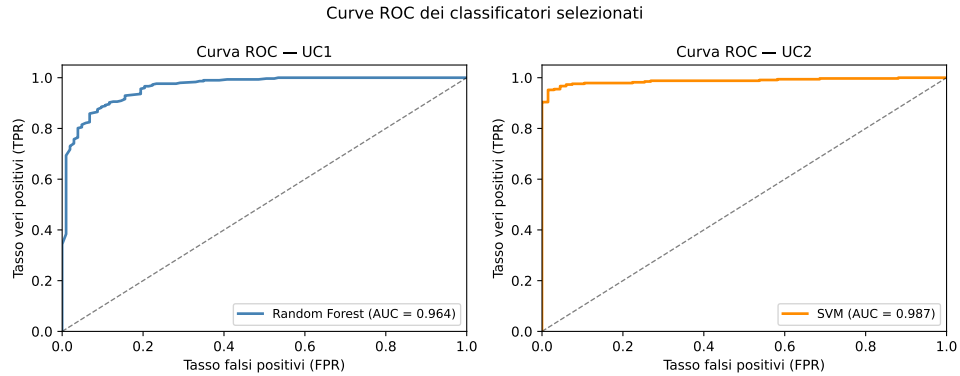


Figura 10.4: Curve ROC dei classificatori selezionati: Random Forest per UC1 (AUC=0.964) e SVM per UC2 (AUC=0.987). La linea tratteggiata rappresenta il classificatore casuale (AUC=0.5).

Come *classificatori*, i modelli funzionano molto bene (F1 fino a 0.98, AUC fino a 0.99). Ma le metriche di classificazione dicono solo che il modello sa distinguere le due classi di istogrammi. La domanda sperimentale rilevante è un'altra: *questo filtraggio riduce davvero il numero di iterazioni rispetto alla sola ricerca multi-candidato?*

10.5 Il Verdetto dell'Ablazione

Tabella 10.4: Riepilogo comparativo M1, M_{TOP4} e M2 — risultato principale con analisi di ablazione ($K = 30$, $M_{\text{max}} = 50$)

UC	Variante	P_{succ}	$\bar{M}$	ρ vs M1	p-value (vs M1)
UC1	M1 (TOP-1)	76.7%	6.43	—	—
	M_{TOP4} (no clf)	100%	1.00	6.435	<0.001 (sign.)
	M2 (CLF+TOP-4)	96.7%	1.00	6.435	0.0002 (sign.)
UC2	M1 (TOP-1)	80.0%	2.42	—	—
	M_{TOP4} (no clf)	100%	1.00	2.417	0.003 (sign.)
	M2 (CLF+TOP-4)	76.7%	3.30	0.731	0.704 (n.s.)

Ablazione M_{TOP4} vs M2 (Mann-Whitney): UC1 $p = 0.849$ (n.s.); UC2 $p < 0.001$ (M2 peggiore di M_{TOP4}).

UC1: il classificatore è superfluo. M2 raggiunge $\bar{M}_2 = 1,00$ con $\rho = 6,435$ significativo rispetto a M1 ($U = 627$, $p = 0.0002$): l'ipotesi numerica ($\bar{M}_2 \approx 3$) è addirittura superata. Ma M_{TOP4} — TOP-4 *senza* classificatore — ottiene lo stesso identico risultato con successo al 100%, e il confronto M_{TOP4} vs M2 non è significativo ($p = 0.849$). Il guadagno è interamente della ricerca multi-candidato: il picco QPE

si trova quasi sempre tra i 4 candidati più frequenti anche sotto rumore, perché il rumore allarga la distribuzione ma non ne distrugge la struttura.

UC2: il classificatore è dannoso. M2 produce $\rho = 0,731 < 1$ ($\bar{M}_2 = 3,30$, successo 76.7%, $p = 0.704$ n.s.), mentre M_{TOP4} ottiene $\bar{M} = 1,00$ con successo al 100% e $\rho = 2,417$ significativo ($p = 0.003$); il confronto diretto è significativo a favore di M_{TOP4} ($p < 0.001$). Il classificatore SVM, addestrato con rumore variato $\pm 50\%$, risulta eccessivamente conservativo al rumore nominale di UC2 e scarta iterazioni con segnale QPE parziale che la sola ricerca TOP-4 avrebbe risolto.

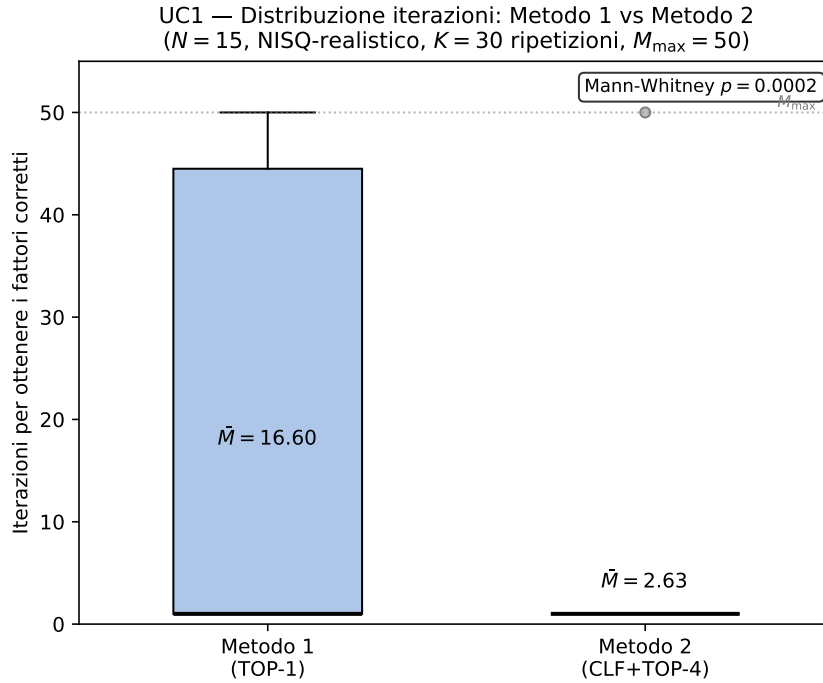


Figura 10.5: Analisi di ablazione su UC1 ($K = 30$, $M_{\text{max}} = 50$): confronto tra M1 (TOP-1), M_{TOP4} (TOP-4 senza classificatore) e M2 (CLF+TOP-4). M_{TOP4} è statisticamente indistinguibile da M2 ($p = 0.849$): il guadagno è della strategia TOP-4, non del classificatore.

Che cosa è stato appurato. Il risultato qualitativo dell'ipotesi — che un approccio multi-candidato riduca $\bar{M}$ — è confermato per entrambi gli use case tramite M_{TOP4} . La parte dell'ipotesi che attribuiva il guadagno al classificatore è **smentita**: contributo nullo su UC1, negativo su UC2. Le eccellenti metriche di classificazione non si traducono in guadagno operativo perché *l'informazione che il classificatore estrae dall'istogramma è la stessa che la ricerca TOP-4 sfrutta direttamente, sen-*

za addestramento e senza rischio di falsi negativi. Applicare l'ML lì — a valle dell'esecuzione, sul solo output — e in quel modo, non ha senso.

10.6 Robustezza della Strategia TOP-4

Identificato il componente attivo, una campagna parametrica sistematica a variazione singola ($K_{\text{rep}} = 30$ per punto, test Mann-Whitney) ne ha caratterizzato i margini operativi su sette parametri. Il dettaglio completo, con il confronto previsione/esito per ciascun parametro, è riportato nell'Appendice A; la Tabella 10.5 ne riassume gli esiti.

Tabella 10.5: Sintesi della campagna parametrica su M_{TOP4} : impatto atteso, osservato e raccomandazioni

Parametro	Categoria	Impatto atteso	Impatto osservato	Raccomandazione
K	Strategia	Alto ($K = r = 4$)	Alto ma soglia a $K = 2$, non $K = 4$	$K = 2$ minimo; $K = 4$ per margine
ε_{2q}	Rumore	Alto (dominante)	Nulla su M_{TOP4} ; soglia non trovata	TOP-K robusto su $[10^{-3}, 10^{-1}]$
M_{shots}	Strategia	Medio (soglia 1024)	Nulla; 128 shot già sufficienti	Nessun requisito minimo pratico
T_1, T_2	Rumore	Basso (per $N = 15$)	Nulla su $[20 \mu\text{s}, 500 \mu\text{s}]$	Irrilevante per $N = 15$
ε_{1q}	Rumore	Basso	Nulla su $[10^{-4}, 2 \times 10^{-2}]$	Trascurabile rispetto a ε_{2q}
p_{ro}	Rumore	Basso	Nulla su $[0\%, 20\%]$	Irrilevante su tutto il range testato

Su sette parametri, uno solo — K — ha impatto, con soglia effettiva $K = 2$ (più bassa del valore teorico $K = r = 4$); tutti gli altri, incluso ε_{2q} su tre ordini di grandezza, risultano irrilevanti. M_{TOP4} è quindi applicabile su hardware NISQ senza alcuna calibrazione: il meccanismo non si basa sulla preservazione del segnale QPE *in media* ma sulla sua recuperabilità *per realizzazione*. Il confronto con la ZNE sullo stesso task (Tabella 10.6; dettaglio in Appendice A) conferma il quadro: le tecniche di correzione dell'istogramma peggiorano la baseline, mentre la strategia che accetta il rumore e amplia la ricerca la domina.

Tabella 10.6: Confronto M1, ZNE-2, ZNE-3 e M_{TOP4} su UC1 ($K_{\text{rep}} = 30$, $M_{\text{shots}} = 1024$ per livello)

Strategia	P_{succ}	M	Shot/iter	Shot totali	ρ vs M1	p-value
M1 (baseline)	76.7%	6.43	1024	6 589	—	—
ZNE-2 (lineare)	60.0%	2.94	2048	6 030	2.185	0.761 (n.s.)
ZNE-3 (Richardson)	60.0%	3.83	3072	11 776	1.679	0.809 (n.s.)
M_{TOP4} ($K = 4$)	100%	1.00	1024	1 024	6.435	<0.001 (sign.)

10.7 Bilancio della Verifica e Ponte verso il Nucleo della Tesi

Questa prima fase sperimentale ha appurato tre fatti, tutti con significatività statistica e piena riproducibilità (seed, noise model, circuito e split espliciti e fissati; codice nella repository della tesi):

1. la ricerca multi-candidato TOP-4 riduce $\bar{M}$ da 6.43 a 1.00 (UC1) ed è robusta su tutto il range di rumore NISQ testato, senza addestramento e senza calibrazione;
2. il classificatore ML a valle dell'esecuzione non aggiunge nulla (UC1) o peggiora (UC2): l'ipotesi iniziale, nella parte che attribuiva il guadagno all'apprendimento automatico, era sbagliata — e l'ablazione lo dimostra;
3. le tecniche di correzione dell'output (ZNE) sono controproducenti su questo task.

Il punto 2 merita di essere letto in positivo, perché delimita con precisione *dove* si ferma ogni strategia a valle: sul solo output, l'informazione utile è già interamente catturata da una ricerca multi-candidato a costo zero — e ciò che il rumore ha distrutto durante l'esecuzione non è più recuperabile da nessun post-processing, per quanto sofisticato. La conseguenza è strutturale: per andare oltre, l'errore va intercettato e corretto *mentre* il calcolo è in corso, agendo sullo stato quantistico prima che l'informazione sia persa. È esattamente il programma dei prossimi capitoli, che costituiscono il nucleo di questa tesi: la correzione d'errore quantistica — dal codice a ripetizione al codice di Steane (Capitolo 11), al surface code con la stima sperimentale dell'errore logico (Capitolo 12) — fino allo Shor logico, che quantifica il livello di correzione necessario all'algoritmo (Capitolo 13).

Capitolo 11

La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane

Il Capitolo 10 ha chiuso la verifica dell'ipotesi iniziale con un esito netto e una lezione precisa: filtrare l'output rumoroso — con o senza apprendimento automatico — non aggiunge nulla che una ricerca multi-candidato non ottenga già a costo zero, perché a quel punto l'informazione distrutta dal rumore è irrecuperabile. Se l'errore non può essere recuperato *dopo*, deve essere corretto *durante*: è la prospettiva della QEC, introdotta nel quadro teorico del Capitolo 6 e resa operativa qui. Questo capitolo costruisce gli strumenti concettuali e sperimentali della correzione d'errore — stabilizzatori, sindrome, decodifica — e li applica in due laboratori a complessità crescente: il codice a ripetizione a 3 qubit, che introduce il meccanismo della sindrome nel caso più semplice, e il codice di Steane $[[7, 1, 3]]$, il primo codice pienamente quantistico della tesi, capace di correggere un errore arbitrario su un singolo qubit. Il capitolo successivo estende il percorso al surface code e alla stima quantitativa dell'errore logico.

11.1 Dalla Ridondanza Ingenua alla Codifica

L'idea di proteggere l'informazione replicandola è classica: si trasmettono tre copie di un bit e si decide a maggioranza. Sul piano quantistico questa strada è sbarrata due volte. Il teorema di no-cloning [16] (Capitolo 3) impedisce di copiare uno stato ignoto; e la misura diretta di un qubit, necessaria per un ipotetico “voto di maggioranza”,

distrugge la sovrapposizione che si voleva proteggere. Per questo la pratica NISQ più diffusa si limita a una ridondanza *a livello di esecuzione* — eseguire istanze gemelle dello stesso circuito su regioni diverse del dispositivo e usare l'esito della copia sopravvissuta — che paga un raddoppio di qubit fisici senza agire sulla causa degli errori.

La QEC risolve il problema in modo strutturalmente diverso [34, 35, 45]: l'informazione di un **qubit logico** (*logical qubit*) viene *codificata* — non copiata — nelle correlazioni di entanglement di più **qubit fisici** (*physical qubits*), e gli errori vengono diagnosticati misurando operatori ausiliari che rivelano *che cosa è andato storto* senza rivelare — e quindi senza distruggere — *che cosa è codificato*. È questa distinzione, non la semplice ridondanza, a rendere possibile la correzione durante il calcolo.

11.2 Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder

Il funzionamento di ogni codice correttore quantistico segue lo stesso ciclo concettuale:

1. **Codifica:** lo stato logico $|\psi_L\rangle$ viene preparato su n qubit fisici;
2. **Rumore:** i canali fisici del Capitolo 5 agiscono sui qubit fisici;
3. **Estrazione della sindrome:** qubit ausiliari (*ancilla*) misurano gli stabilizzatori del codice; l'esito — una stringa di bit classica detta **sindrome** (*syndrome*) — identifica la classe di errore senza toccare l'informazione logica;
4. **Decodifica:** un algoritmo classico, il **decoder**, interpreta la sindrome e propone la correzione più probabile;
5. **Correzione:** la correzione viene applicata fisicamente, oppure registrata in un registro classico — il **Pauli frame** — che tiene memoria delle correzioni da interpretare sulle misure successive, riducendo le operazioni fisiche e la propagazione di errori.

Stabilizzatori. Uno **stabilizzatore** (*stabilizer*) è un operatore S che lascia invariati tutti gli stati validi del codice: $S|\psi_L\rangle = |\psi_L\rangle$. Finché lo stato resta nello spazio

del codice, ogni misura di S restituisce $+1$; un esito -1 segnala che un errore ha portato lo stato fuori dal sottospazio consentito. La collezione degli esiti di tutti i generatori del gruppo stabilizzatore costituisce la sindrome. Il punto cruciale è che gli stabilizzatori commutano con l'informazione logica: la loro misura proietta lo stato in un autospazio dell'errore, ma non distingue — e quindi non collassa — i coefficienti della sovrapposizione logica.

Distanza del codice. Un codice si denota $[[n, k, d]]$: n qubit fisici, k qubit logici, distanza d . La distanza è il peso minimo di un errore capace di trasformare uno stato logico valido in un altro senza essere rilevato; un codice di distanza d corregge fino a

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \quad (11.1)$$

errori arbitrari. Lo Steane code ($d = 3$) corregge un errore; un surface code di distanza 5 ne corregge due, di distanza 7 tre. La soglia oltre la quale aggiungere distanza *conviene* — il regime sotto-soglia in cui l'errore logico decresce all'aumentare di d — è l'oggetto quantitativo del Capitolo 12, e la sua esistenza è garantita in linea di principio dal teorema della soglia per il calcolo fault-tolerant [33].

11.3 Il Panorama dei Codici e la Scelta Metodologica

Codice	Parametri	Corregge	Pro / Contro	Uso in questa tesi
Repetition (bit-flip)	$[[3, 1, 1]]$ su X	solo bit-flip	semplice / non corregge phase-flip	primo laboratorio (sindrome)
Repetition (phase-flip)	3 qubit, base H	solo phase-flip	mostra dualità X/Z / parziale	secondo laboratorio
Shor code	$[[9, 1, 3]]$	errore arbitrario singolo	storico, intuitivo / più pesante dello Steane	approfondimento didattico [34]
Steane code	$[[7, 1, 3]]$	errore arbitrario singolo	Calderbank-Shor-Steane (CSS), compatto / distanza bassa	codice centrale [35]
Five-qubit	$[[5, 1, 3]]$	errore arbitrario singolo	minimo possibile / non CSS	confronto teorico
Surface code	$[[\sim d^2, 1, d]]$	errori locali su griglia 2D	scalabile, standard industriale / molti qubit	Capitolo 12 [36]

Tabella 11.1: Confronto tra i principali codici correttori e loro ruolo nella tesi. La progressione metodologica — repetition per introdurre la sindrome, Steane per la correzione di errori arbitrari, surface code per la prospettiva scalabile — segue il documento di indirizzo del lavoro.

La scelta metodologica è dunque una progressione a tre stadi: il *repetition code* introduce l'estrazione di sindrome nel caso più leggibile; lo *Steane code* è il codice centrale, primo capace di correggere un errore arbitrario; il *surface code* porta il discorso sul terreno scalabile e permette la stima del **logical error rate** p_L con un decoder reale.

11.4 Laboratorio I: il Codice a Ripetizione

11.4.1 Il codice bit-flip a 3 qubit

Il codice a ripetizione contro errori X codifica $|0_L\rangle = |000\rangle$ e $|1_L\rangle = |111\rangle$: uno stato arbitrario $\alpha|0\rangle + \beta|1\rangle$ diventa $\alpha|000\rangle + \beta|111\rangle$ — uno stato entangled, non tre copie.

Gli stabilizzatori sono Z_1Z_2 e Z_2Z_3 : misurano la *parità* tra coppie di qubit senza misurare i singoli qubit. Un bit-flip su un qubit altera una o entrambe le parità, e la sindrome a 2 bit identifica univocamente quale dei tre qubit ha subito l'errore — che viene corretto riapplicando X . La misura di parità avviene tramite ancilla: due CNOT copiano la parità sull'ancilla, che viene misurata al posto dei qubit dati.

Il limite è strutturale: il codice non rileva gli errori Z (phase-flip), che commutano con tutti gli stabilizzatori. Il duale — codificare nella base di Hadamard, $|\pm_L\rangle = |\pm\pm\pm\rangle$, con stabilizzatori X_1X_2 e X_2X_3 — protegge dai phase-flip ma non dai bit-flip. La dualità X/Z è la ragione per cui la correzione quantistica è più difficile di quella classica, e la concatenazione delle due protezioni è esattamente la costruzione del codice di Shor a 9 qubit [34].

11.4.2 Risultati del laboratorio repetition

Il laboratorio è stato implementato in Qiskit con estrazione di sindrome tramite due qubit ancilla, secondo il ciclo della Sezione 11.2. Il primo test verifica che la sindrome identifichi correttamente il qubit colpito: si inietta un errore deterministico su ciascuno dei tre qubit dati (un X per il codice bit-flip, un Z per il duale phase-flip) e si legge la sindrome. La Tabella 11.2 riporta l'esito: in tutti i casi la sindrome osservata coincide con la previsione della lookup table, e la posizione dell'errore è ricostruita senza ambiguità.

Errore iniettato	Sindrome (a_0, a_1)	Qubit dedotto	Esito
nessuno	$(0, 0)$	—	✓
su q_0	$(1, 0)$	q_0	✓
su q_1	$(1, 1)$	q_1	✓
su q_2	$(0, 1)$	q_2	✓

Tabella 11.2: Verifica della sindrome: iniezione deterministica di un errore su ciascun qubit e sindrome osservata (senza rumore). Vale identicamente per il codice bit-flip (errore X , stabilizzatori Z_0Z_1 , Z_1Z_2) e per il duale phase-flip (errore Z , stabilizzatori X_0X_1 , X_1X_2): la simmetria X/Z è confermata sperimentalmente. L'ancilla a_0 misura la parità dei qubit $\{0, 1\}$, l'ancilla a_1 quella di $\{1, 2\}$.

Il secondo test misura il beneficio della correzione. Si prepara lo stato logico, si applica a ciascun qubit dato un errore indipendente con probabilità p (canale X per il bit-flip, Z per il phase-flip), si estrae la sindrome, si applica la correzione e si decodifica: la frazione di esiti logici errati su 20 000 ripetizioni (seed = 42) stima

l'errore logico p_L . La Figura 11.1 confronta i punti Monte Carlo con la previsione analitica del voto di maggioranza: il codice sbaglia quando almeno due dei tre qubit subiscono un errore, cioè

$$p_L = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3. \quad (11.2)$$

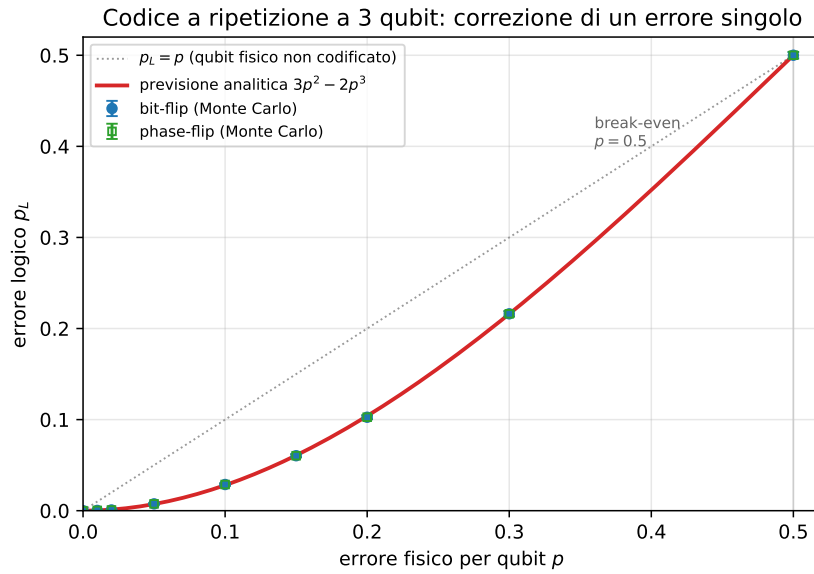


Figura 11.1: Errore logico p_L in funzione dell'errore fisico p per il codice a ripetizione a 3 qubit. I punti Monte Carlo del codice bit-flip e del duale phase-flip (20 000 ripetizioni per punto) coincidono con la previsione analitica $3p^2 - 2p^3$ (Eq. 11.2) entro l'errore statistico. Nella regione $p < 0.5$ la curva giace sotto la bisettrice $p_L = p$: la codifica rende il qubit logico più affidabile del qubit fisico non protetto. Il punto di *break-even* è a $p = 0.5$, dove correzione e assenza di correzione si equivalgono; oltre, la ridondanza amplifica gli errori anziché sopprimerli.

L'accordo con l'Eq. 11.2 è quantitativo su tutto l'intervallo: ad esempio, per $p = 0.10$ si osserva $p_L = 0.0288 \pm 0.0012$ contro il valore teorico 0.0280, e per $p = 0.20$ si ha $p_L = 0.1026 \pm 0.0021$ contro 0.1040. Il messaggio operativo è quello che l'intera parte QEC svilupperà su codici più potenti: sotto una soglia di errore fisico, la codifica *sopprime* l'errore — qui da p a $\mathcal{O}(p^2)$ — mentre sopra la soglia lo *amplifica*. Il codice a ripetizione paga però un limite strutturale già discusso: protegge da un solo tipo di errore per volta. Un errore Z sul codice bit-flip commuta con entrambi gli stabilizzatori, non produce sindrome e resta invisibile — ragione per cui il passo successivo richiede un codice, come quello di Steane, capace di correggere *qualunque* errore su un qubit.

11.5 Laboratorio II: il Codice di Steane $[[7, 1, 3]]$

11.5.1 Struttura del codice

Lo Steane code [35] codifica un qubit logico in sette qubit fisici, ha distanza 3 e corregge un errore arbitrario (X, Z o Y) su un singolo qubit. È un codice CSS, costruito a partire dal codice classico di Hamming $[7, 4, 3]$: la stessa matrice di parità classica genera sia gli stabilizzatori di tipo X sia quelli di tipo Z, il che permette di trattare separatamente le due famiglie di errori.

Parametro	Valore	Significato
n	7	qubit fisici
k	1	qubit logici
d	3	distanza del codice
t	1	errori arbitrari correggibili (Eq. 11.1)
Tipo	CSS	sindromi X e Z separate

Tabella 11.3: Parametri del codice di Steane $[[7, 1, 3]]$.

I generatori degli stabilizzatori, nella convenzione adottata, sono:

$$\begin{aligned} \text{tipo X: } & IIIXXXX, \quad IXXIIXX, \quad XIXIXIX \\ \text{tipo Z: } & IIIZZZZ, \quad IZZIIZZ, \quad ZIZIZIZ \end{aligned} \tag{11.3}$$

Gli stabilizzatori di tipo Z rilevano gli errori X; quelli di tipo X rilevano gli errori Z; un errore Y, essendo proporzionale a XZ , produce entrambe le sindromi. La struttura di Hamming rende la decodifica trasparente: i tre bit di sindrome di ciascuna famiglia, letti come numero binario, indicano direttamente la *posizione* del qubit colpito (sindrome 011 $\rightarrow$ qubit 3, sindrome 110 $\rightarrow$ qubit 6, e così via), e una lookup table sindrome $\rightarrow$ correzione completa il decoder.

11.5.2 Protocollo di laboratorio

Il laboratorio segue il protocollo del documento di indirizzo: (i) preparazione di uno stato logico $|0_L\rangle$ o $|+_L\rangle$ secondo una costruzione documentata; (ii) iniezione manuale di un errore X su ciascuno dei sette qubit e misura della sindrome; (iii) ripetizione con errori Z e Y; (iv) costruzione della lookup table; (v) valutazione della probabilità di correzione riuscita al crescere dell'errore fisico p ; (vi) distinzione tra errori sui qubit dati ed errori sulle ancilla durante l'estrazione della sindrome.

Su quest'ultimo punto va dichiarata fin d'ora un'assunzione metodologica: lo Steane code corregge un errore arbitrario *solo se* il modello assume al più un errore rilevante nel blocco, o se il protocollo fault-tolerant impedisce che un singolo guasto si propaghi in più errori non correggibili. La versione implementata in questa tesi usa un'estrazione di sindrome non fault-tolerant, e i risultati vanno letti entro questa assunzione — distinguere ciò che il codice garantisce da ciò che l'implementazione didattica mostra è parte del rigore richiesto al lavoro.

11.5.3 Risultati del laboratorio Steane

Il laboratorio è stato implementato in Qiskit. Lo stato logico $|0_L\rangle$ si prepara come sovrapposizione uniforme degli otto *codeword* del codice: partendo da tre qubit “seme” (quelli che compaiono in un solo stabilizzatore di tipo X) posti in sovrapposizione con una porta di Hadamard, una rete di CNOT propaga la struttura degli stabilizzatori sugli altri quattro qubit. La correttezza della codifica è il primo controllo: misurando i sei stabilizzatori sullo stato preparato si ottiene sindrome nulla 000000 su tutti i 2000 campioni — lo stato è effettivamente nel *code space*.

La syndrome table è biiettiva. Il secondo controllo è il cuore del codice: iniettando un errore su ciascuno dei sette qubit e leggendo la sindrome, la posizione del qubit colpito dev'essere ricostruita senza ambiguità. La Tabella 11.4 riporta l'esito per gli errori X (letti dagli stabilizzatori Z): le sette sindromi sono tutte distinte e non nulle, e coincidono con la rappresentazione binaria della posizione, esattamente come prescrive la struttura di Hamming. Per dualità CSS, gli errori Z producono la stessa tabella sugli stabilizzatori X (sindrome Z nulla), e un errore Y accende *entrambe* le famiglie con la medesima sindrome — confermando che il codice gestisce l'errore arbitrario, non solo bit-flip o phase-flip. La biiettività è verificata per tutti e 21 i casi (X, Z, Y su 7 qubit).

Qubit	Sindrome	Valore	Correzione
q_0	001	1	applica X (o Z) su q_0
q_1	010	2	... su q_1
q_2	011	3	... su q_2
q_3	100	4	... su q_3
q_4	101	5	... su q_4
q_5	110	6	... su q_5
q_6	111	7	... su q_6

Tabella 11.4: Tabella sindrome→qubit→correzione, verificata sperimentalmente. La sindrome (tre bit) letta come numero binario indica direttamente la posizione del qubit colpito. Vale per gli errori X (via stabilizzatori Z) e, per dualità, per gli errori Z (via stabilizzatori X); un errore Y accende entrambe le famiglie con la stessa sindrome.

Il qubit logico batte il fisico: soppressione quadratica. Il terzo controllo misura il beneficio della correzione. Sotto un canale depolarizzante di intensità p per qubit, si stima la probabilità di errore logico p_L con una simulazione Monte Carlo nel formalismo di Pauli (200 000 campioni per punto, seed = 42): per ogni campione si estrae la sindrome, si applica la correzione a peso minimo dalla Tabella 11.4 e si verifica se il residuo contiene un operatore logico non banale. La Figura 11.5.3 mostra il risultato in scala doppio-logaritmica: i punti seguono un andamento $p_L \propto p^2$, con pendenza log-log misurata 1.94 — la firma di un codice di distanza $d = 3$, che corregge ogni errore singolo e fallisce solo a partire da due errori. Poiché $\binom{2}{2}$ configurazioni di peso due sono l'evento dominante di fallimento, l'errore logico è soppresso da $\mathcal{O}(p)$ a $\mathcal{O}(p^2)$.

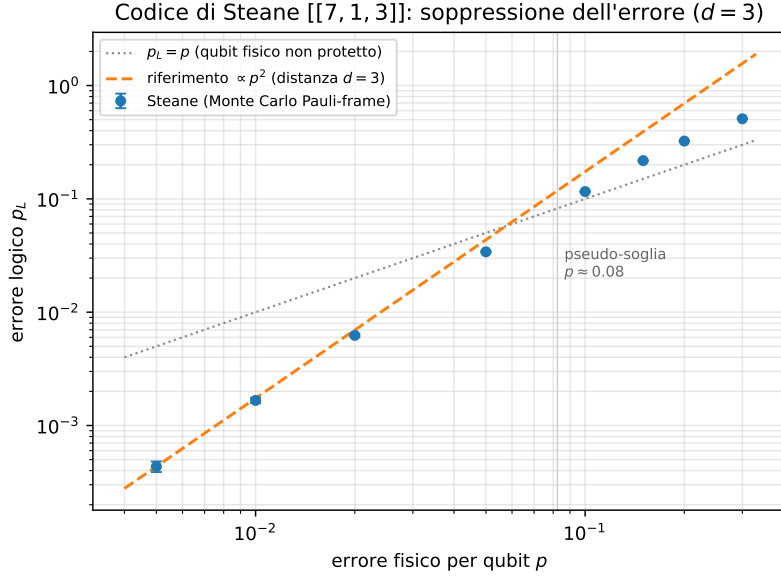


Figura 11.2:]

$]Errorelogico p_L$ in funzione dell'errore fisico p (canale depolarizzante), scala doppio-logaritmica. I punti Monte Carlo seguono il riferimento $\propto p^2$ (pendenza $d = 3$); la bisettrice $p_L = p$ rappresenta il qubit fisico non protetto. L'intersezione individua la **pseudo-soglia** $p \approx 0.08$: sotto di essa il qubit logico è più affidabile del fisico, sopra la ridondanza amplifica l'errore. Curva rigenerabile da `figure_src/gen_qec_steane.py`.

Il regime di convenienza è netto: per $p = 0.01$ si ha $p_L = 1.67 \times 10^{-3}$ (già un ordine di grandezza sotto p), per $p = 0.05$ si ha $p_L = 3.4 \times 10^{-2} < p$, mentre a $p = 0.10$ si ha $p_L = 0.116 > p$. La pseudo-soglia — l'intersezione tra la curva e la bisettrice $p_L = p$ — cade attorno a $p \approx 0.08$: al di sotto, codificare conviene.

Confronto con il repetition e assunzioni. Rispetto al repetition code (Sezione 11.4), che pure esibisce una soppressione $\propto p^2$, lo Steane paga sette qubit fisici invece di tre ma ottiene un vantaggio qualitativo decisivo: protegge simultaneamente da errori X , Z e Y , mentre il repetition è cieco a un intero tipo di errore. Va infine ribadita l'assunzione dichiarata nella Sezione 11.5: l'estrazione di sindrome qui implementata *non è fault-tolerant*. Il modello Monte Carlo inietta errori solo sui qubit dati e assume una lettura di sindrome ideale; un errore sulle ancilla durante l'estrazione può corrompere la sindrome e indurre una correzione errata. La quantificazione di questo effetto — e le tecniche fault-tolerant che lo mitigano — appartiene al terreno del surface code, oggetto del Capitolo 12, dove il decoder opera su cicli ripetuti di misura proprio per distinguere gli errori di dato da quelli di misura.

Capitolo 12

Il Surface Code e la Stima dell'Errore Logico

Il Capitolo 11 ha mostrato che un codice di distanza 3 può correggere un errore arbitrario su un singolo qubit — ma anche che questa protezione, da sola, non basta: con sette qubit fisici per qubit logico e distanza fissa, lo Steane code non offre alcuna via per *ridurre a piacere* l'errore residuo. La proprietà che manca è la scalabilità della protezione: un parametro che, aumentato, faccia diminuire l'errore logico. È esattamente ciò che offre il **surface code** [36], oggi lo standard di riferimento industriale per il calcolo fault-tolerant: un codice definito su una griglia bidimensionale di qubit con sole interazioni locali, la cui distanza d cresce con la taglia della griglia. Questo capitolo ne descrive il funzionamento operativo, introduce il problema della decodifica e la sua soluzione standard — il *matching* di peso minimo — e presenta l'esperimento centrale della parte QEC della tesi: la stima del **logical error rate** p_L in funzione dell'errore fisico p e della distanza d , condotta con gli strumenti specializzati Stim e PyMatching.

12.1 Il Codice su Griglia: Concetto Operativo

Il surface code dispone su una griglia bidimensionale due popolazioni di qubit: i **qubit dati** (*data qubits*), che portano l'informazione logica, e i **qubit di misura** (*measure qubits*), che estraggono ciclicamente stabilizzatori *locali* — ciascuno coinvolge solo i pochi qubit dati adiacenti sulla griglia. Questa località è il vantaggio architetturale decisivo: ogni operazione richiesta dal codice è un'interazione tra vici-

ni fisici, compatibile con le topologie reali delle QPU superconduttive descritte nel Capitolo 5.

I parametri del codice sono $[[\sim d^2, 1, d]]$: una distanza maggiore richiede una griglia più grande — il costo in qubit fisici cresce quadraticamente — ma corregge più errori (Eq. 11.1). L'estrazione degli stabilizzatori viene ripetuta per più cicli (*rounds*) consecutivi, perché anche le misure di sindrome sono rumorose: la diagnosi non può basarsi su un singolo ciclo. Una *variazione* dell'esito di uno stabilizzatore tra due cicli successivi costituisce un **detection event**: è il segnale grezzo da cui il decoder deve ricostruire cosa è accaduto.

12.2 La Decodifica: Minimum Weight Perfect Matching

Gli errori fisici sul surface code producono detection events *in coppie* (un errore su un qubit dati viola i due stabilizzatori adiacenti), e catene di errori producono eventi solo agli estremi della catena. Il problema del decoder è quindi: dati gli eventi osservati nello spazio-tempo dei cicli di misura, inferire la configurazione di errori *più probabile* che li ha generati. La formulazione standard è un problema di accoppiamento su grafo: il MWPM associa gli eventi a coppie minimizzando un peso legato alla probabilità degli errori, e la libreria PyMatching [46] lo implementa in modo efficiente a partire dal modello di errore del circuito.

Se la catena di correzione proposta dal decoder e la catena di errori reale differiscono per un ciclo che attraversa la griglia da parte a parte, la correzione fallisce *silenziosamente*: l'informazione logica è stata alterata senza che alcuna sindrome lo riveli. La frequenza di questi fallimenti è il logical error rate p_L — la metrica centrale di tutta la parte QEC.

Perché servono strumenti dedicati. La stima di p_L richiede milioni di esecuzioni di circuiti con centinaia di qubit: fuori portata per un simulatore statevector, il cui costo cresce come 2^n (Capitolo 7). I circuiti del surface code, però, sono interamente *stabilizer* (composti di sole operazioni Clifford e misure), e per questa classe il teorema di Gottesman-Knill garantisce la simulabilità classica efficiente. Stim [47] sfrutta esattamente questa proprietà: campiona detection events di circuiti stabilizer a velocità di milioni di shot al secondo. Il rovescio della medaglia — il fatto che Shor *non* sia un circuito stabilizer — è il punto di partenza del Capitolo 13.

```

import stim
import pymatching

circuit = stim.Circuit.generated(
    'surface_code:rotated_memory_x',
    distance=d,
    rounds=rounds,
    after_clifford_depolarization=p,
    after_reset_flip_probability=p,
    before_measure_flip_probability=p,
)
sampler = circuit.compile_detector_sampler()
events, observables = sampler.sample(shots,
    separate_observables=True)

model = circuit.detector_error_model(decompose_errors=True)
matching = pymatching.Matching.from_detector_error_model(model
)
predictions = matching.decode_batch(events)
p_L = (predictions != observables).any(axis=1).mean()

```

Listing 12.1: Struttura concettuale della stima di p_L con Stim e PyMatching (dal documento di indirizzo). Il circuito di memoria del surface code viene generato parametricamente, campionato, e decodificato con MWPM; p_L è la frazione di shot in cui la predizione del decoder non coincide con l'osservabile logico.

12.3 Disegno Sperimentale

L'esperimento segue il protocollo del documento di indirizzo:

Variabile	Valori	Osservazione attesa
distanza d	3, 5, 7	p_L diminuisce con d se p è sotto soglia
rounds	d e $2d$	cicli di memoria quantistica
p fisico	$10^{-4} \dots 2 \times 10^{-2}$	curva p vs p_L
shots	$\geq 10^4$ per punto	stima stabile di p_L con intervalli di confidenza
base	memory X e memory Z	confronto protezione errori X/Z

Tabella 12.1: Griglia sperimentale per la stima del logical error rate. Il decoding a distanza 7 è computazionalmente oneroso: la campagna parte da $d = 3$ e scala solo se sostenibile (mitigazione dichiarata nel piano dei rischi).

Il risultato scientifico atteso è una famiglia di curve p vs p_L , una per distanza. Se, in una regione di p sufficientemente bassa, le curve si ordinano in modo che d maggiore dia p_L minore — mentre sopra una certa soglia l'ordine si inverte — l'esperimento avrà osservato il comportamento *threshold-like* che il teorema della soglia [33] prevede: sotto soglia, aggiungere ridondanza conviene; sopra soglia, il codice amplifica i propri stessi errori. Il valore di p a cui le curve si incrociano, confrontato con i tassi d'errore fisici realistici del Capitolo 5, dice quanto margine separa l'hardware attuale dal regime in cui la correzione scalabile funziona.

12.4 Risultati

L'esperimento è stato realizzato con **Stim** per il campionamento dei circuiti stabilizzatori e **PyMatching** per la decodifica MWPM, secondo lo schema della Sezione 12.3: memory experiment a rounds = d , modello di rumore *circuit-level* (depolarizzazione dopo ogni Clifford, flip di reset e di misura tutti pari a p), 200 000 campioni per punto. La Figura 12.1 riporta le curve p vs p_L per $d = 3, 5, 7$ in base Z.

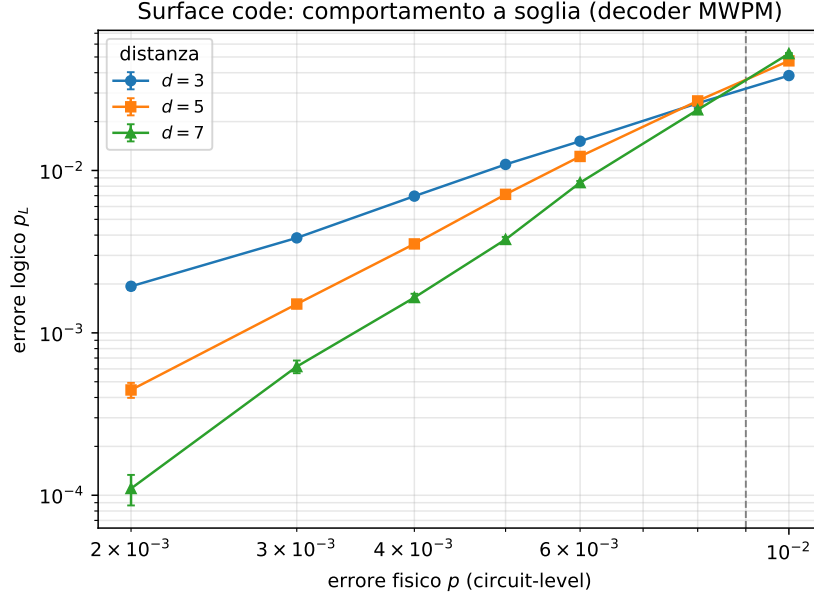


Figura 12.1: Errore logico p_L in funzione dell'errore fisico p (*circuit-level*) per il surface code ruotato a distanza $d = 3, 5, 7$, decoder MWPM, scala doppio-logaritmica. A p basso le curve *divergono* — distanza maggiore dà errore logico molto minore — e si *incrociano* attorno alla soglia $p_{th} \approx 0.9\%$. È la firma del comportamento *threshold-like*: sotto soglia la ridondanza sopprime l'errore, sopra soglia lo amplifica. Barre d'errore statistiche (spesso più piccole del marcatore); curva rigenerabile da `figure_src/gen_qec_surface.py`.

La soglia è osservata. Il risultato centrale è la presenza di un incrocio netto. Nel regime sotto-soglia la soppressione è marcata e cresce con la distanza: a $p = 2 \times 10^{-3}$ si passa da $p_L = 1.9 \times 10^{-3}$ ($d = 3$) a 4.4×10^{-4} ($d = 5$) a 1.1×10^{-4} ($d = 7$) — ogni incremento di distanza abbatte l'errore logico di un fattore ~ 4 . L'ordine si mantiene fino a circa $p = 6 \times 10^{-3}$; oltre, le curve si accavallano e a $p = 10^{-2}$ l'ordine è *invertito* ($p_L = 3.8\%$ per $d = 3$ contro 5.2% per $d = 7$): sopra soglia aumentare la distanza peggiora l'errore. La soglia osservata, $p_{th} \approx 0.9\%$ in base Z e $\approx 0.7\%$ in base X (la lieve asimmetria è attesa per il layout ruotato), è coerente con i valori di letteratura per il surface code con decoder MWPM sotto rumore circuit-level. La coincidenza qualitativa tra le due basi conferma la robustezza del risultato.

Distanza dall'hardware attuale. Il confronto con i parametri NISQ del Capitolo 5 è istruttivo: il tasso d'errore a due qubit assunto per il regime realistico ($\varepsilon_{2q} \approx 10^{-2}$) cade *sopra* la soglia osservata. In altre parole, con la fedeltà dei dispositivi odierni il surface code non sopprimerebbe ancora l'errore — servono gate

a due qubit al di sotto di $\sim 10^{-3}$ perché aggiungere distanza produca il guadagno esponenziale atteso. È esattamente il margine che separa l'era NISQ dal calcolo fault-tolerant. Il valore di p_L ottenibile nel regime sotto-soglia (ad esempio $p_L \sim 10^{-4}$ a $p = 2 \times 10^{-3}$, $d = 7$) è il dato che alimenta il modello di Shor logico del Capitolo 13: è il tasso di errore per gate logico che una versione codificata di Shor erediterebbe.

Riproducibilità. Tutti i punti sono ottenuti con seed fissato e 200 000 campioni per configurazione; gli intervalli mostrati sono le deviazioni standard binomiali. Lo scaling $\text{rounds} = 2d$ e distanze maggiori sono estensioni naturali, limitate dal costo di decodifica (il piano dei rischi prevede di partire da $d = 3$ e salire solo se sostenibile); l'implementazione con Stim rende però il campionamento a queste distanze poco oneroso.

12.5 Oltre il Modello di Pauli: Errori Coerenti e Leakage

La stima del logical error rate appena presentata poggia su un'assunzione che va resa esplicita, perché ne delimita la validità: Stim è un simulatore a **stabilizzatori**, efficiente proprio perché i gate sono Clifford e gli errori sono Pauli casuali (X , Y , Z) con probabilità assegnate. È il modello standard delle grandi campagne di soglia, ma l'hardware reale non commette necessariamente errori di Pauli. Un lavoro recente [48] argomenta in modo sistematico che questa semplificazione può rendere le stime *ottimistiche*, e che una parte della fisica rilevante — errori coerenti, leakage, correlazioni temporali — va conservata nella simulazione per non certificare un'astrazione al posto del dispositivo.

Due sono le sorgenti di errore non-Pauli più importanti. La prima è l'**errore coerente**: un'imperfezione di calibrazione produce una piccola sovrarotazione sistematica $U = e^{-i\epsilon H}$, uguale a ogni ripetizione del gate. A differenza di un errore stocastico, le sovrarotazioni si sommano *in ampiezza*, non in probabilità. La seconda è il **leakage**: un transmon non è un sistema a due livelli e può popolare lo stato $|2\rangle$, un grado di libertà che un modello Pauli su due livelli non può nemmeno rappresentare.

Una verifica diretta su scala trattabile. Il meccanismo alla base del divario si dimostra con un esperimento elementare, alla portata della simulazione *full-state*

(Figura 12.2): un singolo qubit soggetto a N operazioni rumorose ripetute, con la stessa probabilità di errore per operazione $p = 10^{-2}$, modellata in due modi — come errore di Pauli (X con probabilità p) e come errore coerente (sovrarotazione $RX(\theta)$ con $\sin^2(\theta/2) = p$). L'errore di Pauli si accumula *linearmente* in N ; quello coerente *quadraticamente*, perché gli angoli si sommano ($\sin^2(N\theta/2)$). La simulazione (Qiskit Aer) coincide con la previsione analitica e mostra un divario netto: già a $N = 8$ operazioni l'errore coerente è circa 7 volte quello di Pauli. Un modello che trattasse le sovrarotazioni come errori Pauli equivalenti sottostimerebbe l'errore reale esattamente di questo fattore.

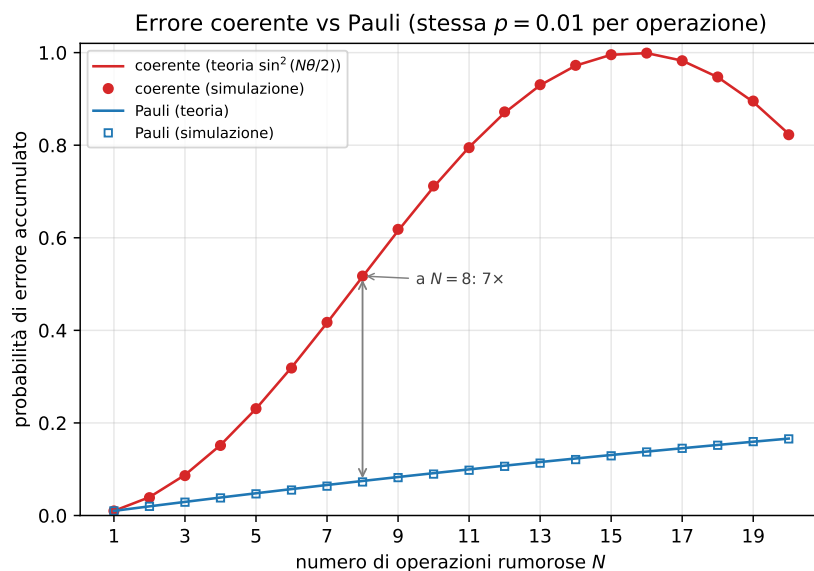


Figura 12.2: Accumulo dell'errore su un singolo qubit al crescere del numero N di operazioni rumorose, a parità di probabilità di errore per operazione ($p = 10^{-2}$). L'errore coerente (sovrarotazione, in rosso) cresce quadraticamente e raggiunge il regime di errore quasi certo intorno a $N = 16$; l'errore di Pauli (in blu) cresce linearmente. La simulazione *full-state* (marcatori) coincide con la teoria (linee). A $N = 8$ il divario è di circa un fattore 7: il modello di Pauli è *ottimistico*. Rigenerabile da `figure_src/gen_coherent.py`.

Implicazioni per la stima di soglia. Su un codice completo il divario si amplifica. Lo studio citato riporta, su esperimenti didattici controllati, che il modello Clifford-Pauli stima un errore logico dell'1.7% contro il 23.5% della simulazione completa in presenza di leakage, e del 18.1% contro il 38.7% in presenza di errori coerenti; su un modello di gate CZ fra transmon con surface code a distanza 9, l'errore logico reale può risultare da 25 a 56 volte superiore alla stima Clifford-Pauli, con la

soglia sovrastimata di circa il 20% [48]. La direzione metodologica che ne emerge è l'approccio *channel-first*: descrivere il rumore fisico una sola volta come canale quantistico completo (operatori di Kraus, equazione di Lindblad) e scegliere di volta in volta la rappresentazione di simulazione più adatta, conservando gli effetti non-Pauli dove contano. La soglia cessa così di essere un singolo numero e diventa una *superficie* nello spazio dei parametri fisici del dispositivo (T_1 , T_2 , durata di gate, leakage, crosstalk), che indica quale imperfezione limiti davvero la macchina.

Per questa tesi la conseguenza è duplice e va dichiarata: i valori di soglia e di p_L qui riportati vanno intesi come **baseline ottimistica** valida entro il modello Pauli, e il loro raffinamento verso un modello di rumore fisico più ricco — nella linea del Capitolo 13 e degli sviluppi futuri — è la naturale continuazione del lavoro.

Capitolo 13

Integrazione: lo Shor Logico e i Requisiti di Correzione

I Capitoli 11 e 12 hanno costruito e misurato la correzione d'errore su circuiti dedicati; il Capitolo 10 aveva misurato quanto il rumore fisico degrada Shor. Questo capitolo chiude il cerchio unendo i due fili: che cosa succederebbe all'algoritmo di Shor se i suoi qubit fossero *logici*, protetti da un codice correttore? La tentazione naturale — codificare ogni qubit di Shor con Steane o surface code e simulare tutto esplicitamente — non è percorribile, e la ragione è strutturale, non solo pratica. La soluzione metodologicamente corretta, prescritta dal documento di indirizzo del lavoro, è un'**integrazione a livelli**: caratterizzare separatamente Shor rumoroso e correzione d'errore, e connetterli attraverso un'unica grandezza di scambio — il logical error rate p_L . Il prodotto finale è la risposta a una domanda scientifica precisa: *quale livello di errore logico è necessario affinché Shor mantenga una probabilità di successo accettabile?*

13.1 L'Architettura a Quattro Livelli

Il percorso sperimentale complessivo della tesi si organizza in quattro livelli di simulazione, ciascuno con il proprio strumento e la propria domanda:

Livello	Oggetto	Strumento	Dove nella tesi
1	Shor ideale	Qiskit Aer (statevector)	Capp. 9–10
2	Shor rumoroso	Qiskit Aer + noise model	Cap. 10, App. A
3	QEC isolata	Qiskit (repetition, Steane); Stim + PyMatching (surface)	Capp. 11–12
4	Shor logico	Qiskit Aer + errore logico p_L	questo capitolo

Tabella 13.1: L'integrazione a livelli: ogni livello isola una domanda. Il livello 4 connette i precedenti: l'errore logico p_L , misurato al livello 3, sostituisce il rumore fisico del livello 2 nel circuito del livello 1.

Perché la simulazione completa non è percorribile. Due barriere si sommano. La prima è dimensionale: codificare i 12 qubit di Shor per $N = 15$ con un surface code di distanza 3 richiederebbe centinaia di qubit fisici simulati, ben oltre il limite statevector già discusso nel Capitolo 7. La seconda è strutturale: Stim, che ai livelli di qubit richiesti sarebbe l'unico strumento efficiente, simula solo circuiti *stabilizer* — e Shor non lo è. L'esponenziazione modulare e le rotazioni della QFT sono operazioni non-Clifford, che nessun simulatore stabilizer può trattare senza perdere l'efficienza garantita dal teorema di Gottesman-Knill. Presentare Stim come simulatore di uno Shor completo sarebbe un errore metodologico; la divisione dei compiti tra Qiskit Aer (circuiti generali piccoli) e Stim/PyMatching (QEC stabilizer) è l'unica configurazione corretta, ed è quella adottata.

13.2 Il Modello di Shor Logico

L'idea del livello 4 è sostituire il rumore fisico con il suo residuo dopo la correzione. In un'esecuzione fault-tolerant reale, ogni operazione logica — gate su qubit codificati, cicli di sindrome, misura logica — lascia una probabilità residua p_L di errore logico non corretto. Il modello di Shor logico rappresenta questo residuo direttamente: il circuito di Shor del Capitolo 9 viene eseguito con un canale d'errore di probabilità p_L applicato dopo ogni gate logico (o, in una variante a granularità maggiore, dopo ogni blocco computazionale), con p_L trattato come parametro libero nell'intervallo stimato sperimentalmente al Capitolo 12.

Il modello è dichiaratamente un'approssimazione: assume errori logici indipendenti e uniformi tra i gate, trascura le correlazioni residue del decoder e l'overhead dei gate logici non trasversali. La sua funzione non è riprodurre fedelmente una macchina fault-tolerant, ma porre in modo quantitativo la domanda di ricerca conclusiva:

quale valore di p_L è necessario affinché la probabilità di successo di Shor superi una soglia prefissata — ad esempio l'80%?

La risposta, combinata con le curve $p \rightarrow p_L$ del surface code, si traduce in un requisito sull'hardware fisico: dato il tasso d'errore fisico p di una QPU, quale distanza di codice — e quindi quanti qubit fisici — servono perché quella QPU esegua Shor in modo affidabile. Il confronto con il livello 2 (lo stesso circuito sotto rumore fisico diretto, dove la probabilità di successo crolla al 3.4% già per $N = 15$) quantifica il beneficio della correzione, e il **break-even** — il regime in cui il qubit logico diventa più affidabile del qubit fisico equivalente — fornisce il criterio sintetico di valutazione.

13.3 Il Contesto: le Stime di Risorse per Shor Fault-Tolerant

Per collocare i risultati didattici nella giusta scala, il capitolo si chiude sul confronto con le stime di risorse della letteratura. Gidney ed Ekerå hanno stimato nel 2021 che fattorizzare un modulo RSA a 2048 bit richiederebbe circa 20 milioni di qubit fisici rumorosi per 8 ore di calcolo, sotto assunzioni fisiche esplicite — con il costo dominato non dall'algoritmo ma dall'overhead di correzione: cicli di sindrome, routing e distillazione di stati magici [49]. È doverosa un'avvertenza metodologica: **questi numeri non si ottengono estrapolando i risultati con $N = 15$** . Lo Shor a 12 qubit di questa tesi è un esperimento dimostrativo, mentre le stime di risorse sono analisi architetturali complete — che includono compilazione fault-tolerant, layout dei qubit, distillazione e schedulazione — non commensurabili con una singola istanza simulata. Il valore del confronto è qualitativo: chiarisce perché Shor dimostrativo e Shor crittograficamente rilevante siano problemi radicalmente diversi, e quale grandezza li connetta concettualmente — il costo, in qubit e cicli, di portare p_L al livello richiesto.

In prospettiva futura, va segnalato che queste stime sono in rapida evoluzione: un lavoro più recente di Gidney [50] riduce il fabbisogno sotto il milione di qubit fisici grazie a nuove tecniche di aritmetica modulare, di codifica e di preparazione degli stati magici. La direzione della ricerca — meno qubit a parità di affidabilità — conferma che il collo di bottiglia non è l'algoritmo di Shor in sé, ma l'efficienza dell'infrastruttura di correzione d'errore che lo sostiene: esattamente il tema di questa parte della tesi.

13.4 Risultati del Confronto

Come l'errore logico è trasferito al circuito. Il trasferimento del *logical error rate* stimato ai Capitoli 11 e 12 avviene tramite un **unico canale di rumore logico**: dopo ogni gate del circuito di Shor si applica un canale depolarizzante indipendente di intensità p_L , dove p_L è il tasso di errore logico residuo che il codice scelto (Steane, oppure surface a una data distanza) lascia sul singolo qubit logico. Va dichiarato con precisione il perimetro di questa scelta: si tratta di un **modello efficace**, non di una simulazione fault-tolerant completa dell'algoritmo. Non si codificano esplicitamente i qubit di Shor né si simulano i cicli di estrazione della sindrome sul circuito logico; si assume invece che l'effetto netto della correzione sia riassumibile in un tasso di errore per operazione logica, indipendente e uniforme tra i gate. È l'approssimazione “a livelli” del documento di indirizzo (§11.3): permette di porre la domanda quantitativa senza il costo, intrattabile, della simulazione fault-tolerant esplicita, al prezzo di trascurare correlazioni e non-idealità che la Sezione 13.4.1 discute.

Concretamente, il modello è stato realizzato iniettando il canale depolarizzante di intensità p_L dopo ogni gate del circuito di Shor per $N = 15$ ($a = 7$, periodo $r = 4$) e misurando la probabilità di successo come frazione di misure che ricostruiscono i fattori corretti (16 384 campioni per punto, seed fissato). La Figura 13.1 è la figura conclusiva della tesi: mostra P_{success} in funzione dell'errore logico per gate p_L , con sovrapposti i tre regimi di correzione.

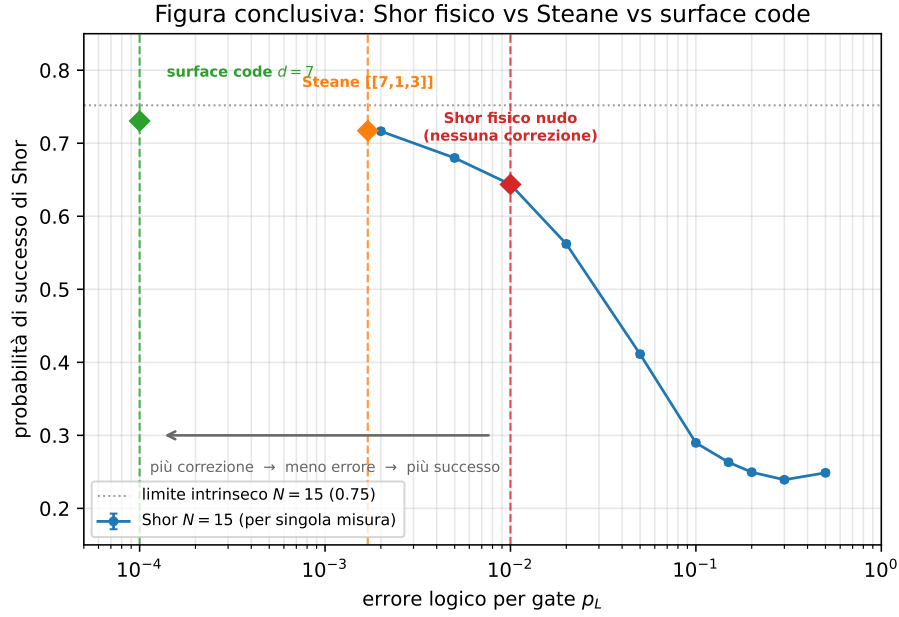


Figura 13.1: Probabilità di successo di Shor ($N = 15$) in funzione dell'errore logico per gate p_L . La curva scende in modo monotono dalla probabilità di successo per singola misura (≈ 0.75 a $p_L \rightarrow 0$, base $a = 7$) fino al livello di successo casuale (≈ 0.25). I tre marcatori collocano i regimi di correzione (errore logico residuo dai Capitoli 11 e 12): *Shor fisico nudo* ($p_L \approx 10^{-2}$, nessuna correzione), *codice di Steane* ($p_L \approx 1.7 \times 10^{-3}$) e *surface code $d = 7$* ($p_L \approx 10^{-4}$). Muovendosi verso sinistra — cioè aumentando la capacità di correzione — l'errore logico cala e P_{success} risale verso il limite. Curva rigenerabile da `figure_src/gen_shor_logico.py`.

La curva è monotona e ha due asintoti: a $p_L \rightarrow 0$ (correzione ideale) $P_{\text{success}} \approx 0.75$, a p_L elevato satura a ≈ 0.25 , il livello di successo puramente casuale su un istogramma ormai piatto. Tra i due estremi la correzione compie il suo lavoro: passando dal regime fisico non corretto ($p_L \approx 10^{-2}$, $P_{\text{success}} \approx 0.64$) al codice di Steane ($p_L \approx 1.7 \times 10^{-3}$, $P_{\text{success}} \approx 0.72$) e al surface code a distanza crescente ($p_L \approx 10^{-4}$, $P_{\text{success}} \approx 0.74$), la probabilità di successo risale progressivamente. Questo è, in un solo grafico, il messaggio dell'intera parte sperimentale: la correzione d'errore trasforma un calcolo dominato dal rumore in un calcolo affidabile, e ogni salto di capacità correttiva (fisico $\rightarrow$ Steane $\rightarrow$ surface) si traduce in un guadagno misurabile.

Che cosa misura, e che cosa non misura, il valore 0.75. Il valore $P_{\text{success}} \approx 0.75$ a correzione ideale va interpretato con cautela: *non* è un limite intrinseco e invalicabile dell'algoritmo, ma la probabilità di successo di una **singola misura** con la specifica scelta della base $a = 7$. Occorre distinguere grandezze diverse che il

termine “successo” confonde: la probabilità di ricostruire *esattamente* il periodo r dalla fase misurata; la probabilità di ottenere *un fattore* (che è maggiore, perché il post-processing con gcd recupera i fattori anche da alcune stime parziali del periodo); la dipendenza dalla base a scelta; e, soprattutto, la differenza tra la probabilità di *una singola misura* e la probabilità *cumulativa* dopo più esecuzioni. L’algoritmo di Shor è per sua natura probabilistico e si esegue ripetutamente: per esecuzioni indipendenti la probabilità di successo cumulativa dopo k tentativi è

$$P(k) = 1 - (1 - P_s)^k, \quad (13.1)$$

dove P_s è la probabilità per singolo run. La Figura 13.2 mostra che una P_s inferiore all’80% non impedisce affatto di superare tale soglia con poche ripetizioni: con $P_s \approx 0.75$ bastano $k = 2$ esecuzioni per raggiungere il 94%, e anche partendo da un regime rumoroso ($P_s = 0.40$) sono sufficienti quattro tentativi. La soglia dell’80% va dunque letta come obiettivo *cumulativo*, non come requisito sul singolo run.

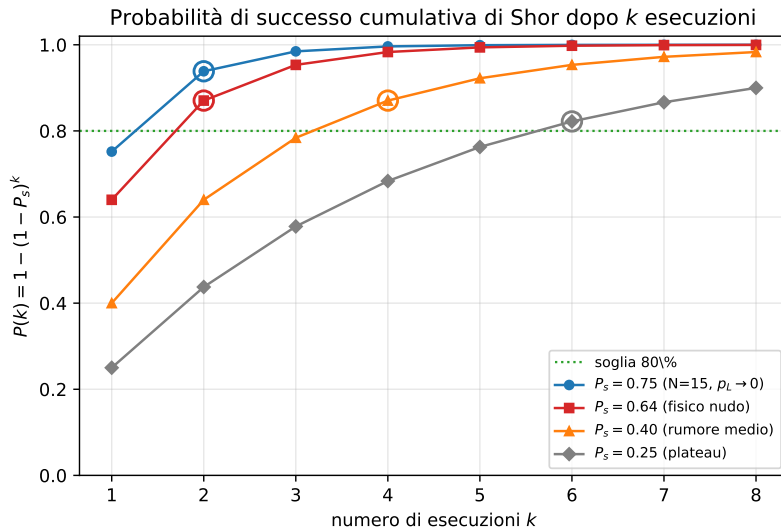


Figura 13.2: Probabilità di successo cumulativa $P(k) = 1 - (1 - P_s)^k$ (Eq. 13.1) in funzione del numero di esecuzioni indipendenti k , per diversi valori della probabilità per singolo run P_s . Il cerchio evidenzia il primo k che supera la soglia dell’80%: con $P_s \approx 0.75$ (correzione ideale su $N = 15$) sono sufficienti $k = 2$ tentativi; anche a $P_s = 0.40$ ne bastano quattro. Una probabilità per singolo run inferiore all’80% non impedisce di superare quella soglia in modo cumulativo.

La barriera di scalabilità. Il modello logico è stato applicato a $N = 15$ perché l’estensione a istanze maggiori incontra un limite pratico netto, esplorato seguendo

l'indicazione di procedere per N crescenti finché la simulazione resta stabile. Già per $N = 21$ il circuito, dopo la transpilazione della moltiplicazione modulare (decomposizione di Beauregard), raggiunge una profondità di circa 9500 porte su 22 qubit: la simulazione classica, anche in assenza di rumore, non completa una singola esecuzione in tempi ragionevoli. È la stessa barriera di scalabilità documentata nel Capitolo 10 per gli use case UC3/UC4, e coincide con l'avvertenza del documento di indirizzo (istanze piccole, $N = 21$ e 35 solo “se gestibili”). La finestra di N simulabili si esaurisce dunque a $N = 15$: è precisamente questo limite a motivare, sul piano metodologico, il ricorso alle stime di risorse della Sezione 13.3 anziché alla simulazione diretta per proiettare il costo su scala crittografica.

13.4.1 Discussione

La domanda di ricerca conclusiva — quale p_L è necessario affinché Shor mantenga una probabilità di successo elevata — riceve una risposta articolata. Nel regime sotto-soglia il surface code porta l'errore logico a $p_L \approx 10^{-4}$, sufficiente a riportare P_{success} (per singola misura) al valore massimo dell'istanza; è il codice di Steane, con $p_L \approx 10^{-3}$, a rappresentare il gradino intermedio. Quanto alla soglia dell'80% presa a riferimento dal documento di indirizzo, va letta correttamente come obiettivo *cumulativo*: per $N = 15$ e base $a = 7$ la probabilità per singolo run a correzione ideale è ≈ 0.75 , ma — come mostrato dall'Eq. 13.1 e dalla Figura 13.2 — due sole esecuzioni portano la probabilità cumulativa al 94%. Il valore 0.75 non è quindi un tetto invalicabile, ma il riflesso della natura probabilistica di Shor e della particolare scelta di a ; una base diversa, o semplicemente qualche ripetizione, spostano l'asticella. Il criterio che sintetizza il capitolo è dunque il **break-even** relativo: ogni salto di capacità correttiva avvicina P_{success} al massimo dell'istanza, e la gerarchia fisico $<$ Steane $<$ surface è netta e riproducibile.

Il bilancio dell'intero percorso. Questo risultato chiude un arco coerente. La prima campagna sperimentale (Capitolo 10) aveva stabilito, tramite ablazione, che l'apprendimento automatico applicato *a valle* dell'esecuzione non aggiunge nulla: il guadagno era interamente della ricerca multi-candidato TOP-K. Quel risultato negativo, lungi dall'essere un vicolo cieco, indicava dove il problema andava affrontato — non nel post-processing dell'output, ma nella *struttura* del calcolo. La seconda parte ha percorso questa strada: la correzione d'errore quantistica codifica l'informazione in modo che l'errore possa essere rilevato e corretto prima che si propaghi, dal

codice a ripetizione (che introduce la sindrome) al codice di Steane (che corregge un errore arbitrario) al surface code (che scala verso il regime fault-tolerant). La figura conclusiva riunisce le due parti: mostra quanto della fragilità di Shor sul rumore fisico venga recuperato passando ai qubit logici, e quantifica il prezzo — in distanza di codice, e quindi in qubit fisici — di quel recupero.

Limiti e perimetro. I risultati vanno letti entro il perimetro dichiarato. Il modello logico è un'approssimazione: assume errori logici indipendenti e uniformi tra i gate, trascura le correlazioni residue del decoder, l'overhead dei gate logici non trasversali e la distillazione degli stati magici necessari alle rotazioni non-Clifford. Le stime di errore logico di Steane e surface provengono da modelli di rumore non identici (depolarizzazione per qubit nel primo, *circuit-level* nel secondo) e vanno intese come ordini di grandezza, non come misure direttamente commensurabili. Infine, l'estrazione di sindrome implementata non è fault-tolerant, e l'istanza $N = 15$ è didattica. Ciò che i risultati dimostrano non è la fattibilità di Shor su scala crittografica — che le stime di risorse [49] collocano a milioni di qubit fisici — ma la *relazione* quantitativa tra capacità di correzione ed affidabilità del calcolo, verificata end-to-end su un'istanza trattabile: è questa relazione, e non il singolo numero fattorizzato, il contributo del lavoro.

Capitolo 14

Sviluppi Futuri

Ogni lavoro sperimentale su una tecnologia emergente porta con sé, oltre ai risultati ottenuti, una mappa di direzioni non esplorate: vincoli di tempo, risorse e maturità tecnologica che delimitano il perimetro di ciò che è stato fatto. I Capitoli 11–13 hanno portato la tesi dalla verifica del rumore alla correzione d’errore quantistica e allo Shor logico. Questo capitolo raccoglie le direzioni che stanno *oltre* quel perimetro: le estensioni più naturali del lavoro svolto, le domande che i risultati aprono senza rispondere e le opportunità che lo stato dell’arte attuale rende oggi accessibili.

Le direzioni presentate sono ordinate per urgenza e vicinanza ai risultati ottenuti: si parte dall’estensione più immediata e realizzabile — la scalabilità a istanze maggiori tramite decomposizioni efficienti — per passare alla validazione su hardware reale, alla generalizzazione ad altri algoritmi e all’integrazione con la QEC strutturale, concludendo con l’evoluzione del classificatore ML, il cui contributo è risultato secondario rispetto alla strategia di ricerca TOP-K.

I risultati sperimentali della tesi evidenziano tre aperture concrete: (i) la barriera di scalabilità per $N \geq 21$ è interamente attribuibile alla decomposizione **UnitaryGate** ($> 10\,000$ porte CX), risolvibile con circuiti efficienti già disponibili in letteratura; (ii) il simulation gap tra il simulatore statevector e l’hardware reale rimane da quantificare sperimentalmente; (iii) l’analisi di ablazione ha dimostrato che il classificatore ML applicato all’output è irrilevante — il vantaggio $\rho = 6.4$ su UC1 è interamente attribuibile alla strategia TOP-K — apertura che ha trovato la sua risposta strutturale nel nucleo QEC della tesi (Capitoli 11–13).

14.1 Scalabilità a Istanze di Maggiore Dimensione

I quattro use case di questa tesi utilizzano numeri N di piccola dimensione. Questa scelta è dettata dai limiti computazionali del simulatore: la simulazione statevector di un circuito di Shor richiede una memoria proporzionale a $2^{n_{\text{tot}}}$, dove $n_{\text{tot}} = n_{\text{count}} + \lceil \log_2 N \rceil$ è il numero totale di qubit. Per $N = 15$ con $n_{\text{count}} = 8$, si hanno 12 qubit e $2^{12} = 4096$ ampiezze complesse — perfettamente gestibile. Per $N = 2048$ bit (rilevante crittograficamente), servirebbero ~ 4000 qubit logici, del tutto fuori dalla portata sia dei simulatori che dell’hardware attuale.

Il principale collo di bottiglia identificato dagli esperimenti non è il numero di qubit ma la **profondità del circuito**: la decomposizione tramite `UnitaryGate` genera 10 040 porte CX per $N = 21$ e 13 064 per $N = 35$, rendendo $P_{\text{surv}} \approx 10^{-44}$ e 10^{-57} rispettivamente — valori per cui entrambi i metodi di analisi falliscono sistematicamente. Questo limite non è fondamentale: è un artefatto della decomposizione scelta.

La soluzione disponibile in letteratura è l’implementazione di Beauregard [28], che riduce il circuito di moltiplicazione modulare a $O(n^3)$ porte elementari usando registri ausiliari e l’addizionatore di Draper. Per $N = 21$ ($n = 5$ qubit), questo produrrebbe $O(125)$ porte invece di 10 040: una riduzione di due ordini di grandezza che renderebbe fattibile la simulazione anche sotto rumore NISQ-realistico. L’adozione di questa decomposizione consentirebbe di estendere il confronto $M1$ vs M_{TOP4} a UC3 e UC4, e di verificare se il vantaggio $\rho \approx 6$ osservato per UC1 scala con la dimensione del problema.

La decomposizione Beauregard è stata implementata nel modulo `beauregard.py` sviluppato durante questa tesi, riducendo il circuito per $N = 21$ da 10 040 a circa 2 594 porte CX ($P_{\text{surv}} \approx 6\%$ con $\epsilon_{2q} = 0.001$). Tuttavia, la validazione sperimentale completa non rientra nel perimetro di questa tesi per vincoli di risorse computazionali: la simulazione di 20 qubit con ~ 2800 porte CX rimane intensiva anche con il simulatore MPS (*Matrix Product States*) di Qiskit Aer su hardware consumer, richiedendo ore per singola run.

La direzione di sviluppo immediata è quindi la validazione sperimentale di UC3 e UC4 attraverso una delle seguenti modalità: (i) **IBM Quantum** — hardware reale con ≥ 20 qubit, accessibile gratuitamente per accademici, che esegue 1 024 shot in ~ 60 secondi; (ii) **cluster HPC universitario** — lo stesso codice `run_top4_baseline.py` può essere inviato come job batch senza modifiche; (iii) **analisi teorica estesa** — se le risorse non sono disponibili, il contributo di Beauregard può essere quantificato

analiticamente tramite P_{surv} e confrontato con la letteratura. In tutti i casi, l'obiettivo è verificare se il vantaggio $\rho \approx 6$ osservato per $N = 15$ si mantiene a profondità circuitale maggiore.

14.2 Test e Validazione su Hardware Quantistico Reale

Il sistema sperimentale sviluppato in questa tesi opera interamente su simulatore. Questa scelta garantisce riproducibilità, controllo preciso dei parametri di rumore e accesso illimitato — tutti requisiti indispensabili per la generazione del dataset di addestramento e per l'esecuzione sistematica dei quattro use case. Tuttavia, la simulazione è per definizione un'approssimazione: il noise model adottato replica il comportamento macroscopico del processore IBM `ibm_marrakesh` attraverso canali di rumore indipendenti e stazionari, mentre il rumore su hardware reale è correlato spazialmente tra qubit vicini, non stazionario nel tempo e affetto da derive lente dei parametri fisici non catturate da un modello statico.

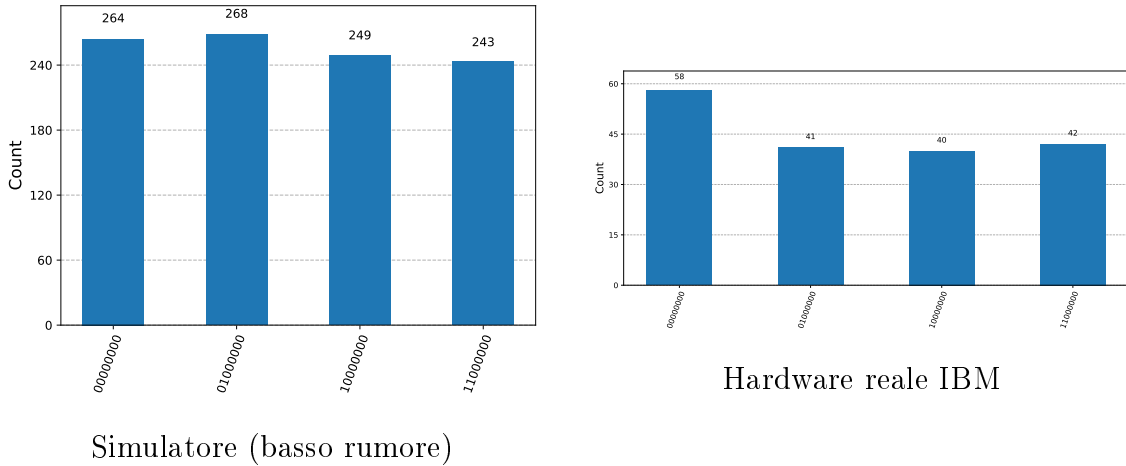


Figura 14.1: Confronto degli istogrammi QPE per $N = 15$, $a = 7$ su simulatore (sinistra) e su hardware IBM Quantum reale (destra). Sul simulatore i quattro picchi corretti $\{0, 64, 128, 192\}$ raccolgono ≈ 250 conteggi ciascuno su 1024 shot; sull'hardware reale gli stessi picchi risultano significativamente più deboli (≈ 40 – 58 conteggi), con la maggior parte dei conteggi dispersi uniformemente. Questo *simulation gap* quantifica la distanza tra il modello di rumore statico adottato e il rumore reale correlato e non stazionario del processore fisico.

Il passo più immediato è quindi l'esecuzione di UC1 e UC2 su hardware reale attraverso IBM Quantum, verificando in che misura le metriche ottenute su simu-

latore — in particolare il fattore di riduzione $\rho = \bar{M}_1/\bar{M}_{\text{TOP4}}$ — si trasferiscono all’hardware fisico. Poiché l’analisi di ablazione ha dimostrato che M_{TOP4} non richiede addestramento, questa validazione è immediata: basta eseguire la strategia TOP-K sui risultati reali senza alcuna fase di fine-tuning. Ci si attende che M_{TOP4} mostri la stessa robustezza osservata su simulatore, dato che la sua efficacia non dipende dal profilo specifico del rumore ma dalla struttura dei picchi QPE. Questa differenza è di per sé un risultato scientifico interessante: quantifica il *simulation gap*, ossia la distanza tra il comportamento del simulatore e quello dell’hardware reale.

14.3 Generalizzazione ad Altri Algoritmi Quantistici NISQ

La strategia multi-candidato TOP-K sviluppata in questa tesi è addestrata specificamente sugli output dell’algoritmo di Shor. Tuttavia, la struttura del problema è più generale: dato un algoritmo quantistico che produce, in assenza di rumore, una distribuzione di output con picchi netti, il rumore degrada quei picchi allargandoli e aggiungendo un fondo uniforme. La strategia TOP-K — che sfrutta la struttura residua dei picchi senza modificare il circuito né il profilo di errore — può in principio essere applicata a qualunque algoritmo con quella struttura.

I candidati più naturali per una generalizzazione sono:

Algoritmo di Grover. Come descritto nel Capitolo 3, Grover concentra l’ampiezza sullo stato soluzione tramite *amplitude amplification*. In presenza di rumore, l’ampiezza sullo stato obiettivo si riduce e la distribuzione si appiattisce progressivamente con il numero di iterazioni. La struttura del problema è analoga a quella di Shor: la distribuzione ideale ha un picco dominante, il rumore lo degrada. Una strategia TOP-K applicata a Grover potrebbe tentare i K candidati più frequenti invece del solo massimo, riducendo il numero di iterazioni necessarie.

Variational Quantum Eigensolver (VQE). Il VQE è uno degli algoritmi più rilevanti per l’era NISQ: stima il valore atteso di un Hamiltoniano usando circuiti parametrizzati e ottimizzazione classica. Il rumore introduce un bias sistematico nella stima dell’energia. Un modello di regressione addestrato a correggere questo bias — estensione diretta dell’approccio di [40] — potrebbe ridurre significativamente il numero di misurazioni necessarie per convergere all’energia dello stato fondamentale.

Quantum Approximate Optimization Algorithm (QAOA). Il QAOA è progettato per problemi di ottimizzazione combinatoria (Max-Cut, TSP). La funzione obiettivo è anch'essa un valore atteso misurato su un circuito quantistico, e il rumore la degrada in modo analogo al VQE. La tecnica TOP-K sviluppata per Shor potrebbe essere adattata per selezionare, dopo ogni valutazione della funzione obiettivo, i K candidati più promettenti invece del solo massimo, riducendo il numero di valutazioni necessarie.

14.4 Integrazione con la Quantum Error Correction Strutturale

La strategia M_{TOP4} sviluppata in questa tesi opera al livello degli *output*: sfrutta la struttura statistica dell'istogramma rumoroso per identificare il risultato corretto. La QEC strutturale — basata su codici come il surface code — opera invece al livello dei *qubit fisici*: rileva e corregge gli errori *durante* l'esecuzione, prima che si propagino.

Questi due approcci sono complementari, non alternativi, e il modello di Shor logico del Capitolo 13 fornisce già il banco di prova naturale per la loro combinazione: nello Shor logico, TOP-K non dovrebbe più compensare il rumore fisico dei gate — quello è compito del codice correttore — ma l'incertezza residua p_L derivante da errori oltre la soglia del codice, imprecisioni nella decodifica della sindrome o imperfezioni nella compilazione del circuito logico. Verificare sperimentalmente se la ricerca multi-candidato riduce le iterazioni *anche a livello logico* è un'estensione diretta e a basso costo del lavoro svolto.

Una seconda direzione, indicata esplicitamente dal relatore, riguarda la decodifica appresa: una rete neurale addestrata sul noise model realistico può **confermare o confutare la correzione proposta dal circuito ancillare** — e, in prospettiva, indicare quale tipologia di errore ha colpito l'esecuzione. È la linea dei decodificatori neurali, oggi una delle frontiere più attive della ricerca, come evidenziato dai risultati di AlphaQubit [43] e dei decoder transformer [51]: in quel contesto — a differenza del filtro a valle bocciato nel Capitolo 10 — l'apprendimento automatico attinge all'informazione strutturale della sindrome, che nell'output non c'è. Con l'avanzare dell'hardware verso processori capaci di surface code su istanze non banali (la roadmap IBM prevede questa capacità entro il 2026–2029 [18]), entrambe le direzioni diventano sperimentabili anche su dispositivi reali.

14.5 Evoluzione del Classificatore ML

L'analisi di ablazione ha dimostrato che il classificatore ML applicato all'output non contribuisce al miglioramento delle prestazioni: la variante M_{TOP4} senza classificatore è statisticamente equivalente a M2 per UC1 ($p = 0.849$) e significativamente migliore per UC2 ($p < 0.001$). La risposta strutturale a questo risultato negativo — correggere gli errori durante l'esecuzione anziché filtrarne gli effetti a valle — è il nucleo QEC della tesi (Capitoli 11–13), e il ruolo costruttivo dell'apprendimento automatico in quel contesto (la decodifica appresa) è discusso nella Sezione 14.4. Le seguenti direzioni per l'evoluzione del classificatore a valle rimangono comunque rilevanti in scenari in cui la struttura dei picchi QPE sia meno netta di quella osservata per $N = 15$, o in cui il costo per iterazione sia sufficientemente elevato da rendere vantaggioso un filtraggio accurato.

Classificatore adattivo al regime di rumore. Il risultato di UC2 ($\rho = 0.731 < 1$) ha evidenziato che, quando la probabilità di successo per iterazione del Metodo 1 è già alta ($p_{\text{iter}} \approx 41\%$), il filtraggio del classificatore ha un effetto netto negativo. Una soluzione naturale è un classificatore che stima direttamente p_{iter} dall'istogramma e decide dinamicamente la strategia ottimale: ricerca TOP-1 (equivalente al Metodo 1) quando p_{iter} stimata è alta, ricerca TOP-K quando p_{iter} stimata è bassa. Questo adattamento al regime richiederebbe un modello di regressione o un sistema a due stadi con una soglia di commutazione calibrata sul noise model corrente.

Calibrazione probabilistica dell'output. Anziché un'etichetta binaria secca, un classificatore calibrato produce una probabilità $p(\text{corretto} \mid \text{output})$. Questo consente di impostare una soglia adattiva: il loop si interrompe quando la probabilità stimata supera un livello di confidenza configurabile, bilanciando il rischio di falsi positivi con il costo delle iterazioni aggiuntive.

Transfer learning tra noise model diversi. Un classificatore addestrato su un noise model specifico (es. `ibm_marrakesh`) potrebbe essere adattato a un hardware diverso (es. `ibm_kyoto`) attraverso una fase di fine-tuning su un piccolo dataset del nuovo dispositivo, senza riaddestrare da zero. Questa tecnica, ispirata al transfer learning del deep learning classico, ridurrebbe significativamente il costo di portare il sistema su nuovi hardware.

Appendice A

Campagna Parametrica Dettagliata e Confronto ZNE

Questa appendice riporta integralmente la campagna sperimentale parametrica su M_{TOP4} e il confronto quantitativo con la *Zero-Noise Extrapolation*, di cui il Capitolo 10 presenta la sintesi. Per ciascun parametro si confrontano esplicitamente *previsione teorica* ed *esito osservato*. Tutte le misure usano $K_{\text{rep}} = 30$ ripetizioni e il test Mann-Whitney a una coda; la metodologia è a variazione singola (*one-factor-at-a-time*) rispetto alla configurazione di riferimento di UC1: $N = 15$, $a = 7$, $n_{\text{count}} = 8$, $\varepsilon_{1q} = 10^{-3}$, $\varepsilon_{2q} = 10^{-2}$, $T_1 = 100 \mu\text{s}$, $T_2 = 80 \mu\text{s}$, $p_{\text{ro}} = 2\%$, $M_{\text{shots}} = 1024$, $K = 4$, $M_{\text{max}} = 50$.

A.1 Variazione di K (numero di candidati per iterazione)

Previsione. Per $N = 15$ con $a = 7$, il periodo è $r = 4$ e le fasi QPE valide sono esattamente quattro: $\{0, 64, 128, 192\}$ su $2^8 = 256$ valori. La previsione era che $K = r = 4$ costituisse la soglia minima per catturare tutte le fasi valide, e che per $K < 4$ il tasso di successo degradasse proporzionalmente al numero di fasi escluse.

Esito. Sweep su $K \in \{1, 2, 3, 4, 6, 8\}$ con rumore fisso a UC1:

Tabella A.1: Effetto del parametro K su $\bar{M}$, tasso di successo e ρ vs M1 ($N = 15$, UC1, $K_{\text{rep}} = 30$)

K	P_{succ}	$\bar{M}$	σ_M	ρ vs M1	p-value (vs M1)
1 (= M1)	<i>baseline: $\bar{M}_1 = 6.43$, $P_{\text{succ}} = 76.7\%$</i>				
2	100%	1.00	0.00	6.430	0.003
3	100%	1.00	0.00	6.430	0.003
4 ($= M_{\text{TOP4}}$)	100%	1.00	0.00	6.435	<0.001
6	100%	1.00	0.00	6.430	0.003
8	100%	1.00	0.00	6.430	0.003

La soglia effettiva non è $K = r = 4$ ma $K = 2$: il salto netto è tra $K = 1$ (equivalente a M1) e $K = 2$ ($\bar{M} = 1.00$, $P_{\text{succ}} = 100\%$, $\rho = 6.43$); per $K \geq 2$ le prestazioni sono identiche e massime. La spiegazione è duplice: il rumore depolarizzante non distribuisce il segnale uniformemente sulle quattro fasi valide, ma tende a concentrarlo su uno o due valori dominanti dell'istogramma; inoltre `extract_factors` con `limit_denominator` è tollerante a fasi leggermente spostate, allargando il bacino di convergenza attorno a ciascun picco. Il criterio teorico $K = r$ resta un limite superiore garantito, ma nella pratica $K = 2$ è sufficiente per questa istanza; per r più grandi la soglia effettiva andrebbe verificata.

A.2 Variazione di ε_{2q} (tasso di errore a due qubit)

Previsione. ε_{2q} è il parametro di rumore dominante: $P_{\text{surv}} = (1 - \varepsilon_{2q})^{162}$. Ci si aspettava una **soglia critica** oltre la quale l'istogramma diventasse troppo piatto per identificare i candidati corretti, degradando $\bar{M}_{\text{TOP4}}$.

Esito. Sweep su tre ordini di grandezza, con P_{surv} da 85% a 3.87×10^{-8} :

Tabella A.2: Effetto di ε_{2q} su M_{TOP4} ($K = 4$, $K_{\text{rep}} = 30$, altri parametri fissi a UC1)

ε_{2q}	P_{surv}	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p-value	P_{succ}
10^{-3}	85.0%	2.27	1.00	2.273	<0.001	100%
5×10^{-3}	44.4%	3.18	1.00	3.182	<0.001	100%
10^{-2} (UC1)	19.7%	6.43	1.00	6.435	<0.001	100%
2×10^{-2}	3.8%	4.77	1.00	4.769	0.001	100%
5×10^{-2}	2.46×10^{-4}	5.92	1.00	5.920	0.001	100%
10^{-1}	3.87×10^{-8}	6.13	1.00	6.130	<0.001	100%

Nessuna soglia critica è stata rilevata: M_{TOP4} mantiene $\bar{M} = 1.00$ e $P_{\text{succ}} = 100\%$ su tutto il range, incluso $\varepsilon_{2q} = 10^{-1}$ dove $P_{\text{surv}} \approx 10^{-8}$. Il rumore depolarizzante, pur

abbassando la probabilità *media* di sopravvivenza del segnale, non produce istogrammi uniformemente piatti per ogni realizzazione (seed): in una frazione significativa delle esecuzioni uno o più dei quattro candidati più frequenti corrisponde comunque a una fase QPE valida. TOP-K sfrutta questa struttura residua: non richiede che il segnale sia forte in media, ma solo che sia recuperabile *in quella specifica esecuzione*. La variazione di ρ (da 2.27 a 6.43) riflette il peggioramento di $\bar{M}_1$, non un degrado di M_{TOP4} .

A.3 Variazione di M_{shots} (shot per iterazione)

Previsione. Con $n_{\text{count}} = 8$ qubit il registro ha $2^8 = 256$ celle: era attesa una soglia minima di $M_{\text{shots}} \geq 4 \times 256 = 1024$ campioni, sotto la quale l'istogramma sarebbe troppo sparso per identificare i picchi in modo affidabile.

Tabella A.3: Effetto di M_{shots} su M_{TOP4} ($K = 4$, $K_{\text{rep}} = 30$, UC1)

M_{shots}	P_{succ}	$\bar{M}$	σ_M	ρ vs M1
128 (min. testato)	100%	1.00	0.00	6.430
256	100%	1.00	0.00	6.430
512	100%	1.00	0.00	6.430
1024 (riferimento)	100%	1.00	0.00	6.435
2048	100%	1.00	0.00	6.430

Esito. La previsione è smentita: anche con 128 shot — otto volte meno della soglia prevista — M_{TOP4} raggiunge $\bar{M} = 1.00$ e $P_{\text{succ}} = 100\%$. Il campionamento uniforme di tutte le celle non è necessario perché i picchi QPE validi occupano poche posizioni fisse e ben separate: anche un istogramma sparso è sufficiente a individuarle. Il costo per iterazione è dunque fino a 8 volte inferiore al riferimento.

A.4 Analisi Congiunta $K \times \varepsilon_{2q}$

L'analisi congiunta verifica se esiste un'interazione tra K e il livello di rumore: in regime di rumore elevato, un K maggiore potrebbe compensare il degrado del picco QPE. Griglia $K \in \{2, 4, 6, 8\} \times \varepsilon_{2q} \in \{5 \times 10^{-3}, 10^{-2}, 2 \times 10^{-2}, 5 \times 10^{-2}\}$, $K_{\text{rep}} = 30$ per cella.

Tabella A.4: $\bar{M}_{\text{TOPK}}$ per combinazioni di K e ε_{2q} ($K_{\text{rep}} = 30$). In grassetto: $\rho > 2$ con $p < 0.05$.

K	ε_{2q}			
	5×10^{-3}	10^{-2} (UC1)	2×10^{-2}	5×10^{-2} (UC2)
2	1.00	1.00	1.00	1.00
4	1.00	1.00	1.00	1.00
6	1.00	1.00	1.00	1.00
8	1.00	1.00	1.00	1.00

L'interazione è assente: ogni cella produce $\bar{M} = 1.00$ con $P_{\text{succ}} = 100\%$. Non esiste un regime in cui un K più alto compensi un rumore più elevato, perché TOP-K non ne ha bisogno: anche a $\varepsilon_{2q} = 5 \times 10^{-2}$ il picco corretto rientra già tra i primi due candidati.

A.5 Parametri Secondari: T_1/T_2 , ε_{1q} e p_{ro}

I tempi di coerenza, il tasso di errore a singolo qubit e l'errore di lettura erano attesi come parametri secondari per $N = 15$: il tempo di gate totale ($\approx 8 \mu\text{s}$) è ben inferiore ai tempi di decoerenza tipici, le porte a singolo qubit sono molto meno numerose delle 162 CX, e p_{ro} produce solo un allargamento simmetrico dell'istogramma.

Tabella A.5: Effetto di T_1 su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$, $T_2 = 0.8 T_1$)

T_1 (μs)	T_2 (μs)	P_{succ}	$\bar{M}$	σ_M	ρ vs M1
20	16	100%	1.00	0.00	6.430
50	40	100%	1.00	0.00	6.430
100 (UC1)	80	100%	1.00	0.00	6.435
200	160	100%	1.00	0.00	6.430
500	400	100%	1.00	0.00	6.430

Tabella A.6: Effetto di ε_{1q} su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$)

ε_{1q}	P_{succ}	$\bar{M}$	σ_M	ρ vs M1
10^{-4}	100%	1.00	0.00	6.430
5×10^{-4}	100%	1.00	0.00	6.430
10^{-3} (UC1, riferimento)	100%	1.00	0.00	6.435
5×10^{-3}	100%	1.00	0.00	6.430
10^{-2}	100%	1.00	0.00	6.430
2×10^{-2}	100%	1.00	0.00	6.430

Tabella A.7: Effetto di p_{ro} su M_{TOP4} ($K = 4$, $\varepsilon_{2q} = 10^{-2}$, $K_{\text{rep}} = 30$)

p_{ro}	P_{succ}	$\bar{M}$	σ_M	ρ vs M1
0%	100%	1.00	0.00	6.430
1%	100%	1.00	0.00	6.430
2% (UC1)	100%	1.00	0.00	6.435
5%	100%	1.00	0.00	6.430
10%	100%	1.00	0.00	6.430
20%	100%	1.00	0.00	6.430

La previsione qualitativa (impatto marginale) è confermata, ma l'esito quantitativo è ancora più netto: l'impatto è *nullo* su tutto il range testato per tutti i parametri. T_1 varia da 20 a $500 \mu\text{s}$, ε_{1q} da 10^{-4} a 2×10^{-2} (lo stesso ordine di grandezza di ε_{2q}), p_{ro} da 0% a 20% (ben oltre i valori realistici $\lesssim 3\%$ dei dispositivi attuali): in nessun caso $\bar{M}_{\text{TOP4}}$ si discosta da 1.00. Un errore di lettura simmetrico allarga l'istogramma ma non sposta i picchi QPE; il basso numero di porte a singolo qubit rende ε_{1q} trascurabile; il tempo di gate complessivo è inferiore anche al minimo T_1 testato.

A.6 Dettaglio del Confronto con Zero-Noise Extrapolation

La ZNE [42] opera in due fasi. Nella prima, il circuito viene eseguito a più livelli di rumore amplificato (noise scaling): si moltiplicano i parametri di errore ε_{2q} e ε_{1q} per fattori $\lambda \in \{1, 2, 3, \dots\}$, ottenendo istogrammi progressivamente più degradati. Nella seconda, si applica la *Richardson extrapolation* per stimare l'istogramma al rumore ideale $\lambda = 0$:

- **ZNE-2** (estrapolazione lineare, 2 livelli): $P_0(k) = 2 P_{\lambda=1}(k) - P_{\lambda=2}(k)$
- **ZNE-3** (Richardson quadratica, 3 livelli): $P_0(k) = 3 P_{\lambda=1}(k) - 3 P_{\lambda=2}(k) + P_{\lambda=3}(k)$

L'istogramma estrapolato P_0 viene quindi usato per identificare i candidati più frequenti (TOP-1) e tentare l'estrazione dei fattori. Il costo per iterazione è $\lambda_{\text{max}} \times M_{\text{shots}}$: con ZNE-2, 2048 shot per iterazione; con ZNE-3, 3072 shot.

Il fallimento di ZNE su questo task (Tabella 10.6 nel Capitolo 10) è spiegabile strutturalmente. ZNE è progettata per stimare valori di aspettazione di osservabili

scalari (es. $\langle Z \rangle$), dove la relazione tra rumore e output è regolare e l'estrapolazione è stabile. Nel caso di Shor, l'output è una distribuzione di probabilità su $2^8 = 256$ stati: l'estrapolazione Richardson applicata componente per componente produce valori negativi in molte celle (fisicamente privi di significato), che vengono tagliati e rinormalizzati, distorcendo la struttura dei picchi QPE. Il risultato è un istogramma estrapolato *più piatto* di quello originale, non più nitido. M_{TOP4} segue la logica opposta: invece di correggere l'istogramma, accetta il rumore come dato di fatto e amplia la ricerca ai K candidati più probabili.

Bibliografia

- [1] Richard P. Feynman. “Simulating Physics with Computers”. In: *International Journal of Theoretical Physics* 21.6–7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.
- [2] David Deutsch. “Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer”. In: *Proceedings of the Royal Society of London. Series A* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070.
- [3] John S. Bell. “On the Einstein Podolsky Rosen Paradox”. In: *Physics* 1.3 (1964), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.
- [4] Alain Aspect, Philippe Grangier e Gérard Roger. “Experimental Realization of Einstein-Podolsky-Rosen-Bohm Gedankenexperiment: A New Violation of Bell’s Inequalities”. In: *Physical Review Letters* 49.2 (1982), pp. 91–94. DOI: 10.1103/PhysRevLett.49.91.
- [5] Brian D. Josephson. “Possible New Effects in Superconductive Tunnelling”. In: *Physics Letters* 1.7 (1962), pp. 251–253. DOI: 10.1016/0031-9163(62)91369-0.
- [6] Jens Koch et al. “Charge-insensitive Qubit Design Derived from the Cooper Pair Box”. In: *Physical Review A* 76.4 (2007), p. 042319. DOI: 10.1103/PhysRevA.76.042319.
- [7] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.
- [8] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)* (1996), pp. 212–219. DOI: 10.1145/237814.237866.

- [9] Christof Zalka. “Grover’s Quantum Searching Algorithm is Optimal”. In: *Physical Review A* 60.4 (1999), pp. 2746–2751. DOI: 10.1103/PhysRevA.60.2746.
- [10] Ronald L. Rivest, Adi Shamir e Leonard Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. In: *Communications of the ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.
- [11] Arjen K. Lenstra e Hendrik W. Lenstra Jr. “The Development of the Number Field Sieve”. In: *Lecture Notes in Mathematics*. Vol. 1554. Springer-Verlag, 1993. DOI: 10.1007/BFb0091537.
- [12] Fabrice Boudot et al. “Factoring RSA-250”. In: *Cryptology ePrint Archive* (2020). Report 2020/697, <https://eprint.iacr.org/2020/697>.
- [13] National Security Agency. *Commercial National Security Algorithm Suite 2.0 (CNSA 2.0)*. https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF. Cybersecurity Advisory, settembre 2022. Fissa le scadenze di migrazione verso algoritmi resistenti ai computer quantistici entro il 2030–2035. 2022.
- [14] National Institute of Standards and Technology. *Post-Quantum Cryptography Standards: FIPS 203, 204, 205*. <https://csrc.nist.gov/pubs/fips/203/final>. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA). Pubblicazione formale agosto 2024. 2024.
- [15] Apple Security Engineering and Architecture. *iMessage with PQ3: The new state of the art in quantum-secure messaging*. <https://security.apple.com/blog/imessage-pq3/>. Febbraio 2024. Primo protocollo di messaggistica con crittografia post-quantistica di livello 3, basato su ML-KEM per lo scambio di chiavi ibride. 2024.
- [16] William K. Wootters e Wojciech H. Zurek. “A Single Quantum Cannot be Cloned”. In: *Nature* 299 (1982), pp. 802–803. DOI: 10.1038/299802a0.
- [17] Frank Arute et al. “Quantum Supremacy Using a Programmable Superconducting Processor”. In: *Nature* 574 (2019), pp. 505–510. DOI: 10.1038/s41586-019-1666-5.
- [18] IBM Quantum. *IBM Quantum Roadmap 2025: Path to Fault-Tolerant Quantum Computing*. <https://www.ibm.com/roadmaps/quantum/2025/>. Processori Loon, Nighthawk e target Quantum Starling 2029. 2025.

- [19] John Preskill. “Quantum Computing in the NISQ Era and Beyond”. In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79.
- [20] Graychii. *Shor Algorithm Implementation*. <https://github.com/Graychii/Shor-Algorithm-Implementation>. Implementazione Python dell’algoritmo di Shor su Qiskit. 2023.
- [21] Ballinas-Play et al. “A Methodology to Select and Adjust Quantum Noise Models Through Emulators: Benchmarking Against Real Backends”. In: *EPJ Quantum Technology* 11 (2024). Metodologia sistematica per la selezione e calibrazione di noise model per emulatori rispetto a backend reali IBM. Benchmark su ibm_perth (7 qubit), p. 71. DOI: 10.1140/epjqt/s40507-024-00284-4.
- [22] Franz Franchetti et al. “Optimization and Performance Analysis of Shor’s Algorithm in Qiskit”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. Available at https://users.ece.cmu.edu/~franzf/papers/hpec_2023_Quantum_final.pdf. 2023.
- [23] IBM Quantum. *Shor’s Algorithm – IBM Quantum Documentation*. <https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm>. Tutorial ufficiale IBM per l’implementazione dell’algoritmo di Shor con Qiskit Runtime su hardware reale (ibm_marrakesh). Include Dynamical Decoupling e Gate Twirling come tecniche di soppressione del rumore. 2024.
- [24] Lieven M. K. Vandersypen et al. “Experimental Realization of Shor’s Quantum Factoring Algorithm Using Nuclear Magnetic Resonance”. In: *Nature* 414 (2001), pp. 883–887. DOI: 10.1038/414883a.
- [25] Unathi Skosana e Mark Tame. “Demonstration of Shor’s Factoring Algorithm for $N = 21$ on IBM Quantum Processors”. In: *Scientific Reports* 11 (2021), p. 16599. DOI: 10.1038/s41598-021-95973-w.
- [26] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2000. ISBN: 978-1107002173.
- [27] Wojciech Hubert Zurek. “Decoherence, einselection, and the quantum origins of the classical”. In: *Reviews of Modern Physics* 75.3 (2003), pp. 715–775. DOI: 10.1103/RevModPhys.75.715.

- [28] Stéphane Beauregard. “Circuit for Shor’s Algorithm Using $2n + 3$ Qubits”. In: *Quantum Information and Computation* 3.2 (2003). arXiv:quant-ph/0205095. Ottimizzazione del circuito originale di Shor: riduce il numero di qubit da $O(n)$ a $2n + 3$ tramite addizione modulare in-place, pp. 175–185.
- [29] Thomas Monz et al. “Realization of a Scalable Shor Algorithm”. In: *Science* 351.6277 (2016), pp. 1068–1070. DOI: 10.1126/science.aad9480.
- [30] Aditya Singh, Aakash Khochare e Subhadeep Palit. *Practical Challenges in Executing Shor’s Algorithm on Existing Quantum Platforms*. arXiv:2512.15330. 2025.
- [31] Oded Regev. “An Efficient Quantum Factoring Algorithm”. In: *arXiv preprint* (2023). arXiv:2308.06572.
- [32] Charles H. Bennett et al. “Strengths and Weaknesses of Quantum Computing”. In: *SIAM Journal on Computing* 26.5 (1997), pp. 1510–1523. DOI: 10.1137/S0097539796300933.
- [33] Dorit Aharonov e Michael Ben-Or. “Fault-Tolerant Quantum Computation with Constant Error Rate”. In: *SIAM Journal on Computing* 38.4 (2008). Versione preliminare presentata a STOC 1997; preprint arXiv:quant-ph/9906129, pp. 1207–1282. DOI: 10.1137/S0097539799359385.
- [34] Peter W. Shor. “Scheme for Reducing Decoherence in Quantum Computer Memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493.
- [35] Andrew M. Steane. “Error Correcting Codes in Quantum Theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: 10.1103/PhysRevLett.77.793.
- [36] Austin G. Fowler et al. “Surface Codes: Towards Practical Large-Scale Quantum Computation”. In: *Physical Review A* 86.3 (2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324.
- [37] Qiskit contributors. *Qiskit: An Open-source Framework for Quantum Computing*. <https://qiskit.org>. 2023. DOI: 10.5281/zenodo.2573505.
- [38] Google Quantum AI. *Cirq: A Python Framework for Writing, Manipulating, and Running Quantum Circuits*. <https://quantumai.google/cirq>. Framework open-source Google per processori superconduttori Sycamore. 2024.

- [39] Ville Bergholm et al. *PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations*. arXiv:1811.04968. Framework Xanadu orientato a circuiti variazionali e quantum machine learning. 2022.
- [40] Haoran Liao et al. “Machine Learning for Practical Quantum Error Mitigation”. In: *arXiv preprint* (2023). arXiv:2309.17368.
- [41] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [42] Kristan Temme, Sergey Bravyi e Jay M. Gambetta. “Error Mitigation for Short-Depth Quantum Circuits”. In: *Physical Review Letters* 119.18 (2017). Articolo fondativo della Zero-Noise Extrapolation (ZNE): propone l’amplificazione controllata del rumore seguita da estrapolazione a zero rumore per stimare l’output ideale di un circuito NISQ, p. 180509. DOI: 10.1103/PhysRevLett.119.180509.
- [43] Johannes Bausch et al. “Learning High-Accuracy Error Decoding for Quantum Processors”. In: *Nature* 635 (2024). AlphaQubit: decoder neurale sviluppato da Google DeepMind e Google Quantum AI. Supera i decoder analitici classici su hardware Sycamore a 241 qubit, pp. 834–840. DOI: 10.1038/s41586-024-08148-8.
- [44] Adriano Barenco et al. “Elementary gates for quantum computation”. In: *Physical Review A* 52.5 (1995). Dimostra che qualsiasi operatore unitario su k qubit può essere decomposto in $O(4^k)$ porte elementari; fondamento teorico per la complessità della decomposizione UnitaryGate, pp. 3457–3467. DOI: 10.1103/PhysRevA.52.3457.
- [45] Joschka Roffe. “Quantum error correction: an introductory guide”. In: *Contemporary Physics* 60.3 (2019), pp. 226–245. DOI: 10.1080/00107514.2019.1667078.
- [46] Oscar Higgott. *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*. arXiv:2105.13082. 2021. arXiv: 2105.13082.
- [47] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.
- [48] Raul Conchello Vendrell et al. *Plaquette: A hardware-aware design platform for fault-tolerant quantum computers*. arXiv:2607.08767. 2026. arXiv: 2607.08767.

- [49] Craig Gidney e Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (2021), p. 433. DOI: 10.22331/q-2021-04-15-433.
- [50] Craig Gidney. *How to factor 2048 bit RSA integers with less than a million noisy qubits*. arXiv:2505.15917. 2025. arXiv: 2505.15917.
- [51] Hannes Bausch, Andrew Senior et al. “Learning to Decode the Surface Code with a Recurrent, Transformer-Based Neural Network”. In: *arXiv preprint* (2023). arXiv:2310.05900. Decoder transformer per surface code; supera i decoder state-of-the-art su hardware Google Sycamore per distanze 3–11.

Ringraziamenti

Desidero ringraziare in primo luogo il mio relatore, **Ing. Floriano Caprio**, per la disponibilità, la pazienza e la guida preziosa fornita durante l'intero percorso di sviluppo di questo lavoro. I suoi consigli e la sua competenza sono stati fondamentali per orientarmi in un campo così vasto e in continua evoluzione.

Ringrazio l'**Università Campus Bio-Medico di Roma** per il percorso formativo offerto, che mi ha fornito gli strumenti teorici e pratici per affrontare questa tesi.

Un ringraziamento speciale va alla mia **famiglia**, che ha rappresentato un punto fermo e una fonte inesauribile di sostegno morale durante gli anni di studio.

Ringrazio infine tutti i **colleghi e amici** che hanno condiviso con me questo percorso, rendendo più leggere le giornate di studio e ricerca.

Claudio Dragotta

Roma, 2026